

Model checking for Self-Replicating Robotic Systems

Master's Thesis

Dmitry Babaytsev (124065)

1. Referee: Prof. Dr. Jan Oliver Ringert
2. Referee: PD Dr. Andreas Jakoby

Submission date: June 31, 2025



Declaration

Unless otherwise indicated in the text or references, this thesis is entirely the product of my own scholarly work.

Weimar, June 31, 2025

Dmitry Babaytsev (124065)

Abstract

Self-replicating robotic systems (SRRS) promise great potential for space exploration, in-situ construction, and resilient infrastructure, yet their critical importance and complex interactions make traditional testing insufficient to guarantee safety or correctness of such systems.

This initial research investigates how formal verification and model checking – can be integrated into the design of SRRS to provide provable guarantees of liveness, safety, and functional correctness.

We first survey existing verification techniques and classify state-of-the-art model checking algorithms with respect to their scalability and suitability for robotics. Building on this analysis, we propose a tool-supported workflow that automatically generates NuSMV models corresponding to specific stages of self-replication, based on the initial description of the system. The resulting models are checked against temporal logic specifications that encode key replication properties.

In order to validate the proposed approach, we develop a simulation of the system using Gazebo and ROS. An experimental testbed based on a modular robot simulation serves as the reference platform; we evaluate model-checking variants on representative replication scenarios, and try to use counterexample for control and decision-making in the system.

By linking robot self-replication simulation to rigorous formal analysis, the work provides a reproducible pathway for checking of next-generation self-replicating robots.

Contents

1	Introduction	1
2	Background	3
2.1	Formal verification overview	3
2.1.1	SAT solvers	4
2.1.2	SMT	6
2.2	Model checking	7
2.2.1	Classification of Model checking methods	8
2.3	Self-replicating robotic systems	15
2.3.1	Self-replication introduction	15
2.3.2	Classification of Self-replication systems	16
2.3.3	Self-replication models	21
2.3.4	Degree of self-replication	23
2.3.5	Implementations of kinematic self-replicating robotic systems .	24
3	Research questions	32
3.1	Analysis of model checking approaches in self-replicating systems do- main	32
3.2	Experimental checking	33
3.3	Limitations and challenges	33
4	Methodology	35
4.1	Research tools	35
4.2	Proposed approach	37
5	Implementation of model checking in SRRS	39
5.1	Subject description	39
5.2	Analysis of the system	42
5.3	Model checking based control in local space	45
5.3.1	Overview	45
5.3.2	Robot NuSMV model	46
5.3.3	Reachability specifications	51
5.3.4	Counterexample processing	53

5.3.5	Evaluation	57
5.4	Robot configuration model checking	58
5.4.1	Sequential checking of possible configurations	58
5.4.2	Implementation in Python	60
5.5	Aggregated configuration checking in NuSMV	63
5.6	Reachability model checking in global space	66
5.6.1	Overview	66
5.6.2	Evaluation	66
5.7	General System Model checking	66
5.7.1	Overview	66
5.7.2	Evaluation	66
5.8	Collective behavior model checking	66
5.8.1	Overview	66
6	Simulation of the SRRS	67
6.1	System Architecture	67
6.2	Gazebo simulation elements	70
6.2.1	Base model	70
6.2.2	Parts simulation and behavior	71
6.3	Robot simulation and control	73
6.3.1	Gazebo representation of a robot	73
6.3.2	Low level robot controller	76
6.4	Base and Part Interaction	78
6.5	Supervisory Control and GUI Integration	80
6.5.1	Mathematical Abstraction of Assembly Process	83
6.5.2	Implementation Challenges and Design Considerations	83
6.6	Simulation Workflow	84
6.6.1	Conclusion	84
7	Evaluation and validation	86
7.1	Validation of the counterexample based robot control	86
7.2	Validation of the robot configuration checking	89
7.3	Complexity evaluation	89
8	Conclusion	90
A	Appendix	91
A.1	NuSMV models	91
A.1.1	robot_structure.smv	91
A.2	Python scripts	95
A.2.1	Counterexample.py class	95

A.3	ROS2 nodes	98
A.3.1	RobotController class	98
A.3.2	PartController class	102
A.4	Gazebo sdf,urdf and xacro code	104
A.4.1	base.sdf	104
A.4.2	robot.xacro	104
A.4.3	part.xacro	110

1 Introduction

TODO: check grammar and content

A concept of self-replication, has steadily gained interest within the field of robotics as a potential paradigm for enabling autonomous construction, scalability, adaptability, and resilience. Self-replicating robotic systems (SRRS) possess the ability to autonomously build replicas of themselves using available components and resources. These systems hold significant promise for applications in remote environments such as planetary exploration, in-situ resource utilization, and disaster recovery, where manual intervention is limited or infeasible [Suthakorn2002SpaceUtilization] [Nasa1980Conference]

However, the complexity and autonomy involved in such systems pose considerable challenges to ensuring their correctness, safety, and reliability.

As SRRS evolve from theoretical constructs to practical experiments, guaranteeing their functional integrity becomes crucial—particularly given their recursive nature and potential to affect their environments in unanticipated ways. Traditional testing approaches are not enough when dealing with critical applications, vast state spaces and non-deterministic behavior inherent in self-replicating systems. In response to these challenges, formal verification methods, and in particular model checking, have emerged as powerful tools for rigorously analyzing system behaviors against formal specifications.

Formal methods offer a mathematically grounded approach to verifying that a system adheres to desired properties such as liveness, safety, and correctness.

This research explores the integration of formal verification into the design and analysis of self-replicating robotic systems. The goal is to bridge the gap between the physical nature of replication in robotics and the abstract frameworks used in model checking.

The structure of this work is as follows. Section 2 provides background on formal verification techniques, model checking strategies, and existing self-replicating robotic frameworks. Section 3 outlines the core research questions for the thesis. Section 4 describes the methodology and tools planned to use for robotic simulations, and presents an approach for experimental validation.

1 Introduction

Together, these sections aim to contribute a formal framework for analyzing self-replicating robotic systems, supporting their future deployment in safety-critical and autonomous environments.

2 Background

2.1 Formal verification overview

Formal Methods refers to mathematically rigorous techniques and tools for the specification, design and verification of software and hardware systems.

Expanding this definition Formal methods can be used for formal description of the system (specification) on some level of abstraction, program synthesis (design) according to specification and verification of a system to prove that model of system under consideration satisfies some properties.

Formal verification techniques can be thought of as comprising three parts [HuthR1]:

- a framework for modelling systems, typically a description language;
- a specification language for describing the properties to be verified;
- a verification method to establish whether the description of a system satisfies the specification.

These methods and techniques of verification include:

1. SAT solving - operate on propositional logic
2. SMT solving (and tools like Z3) extend this to first-order logic, integrating theories such as arithmetic or arrays.
3. Relational model finding (e.g., Alloy)
4. Model checking (e.g., NuSMV) exhaustively explore state-space against CTL/LTL specifications.
5. Interactive theorem provers (e.g., Coq/Rocq, Isabelle/HOL)

Approaches to verification can be classified according to the following criteria [HuthR1] pages 172-173:

Proof-based vs. model-based

In a proof-based approach, the system description is a set of formulas Γ (in a suitable logic) and the specification is another formula ϕ . The verification method consists of

2 Background

trying to find a proof that $\Gamma \vdash \phi$. In a model-based approach, the system is represented by a model M for an appropriate logic. The specification is again represented by a formula ϕ and the verification method consists of computing whether a model M satisfies ϕ ($M \models \phi$). This computation is usually automatic for finite models. Logical proof systems are often sound and complete, meaning that $\Gamma \vdash \phi$ (provability) holds if, and only if, $\Gamma \models \phi$ (semantic entailment) holds, where the latter is defined as follows: for all models M , if for all $\psi \in \Gamma$ we have $M \models \psi$, then $M \models \phi$. Thus, we see that the model-based approach is potentially simpler than the proof-based approach, for it is based on a single model M rather than a possibly infinite class of them.

Degree of automation.

Approaches differ on how automatic the method is; the extremes are fully automatic and fully manual. Many of the computer-assisted techniques are somewhere in the middle.

Full- vs. property-verification.

The specification may describe a single property of the system, or it may describe its full behavior. The latter is typically expensive to verify. Intended domain of application, which may be hardware or software; sequential or concurrent; reactive or terminating; etc. A reactive system is one which reacts to its environment and is not meant to terminate (e.g., operating systems, embedded systems and computer hardware).

Pre- vs. post-development.

Verification is of greater advantage if introduced early in the course of system development, because errors caught earlier in the production cycle are less costly to rectify. (It is alleged that Intel lost millions of dollars by releasing their Pentium chip with the FDIV error.

2.1.1 SAT solvers

Propositional logic - consider propositions and relations between them and constructions of argument based on them.

Propositions - facts or statements about the world that can be true or false. Also defined as atomic propositions or Atoms. Some propositions can be described in natural language as atomic or indecomposable declarative sentences) [HuthR1].

Atomic propositions can be connected to each other with logical connectives and compose propositional formulas. The most common connective relations are:

- Negation $\neg$
- Conjunction $\wedge$

2 Background

- Disjunction $\vee$
- Implication $\rightarrow$

The Backus Naur form (BNF) for propositional logic well-formed formula:

$$\phi ::= p \mid (\neg\phi) \mid (\phi \wedge \phi) \mid (\phi \vee \phi) \mid (\phi \rightarrow \phi)$$

where p - is any atomic proposition.

Based on these we can construct a system of reasoning: from some set of formulas we can infer (prove) another formula that can be true or false:

$$\phi_1, \phi_2, \phi_3, \dots, \phi_n \vdash \psi$$

Such formulas called sequents, where on the left side there is a set of formulas called premises and on the right side - a formula called conclusion

Another relation between logical formulas is semantic entailment:

$$\phi_1, \phi_2, \phi_3, \dots, \phi_n \models \psi$$

it means that if propositional logic formulas $\phi_1.. \phi_n$ evaluate to True, then formula ψ also evaluates to True.

The formula ϕ is valid if for all valuations of atomic propositions in formula it evaluates to True:

$$\models \phi$$

SAT solver is a program which aims to solve the Boolean satisfiability problem (SAT), where the Boolean satisfiability problem sometimes called propositional satisfiability asks whether there exists an interpretation that satisfies a given Boolean formula.

Given formula ϕ is **satisfiable** if it has a valuation in which it evaluates to True.:

In other words, it asks whether the formula's variables can be consistently replaced by the values TRUE or FALSE to make the formula evaluate to TRUE. If this is the case, the formula is called satisfiable, else unsatisfiable.[**HuthR1**].

SAT problem is NP-complete, as a result, only algorithms with exponential worst-case complexity are known. In spite of this, efficient and scalable algorithms for SAT were developed during the 2000s, which have contributed to dramatic advances in the ability to automatically solve problem instances involving tens of thousands of variables and millions of constraints.

SAT solvers usually begin by converting a problem to conjunctive normal form (CNF). **Conjunctive normal form (CNF)** - as a conjunction of clauses, where each clause is a

2 Background

disjunction of literals. Literal is an atomic proposition or a negated atomic proposition. Conjunctive normal form allows easy check of validity of a formula which otherwise take times exponential to number of atoms.

2.1.2 SMT

Propositional logic is limited to express relations between logical components other than equivalent to natural language: *not, and, or, if..then* and declarative sentences that can only be True or False.

In order to be able to operate with finer logical structures we need to enrich the language. The necessity to solve this problem lead to design of Predicate logic or first-order logic.

In First-order logic we introduce:

- Variables that represent individual objects.
- Functions $\mathcal{F}$ that refer to objects.
- Predicates $\mathcal{P}$ - symbols that semantically represents some property or relation.
- Quantifiers: universal $\forall$ and existential $\exists$.

The Backus Naur form (BNF) for predicate logic formulas:

$$\phi ::= P(t_1, t_2, \dots, t_n) \mid (\neg \phi) \mid (\phi \vee \phi) \mid (\phi \wedge \phi) \mid (\phi \rightarrow \phi) \mid (\forall x \phi) \mid (\exists x \phi)$$

where,

t - called Terms and can be variable, constants or functions on terms

$$t ::= x \mid c \mid f(t, \dots, t)$$

Atomic formulas are predicates applied to terms: $P(t_1, t_2, \dots, t_n)$

Variables can be free (supplied from outside the formula) or bounded by some quantifier.

Unlike propositional logic, in predicate logic we cannot just evaluate formulas based on given valuation by assigning True or False to the variables. We require a model of all function and predicate symbols involved.

A model M for a first-order logic contains:

2 Background

- Domain A is a non-empty set of individuals, mapping each variable to a value in A
- Set of functions where n -ary function is interpreted as a concrete function $f^m : A^n \rightarrow A$.
- Set of predicates n -ary predicate is interpreted as a relation $P^m \subseteq A^n$

Truth in a model is defined recursively:

$$M \models P(t_1, \dots, t_n) \text{ iff } f(t_1, \dots, t_n) \in P^m$$

$$M \models \text{forall } x \phi \text{ iff for every } a \in A, M \models [x \rightarrow a] \phi$$

$$M \models \text{exists } x \phi \text{ iff for some } a \in A, M \models [x \rightarrow a] \phi$$

A sentence has no free variables, so its truth value depends solely on M .

We write $\Gamma \models \psi$ (semantic entailment) when every model that makes all formulas in Γ true also makes ψ true.

Soundness and completeness ensure $\Gamma \vdash \psi$ (provability) exactly when $\Gamma \models \psi$, but the two perspectives complement each other: proofs are constructive, while counter-models establish non-entailment.

2.2 Model checking

Model checking is based on temporal logic. The model in temporal logic described as a system with few states that can change with time. The formula is not statically true or false in a model, as it is in propositional and predicate logic, it can be true in some states and false in others, the static notion is replaced by a dynamic one [HuthR1].

In model checking, the models M are transition systems and the properties ϕ are formulas in temporal logic. To verify that a system satisfies a property, we must do three things:

1. model the system using the description language of a model checker, arriving at a model M ;
2. code the property using the specification language of the model checker, resulting in a temporal logic formula ϕ ;
3. Run the model checker with inputs M and ϕ .

2 Background

Model checking is a core approach in Formal verification that involves an algorithmic search of the state space of a system model to check if a given property (specification) holds, where:

Model- is an abstract representation of the system (often a state-transition graph or a program) capturing its possible states and transitions. The model can be described in a modeling language (like Promela for SPIN, or as a transition system in a tool like NuSMV).

Specification- is a formal property that the model should satisfy, commonly expressed in a temporal logic such as LTL or CTL. For example, an LTL specification might state that “whenever request happens, eventually response occurs” or a CTL property might require that “in all execution paths, a certain error state is never reached.”

2.2.1 Classification of Model checking methods

Model checking can be classified based on time concept into: **linear-time temporal logic** and **branching time logic**. [HuthR1]

Linear-time temporal logic

Linear-time temporal logic (LTL) considers time as a set of paths, where each path is a sequence of time instances [HuthR1]. LTL has the following syntax in Backus Naur form:

$$\begin{aligned} \phi ::= & T \mid \perp \mid p \mid (\neg\phi) \mid (\phi \wedge \phi) \mid (\phi \vee \phi) \mid (\phi \rightarrow \phi) \\ & \mid (X\phi) \mid (F\phi) \mid (G\phi) \mid (\phi U \phi) \mid (\phi W \phi) \mid (\phi R \phi) \end{aligned}$$

The connectives X, F, G, U, W, R called temporal connectives, because they describe relations between states in different moments of time. The meaning of these connectives described in Table 2.2.1

In LTL, model can be represented as a transition system [DGL1].

Transition system T :

$$T = (S, \xrightarrow{a}, L, s)$$

2 Background

X	Next	ϕ must hold in the very first successor state on the chosen path.
F	Finally ($F\phi$)	ϕ must hold at some future state on the path (i.e., “eventually ϕ ”).
G	Globally ($G\phi$)	ϕ must hold at every state on the entire path (“always ϕ ”).
U	Until ($\phi U \psi$)	ψ must be true at a future state, ϕ must stay true at every state before that one.
W	Weak until ($\phi W \psi$)	either ($\phi U \psi$) holds or ϕ stays true forever if ψ never becomes true.
R	Release ($\phi R \psi$)	ψ must hold up to and including the first state where ϕ becomes true; if ϕ never becomes true, ψ must hold forever (the dual of Until).

Table 2.1: LTL temporal connectives

where:

S - set of states, called the space states of T,

$\xrightarrow{a}$ - transitions (binary relation of states), associated with some action $a \in A$

L - labelling function that reflects states to propositional atoms

s - initial state of T, can be omitted if we want to describe only global structure.

Sometimes if we consider only one action label we can omit labeling transition: $\rightarrow$.

A path π in a transition system T is a finite or infinite sequence of states and (optionally) actions which transform each state into its successor: $s_0 \xrightarrow{a_0} s_1 \xrightarrow{a_1} \dots$

Trace or computation in a transition system is a finite or infinite sequence of state descriptions along the path: $L(s_0) \xrightarrow{a_0} L(s_1) \xrightarrow{a_1} \dots$

Branching temporal logic (CTL)

Branching temporal logic represents time as a tree rooted in present and branching into the future, the other name is Computation Tree logic (CTL). LTL implicitly quantifies universally over paths [DGL1]. Therefore, properties which assert the existence of a path cannot be expressed in LTL. We can solve this problem by considering the negation of the property in question. For example to check whether there exists a path from s satisfying the LTL formula ϕ , we check whether all paths satisfy $\neg\phi$; a positive answer to this is a negative answer to our original question, and vice versa. We used this approach when analysing the ferryman puzzle in the previous section. However, properties which mix universal and existential path quantifies cannot in general be model checked using LTL approach, because the complement formula still has a mix. Branching-time logics solve this problem by allowing us to quantify explicitly over paths.

2 Background

CTL has the following syntax in Backus Naur form:

$$\phi ::= T \mid \perp \mid p \mid (\neg\phi) \mid (\phi \wedge \phi) \mid (\phi \vee \phi) \mid (\phi \rightarrow \phi)$$

$$\mid (AX\phi) \mid (EX\phi) \mid (AF\phi) \mid (EF\phi) \mid (AG\phi) \mid (EG\phi) \mid A[\phi U \phi] \mid E[\phi U \phi]$$

New introduced quantifiers A and E mean:

$A\phi$ - formula ϕ for all paths

$E\phi$ - exists a path that formula ϕ

Connectives X,F,G cannot occur without being preceded by A or E.

Weak until W and release R are usually not included in CTL, but can be derived.

Temporal logic (CTL*)

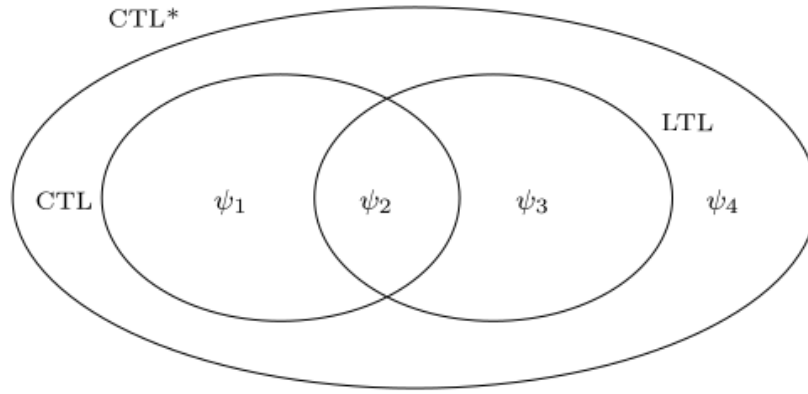


Figure 2.1: LTL, CTL and CTL* logic relation [HuthR1]

Statistical Model Checking (SMC)

Some systems have huge or continuous state spaces (e.g. complicated analog components, or software with random timing), or they exhibit probabilistic behavior (randomized protocols, hardware failures) where one is interested in quantitative properties (like "the request will be served within 1s with probability at least 0.99"). Statistical Model Checking (SMC) is an approximate verification technique that uses simulation and statistical methods instead of exhaustive exploration [Younes2002AcceptanceSampling].

SMC treats the model checking problem as a statistical inference problem [Younes2006StatisticalPMC]. It executes a number of simulations (random or guided) of the system. Checks whether

2 Background

each simulation trace satisfies the property ϕ (which often is a bounded temporal logic property like “within T time units, something happens”). Based on the outcomes, it uses statistics (like hypothesis testing or estimation with confidence intervals) to infer with high confidence whether the property holds with required probability.

SMC was pioneered by Younes in the 2000s [**Younes2002AcceptanceSampling**]. There are two main modes: hypothesis testing (e.g. “check if the probability $P(\text{property}) > p$ with confidence 95%”) and estimation (estimate the probability itself within some error bound).

They adopt the continuous stochastic logic (CSL) as our formalism for expressing probabilistic real-time properties of discrete event systems. CSL has previously been proposed as a formalism for expressing temporal and probabilistic properties of continuous-time Markov chains (CTMCs) by Aziz1996VerifyingCTMC [**Aziz1996VerifyingCTMC**].

A CSL formula is either a state formula or a path formula.

The Backus Naur form (BNF) for CSL logic well-formed formula:

State formulas:

$$\phi ::= \text{true} \mid a \mid \neg\phi \mid \phi \wedge \phi \mid P_{\leq\theta}(\rho)(\phi)$$

Path formulas:

$$\rho ::= X\phi \mid \phi U_{\leq t} \phi$$

Where ϕ is a state formula and ρ is a path formula.

Tools like PRISM and its SMC extension, described in [**Kwiatkowska2011PRISM**], STORM [**Dehnert2017Storm**], or PLASMA Lab [**Legay2016PlasmaLab**] implement statistical model checking for various kinds of stochastic systems.

The obvious advantage is that SMC sidesteps the state explosion by not exploring the full state space – instead, it samples paths. This makes it scalable to systems with enormous or even infinite state spaces, as long as we can simulate the system. It is also applicable to systems with continuous dynamics or very large numbers of states where traditional model checking is infeasible. Another benefit is ease of implementation: if there is a simulator for your system, adding a statistical checker on top is easier than building a full model checker. SMC naturally supports probabilistic properties – e.g., verifying properties of a Markov Chain or Markov Decision Process by sampling behaviors and using results like Chernoff or Hoeffding bounds to determine if the property likely holds. Because it uses actual simulations, SMC can handle very complex nonlinear or black-box system aspects (treating them as black boxes that just produce simulation traces).

2 Background

The trade-off is that SMC provides a confidence result, not a guarantee. It is probabilistic verification: it might say “with 99% confidence the property holds with probability ≤ 0.97 ”. There’s a small chance of error in the verification result (false positives/negatives bounded by the confidence level). Also, SMC may require a large number of simulations to reach high confidence, especially for rare events (if the property failing is a rare event, one might need many samples to observe it even once). Rare property verification is a known challenge – advanced importance sampling or rare event simulation techniques might be needed in such cases.

SMC can’t truly prove a property in the mathematical sense – it can only give statistical evidence. If a property almost always holds but has a corner-case failure, SMC might miss it if that scenario has very low probability and isn’t sampled. Thus, it sacrifices completeness. Another limitation is that SMC typically handles quantitative properties (often expressed in probabilistic temporal logics like PCTL or BLTL) – it’s not usually used for pure logical properties that are either true or false; it’s more for properties like “the probability of an event is at least p ”.

Runtime Verification and Monitoring

Runtime verification (RV) is a technique that involves monitoring a running system (or execution traces produced by it) against a formal specification [Hinrichs2014ModelCheckRuntimeVer]. Unlike the other methods discussed, which are largely offline (analyzing a model mathematically), runtime verification is applied during execution. It can be seen as a lightweight formal method, sitting somewhere between testing and model checking: you instrument the system with monitors, and these monitors check whether the execution is satisfying the desired properties. If a violation is detected, it can be logged, or in some cases, a recovery action can be taken.

Key components of runtime verification include:

- A set of monitors or runtime assertions derived from the formal specification (often temporal logic formulae or automata on traces).
- An instrumentation of the system to report events to the monitors.
- The monitors run in parallel with the system (or on logged traces) and detect property satisfaction or violation on the fly.

For example, if you have a property “whenever interrupt is raised, service routine runs within 100ms,” a runtime monitor can watch for interrupts and check timing until service routine is called, flagging an error if the 100ms deadline is missed.

Runtime verification avoids the state explosion problem entirely, because it doesn’t explore all behaviors – it only checks the actual behaviors that occur. Runtime verification avoids the complexity of traditional formal verification techniques (like model

2 Background

checking and theorem proving) by analyzing only one or a few execution traces and by working directly with the actual system. This means it scales well to very complex systems, since you're just running the system (or a high-fidelity simulation of it) and the overhead is only in checking the property on that run. It also has the advantage of checking the real system (no abstraction or modeling error, since the code that runs is what's verified against the property, at least for that run). It can catch errors in scenarios that were not anticipated by designers as soon as they occur in the field or during testing, thus serving as a powerful debugging and validation aid. Runtime verification can be used continuously in deployed systems for critical properties (e.g., a monitor that ensures no unauthorized file access operations occur, and triggers an alarm if they do, providing a layer of security monitoring). Another plus is that runtime verification can sometimes detect issues that static model checking might miss due to environment assumptions. At runtime, the real environment provides inputs and the monitor can see violations under real conditions, including those hard-to-model aspects.

The fundamental limitation is that runtime verification is not exhaustive. It can only inspect the executions that actually happen (or a sample of them). If a particular bug does not manifest in the observed runs, RV cannot detect it. It provides no guarantee that "no bug exists" — only that "no bug was observed in the runs we monitored". In other words, it lacks completeness/coverage, unlike model checking which (for a model) covers all behaviors by design. Another challenge is performance overhead: monitors consume resources, and if properties are complex (say, specified in LTL, which might require monitoring a suffix of the trace), the runtime checking might slow the system or require buffering events. However, in many cases monitors can be optimized or run on a separate core, and the overhead is manageable (reports of a few percent overhead for simple monitors are common). Additionally, writing good monitors (formal properties) and instrumenting the system requires effort. It's easier than full verification, but still requires formal specification skills. If one instrument too much or monitors too many properties, the overhead can become significant, or the system's real-time behavior might be perturbed.

Runtime verification is often used in safety-critical systems as an additional runtime safety net. For instance, in aerospace or automotive software, one might include monitors for conditions like "no two actuators turn on simultaneously" or "if sensor reading exceeds threshold, system enters safe mode within 1 second," etc. NASA has explored RV for spacecraft software monitoring during missions [**Slagel2024FormalRuntimeAssurance**](since you cannot model check the entire flight code with all analog intricacies, but you can monitor key properties as the mission progresses). Runtime Verification has also become popular in security (e.g., monitoring program execution for compliance with security policies). For example, a monitor can watch system calls to detect a sequence that matches a known attack pattern or a violation of a security automaton. Another

2 Background

domain is testing: during testing, using runtime monitors can greatly increase the chance of noticing an issue. Instead of manually inspecting outputs, formal monitors can rigorously check that each test execution satisfies the expected temporal properties. This is often called “monitoring oracles” in testing – formal specifications serve as test oracles at runtime. In summary, runtime verification does not replace model checking but complements it. Where model checking cannot be applied (due to scale or undecidability), or in deployed scenarios where you want continual assurance, RV provides a way to leverage formal specifications dynamically. It trades completeness for scalability and immediacy.

2.3 Self-replicating robotic systems

2.3.1 Self-replication introduction

Self-replication is the process by which a dynamical system constructs an identical—or very similar—copy of itself. According to **Sipper1998SelfReplication**, replication is an ontogenetic, purely developmental phenomenon: it involves no genetic- or code-modifying operators and yields an exact duplicate of the parent organism.

Replication must therefore be distinguished from reproduction, a phylogenetic, evolutionary process that employs genetic operators (e.g., crossover and mutation in biology) and ultimately generates diversity and evolution [**Sipper1998SelfReplication**].

This study focuses exclusively on replication and self-replication, code-altering and evolutionary processes are beyond its scope.

Self-assembly refers to the autonomous organization of components into patterns or structures without human intervention. In robotics, this term may describe either (i) a kinematic machine that manipulates discrete parts to build an assembled copy of itself (kinematic self-replication) or (ii) a collective of mobile robots that can autonomously connect, disconnect, and reconfigure themselves (swarm robotics and robotic self-reconfiguration [**Tan2020Framework**]).

When the replication process is controlled solely by the system being reproduced, it is called a self-replicating system [**Lee2007Robotic**]. In contrast, if the replication process requires external elements that actively control and manipulate resources and direct the process, it is simply a replicating system. A biological analogue is a virus, which replicates by exploiting a host cell.

A self-replicating machine or self-replicating robotic system (SRRS) is thus an autonomous robot capable of manufacturing a copy of itself from raw materials, or from minimal possible units related to considered environment, thereby exhibiting self-replication in a manner analogous to that observed in nature.

As shown by **Chirikjian2022Entropy** in [**Chirikjian2022Entropy**] "the basic difficulty is that in order to close the self-replicating loop, there must be a balance between the overall system complexity and the ability of the system to handle the simplest inputs possible. Generally speaking, the more complex the system is, the more capable it is to assemble the simplest parts, or even harvest raw materials. But when the system is very complex, it then requires more to reproduce. For example, if a robot needs microprocessors and a 3D printing system needs a high-energy laser, then producing those from in situ resources becomes an enormous challenge." And "One way around that problem is to focus on production of mechanical parts in situ such as structural

2 Background

members for robots, factories, and habitats, and to send the relatively light-weight “vitamin” components such as computers, lasers, and some chemical reagents to be considered as inputs to the system rather than as items to be replicated. In this way, a practical version of in situ resource utilization (ISRU) can be achieved by reclassifying inputs. Moreover, developing systems that are dependent on some inputs that the systems cannot manufacture from scratch is both more akin to living systems that require specific nutrients, and is also safer than developing self-replicating systems that could continue to replicate without the imposition of resource bottlenecks. Without such intentional bottlenecks there could be fears of things running amuck for generations."

2.3.2 Classification of Self-replication systems

Type-related replication

Building upon the foundational work of **Jones2021Thesis** [**Jones2021Thesis**] and works of **Lee2007Robotic**, self-replicating systems can be systematically classified according to their type-related replication architecture into three primary categories:

Centralized replication system where robots can produce only robots without replication functionality as is shown on Figure 2.2a.

The centralized approach has several distinct advantages: it provides precise control over population growth, ensures quality consistency through standardized manufacturing processes, and reduces resource competition among robots. However it also introduces significant vulnerability through single points of failure - if the replicating robots are damaged or destroyed, the entire system's reproductive capacity is compromised.

Additionally, the concentration of replication resources that have to be located or transported near the replicating robots may create bottlenecks that limit the system's expansion rate and adaptive responsiveness.

Decentralized replication represents a system architecture where every robot within the system possesses full replication capabilities, see Figure 2.2b. This distributed approach is inspired by cellular division processes, where each entity contains the complete blueprint and machinery necessary for creating identical copies of itself.

The decentralized model excels in resilience, scalability and speed of self-replication. The system can continue expanding even if individual robots are eliminated, as each surviving unit has complete reproductive autonomy. This architecture facilitates rapid colonization of new environments and provides redundancy against catastrophic failures. However, decentralized systems face challenges in coordination and control for

2 Background

some systems, possible uncontrolled exponential growth, and consistent quality across independently operating replication processes.

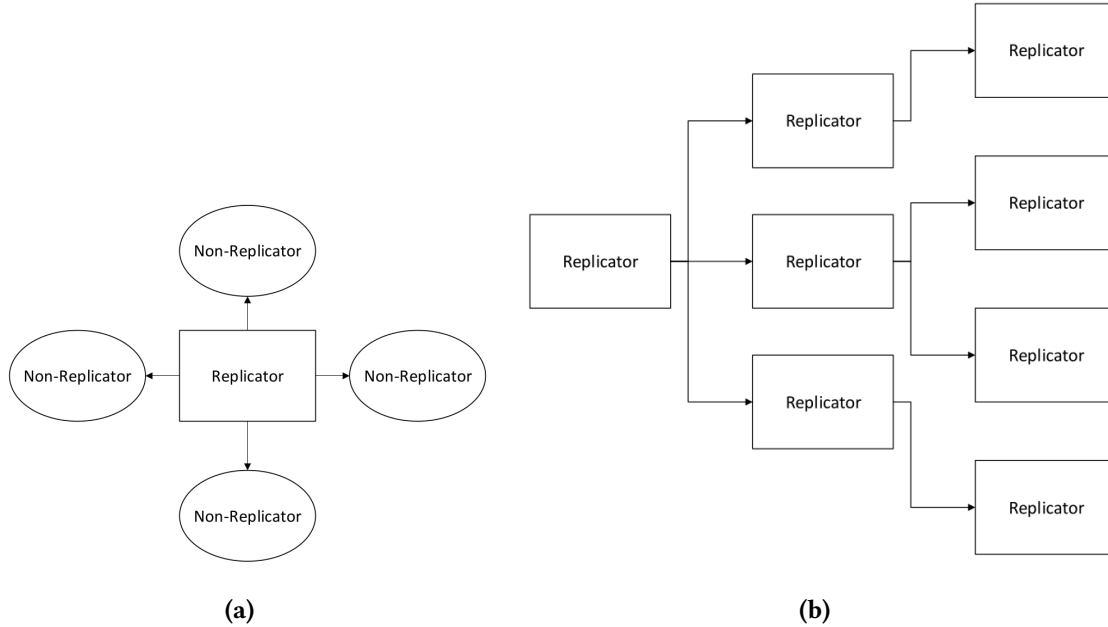


Figure 2.2: Centralized and decentralized replication [Jones2021Simulation]

Hierarchical replication architectures represent a hybrid approach that combines elements of both centralized and decentralized strategies. In these systems, robots with replication capabilities can produce two distinct categories of offspring: specialized robots lacking replication ability (similar to centralized systems) and fully autonomous robots capable of self-replication (similar to decentralized systems), as is shown on Figure 2.6.

This multi-tiered approach enables dynamic adaptation to environmental conditions and mission requirements. The system can produce specialized worker robots for specific tasks while simultaneously maintaining a population of replicator robots to ensure continued expansion and redundancy.

The hierarchical model allows for optimized resource allocation, where replication-capable robots can focus on expansion while non-replicating robots specialize in operational tasks, environmental sensing, or resource collection.

2 Background

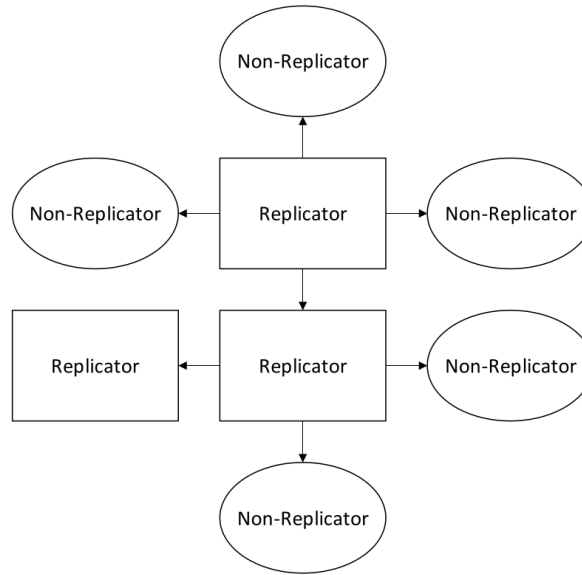


Figure 2.3: Hierarchical replication [Jones2021Simulation]

Population Composition Classification

The composition of robot types within an SRRS fundamentally influences system behavior, specialization potential, and adaptive capacity. This classification dimension addresses the diversity of robotic types operating within the system.

Homogeneous Systems Homogeneous SRRS consist of robotically identical units, where all entities share the same physical design, computational capabilities, and behavioral programming. This uniformity creates predictable system dynamics and simplifies coordination protocols, as all robots operate with identical sensor suites, actuator capabilities, and decision-making algorithms.

The homogeneous approach offers advantages in replication process consistency, simplified communication protocols, and predictable emergent behaviors. Resource allocation becomes straightforward when all units have identical requirements, and system-wide updates or modifications can be uniformly applied. However, homogeneous systems may struggle with task specialization and environmental adaptation, as the lack of diversity limits the system's ability to optimize for varied operational requirements.

Heterogeneous Systems Heterogeneous SRRS incorporate multiple distinct robot types, each potentially specialized for specific functions, environmental conditions, or operational roles. This diversity enables flexible division of labor, where different

2 Background

robot types can be optimized for exploration, resource gathering, construction, maintenance, or replication tasks.

The heterogeneous model provides enhanced adaptability and operational efficiency through specialization. Different robot types can be designed with complementary capabilities, creating synergistic effects that exceed the performance of individual units. This approach enables the system to respond more effectively to complex environmental challenges and mission requirements.

However, heterogeneous systems require sophisticated coordination mechanisms to manage inter-type communication, resource distribution, and collaborative decision-making across diverse robotic platforms.

Interaction with environment or external system

Another classification of self-replicating robotic systems can be made based on the degree of autonomy and reliance on external systems, as it described in [Lee2007Robotic]. A fundamental distinction is made between passive and active self-replication:

Passive self-replicating systems do not possess inherent mechanisms to actively reproduce themselves. However, given an appropriate environment and the right configuration of components, replication can still occur. A canonical example of this is the Penrose block replicators, which demonstrate self-replication through mechanical constraints and spatial arrangement without any active control [Penrose1958Mechanics].

Active self-replicating systems, on the other hand, are equipped with the functional capacity to carry out the replication process. These systems may still rely on passive environmental structures (such as fixtures or gravity) to assist in replication, but are actively engaged in constructing replicas of themselves.

Interaction with environment or external system, alternative classification

A more detailed and operationally useful classification is introduced by Suthakorn2002SpaceUtilization [Suthakorn2002SpaceUtilization]. This framework emphasizes how SRRS interact with their environments and whether replication is accomplished directly or through intermediate stages. Based on this framework, all SRRS are categorized into two primary classes:

Directly replicating SRRS - in which a robot is capable of constructing an exact copy of itself in a single generational step. These systems can be further divided into the following subcategories, based on their reliance on fixtures and replication strategy:

2 Background

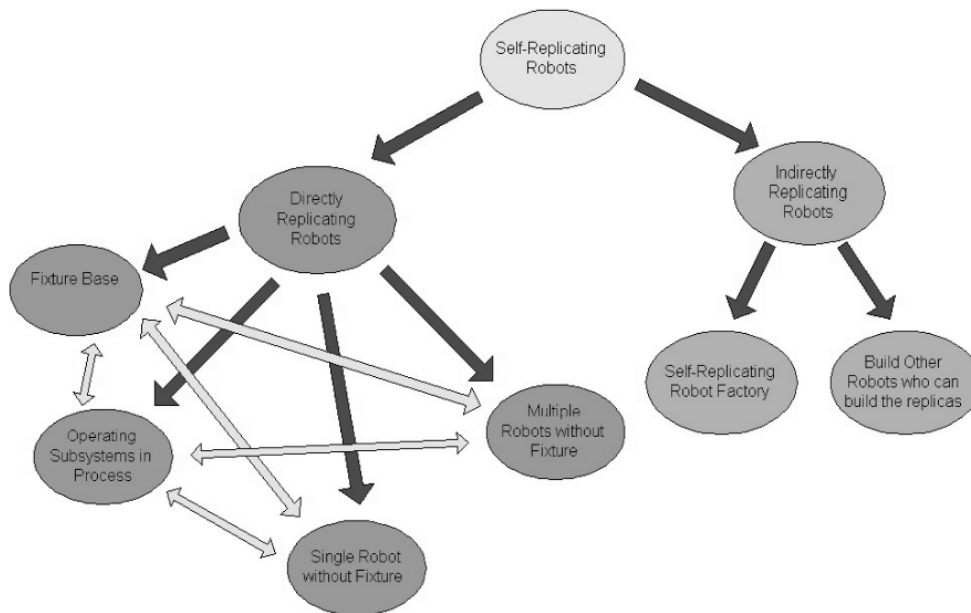


Figure 2.4: Directly and indirectly replicating robots classification [Suthakorn2003SemiAutonomous]

Directly Replicating Robots can be further classified into following groups:

- **Fixture-Based Group** The self-replicating robots in this group depend on external fixtures in order to complete the self-replication process. Passive fixtures are able to assist in this because of the shape constraints that they impose. In some other cases, to unify subsystems, push-pull fixtures are helpful as well.
- **Operating-Subsystem-in-Process Group** In this group one or several subsystems of the replica can operate before the replica itself is fully assembled. These subsystems are able to assist the original self-replicating robot during the assembly of the replica.
- **Single-Robot-Without-Fixture Group** In this group only one robot is used to finish the self-replication process. Thus, the robot in this group depends only on the available environment. Usually, the complexity of the subsystems or the number of subsystems in the replica is very low for this group.
- **Multi-Robot-Without-Fixture Group** In this group more than one robot works together in the self-replication process without the assistance of fixtures. A major advantage is the reduction of the time required for self-replication. A disadvantage is that there may be interference problems among robots. There are

2 Background

several possible ways that a self-replicating robot can be categorized in two or three groups mentioned above. The combination of two or three different concepts can be incorporated in a potential design, such as a combining operating-subsystems-in-process with fixture-based robots. More categories are likely to be developed in the subsequent stages of our research in the area of self-replicating robots.

Indirectly replicating - where replication is achieved through intermediate stages rather than a direct one-to-one duplication. The original robot initiates the creation of other robots or constructs a specialized environment that facilitates replication. Indirect replication strategies include:

- **Self-Replicating Robot Factory** A robot or group of robots constructs a fabrication environment — a factory, that can then autonomously produce replicas of the original. This approach encapsulates the replication process into a fixed infrastructure, enabling high throughput and modular production.
- **Intermediate Robot Builders** Instead of replicating itself directly, a robot creates new robots that are not replicas, but are instead specialized for manufacturing. These intermediate agents then produce replicas of the original robot. This cascading replication strategy introduces flexibility and specialization but may involve increased system complexity.

2.3.3 Self-replication models

The concept of self-replicating systems originates from the foundational work of **vonNeumann1962The** who in **vonNeumann1962Theory** proposed the first theoretical model of a self-reproducing automaton [**vonNeumann1962Theory**]. This model, developed in the context of cellular automata, formalized the minimal components necessary for a machine to replicate itself.

According to this model, a self-reproducing automaton consists of four key components:

A (constructor): an automatic factory that collects raw materials and processes them into outputs using a specified written instruction,

B (copier): that takes an instruction and copies it,

C (controller): that controls A and B, actuating them alternatively,

D (instructions) A coded description of the automaton's structure and function, acting as its "genetic" information.

$$(A + B + C + D) \rightarrow (A + B + C + D), (A + B + C + D)$$

2 Background

And the replication process in von Neumann's model, is when the automaton $(A + B + C + D)$ produces another $(A + B + C + D)$.

However, while von Neumann's model is conceptually complete for information-theoretic or logical systems, it exhibits limitations when applied to physical robotic systems. Specifically, it doesn't describe interaction with a physical environment, resource acquisition, resource transformation and management of by-products, energy, and mechanical constraints.

To address these limitations, a more general and physically grounded model was proposed by **Lee2007Robotic** [**Lee2007Robotic**]. In this extended model, the replication process is formalized as a transformation: **Lee2007Robotic** proposed broader model, where a replication process can be written as

$$(R, M, E) \rightarrow (R', M', E') \quad (2.1)$$

where:

M is a multiset of available parts to be used for building replicas by an initial system R is a multiset of an initial functional system to be reproduced. $R \neq 0$; to be a replicating system.

E is a multiset of environmental structures and/or elements involved in the replication process (but not replicated).

R', M', E' , the replicated entities, the remaining or altered material and the possibly changed environment respectively.

In this model R describes the whole replicator $A + B + C + D$ from Von Neumann's model.

This transformation can also be expressed functionally as:

$$(R', M', E') = \Phi(R, M, E, t) \quad (2.2)$$

Where Φ denotes the replication function and t is time. This formulation enables modeling dynamic replication scenarios over time and accounts for state changes in materials and environment.

The model explicitly incorporates environmental interaction (E), which plays a critical role in physical self-replication. As proposed by **Eno2007ReplicationNoControl** [**Eno2007ReplicationNoControl**], environments can be categorized based on the level of structure and predictability:

2 Background

- Completely structured environment - in which every environmental structure is fixed at a precise location and there is no modification or change in any environmental structures during the self-replication process. If we define E' is the set of environmental structures after self-replication process: $E = E' \neq \phi$
- Partially structured environment is like a structured environment, but with structures whose locations can be permuted and perturbed by a small random error in pose during the self-replication process. $E \neq E \neq \phi$
- Unstructured environment has no predefined or organized structure and can dynamically change during the self-replication. Can be formally depicted as: $E = \phi$ where ϕ represents a stochastic or unknown environmental process.

2.3.4 Degree of self-replication

As shown in [Chirikjian2022Entropy]: "In an artificial self-replicating system, if the number of input parts is small, and if each part is itself a complex machine, then the effort involved in the assembly step of self-replication is relatively low compared to the effort of making the input parts. The assembly task in self-replication is then relatively unchallenging in such a scenario. In contrast, if there are many simple (easy to manufacture) parts that need to be assembled by the robot, the relative difficulty is higher. The difference between these two scenarios can be quantified using the concept of the 'degree of self replication'. The highest degree of self replication is when raw materials are harvested by a system and used to produce a replica, since fabrication and assembly are both done by the self-replicating system in that scenario."

In general as shown in [Chirikjian2022Entropy] degree of self-replication can be abstractly described as:

$$DOSR = \frac{\text{System Complexity}}{\text{Parts Complexity}}$$

In Lee2007Robotic Lee2007Robotic proposed a measure of degree of replication [Lee2007Robotic]: measure of how much order is created by the assembly process relative to the existing order in the unassembled parts. In this section, we first define a combined measure of system complexity distribution and relative complexity for ARS.

As a simple measure of subsystem complexity, we count the number of active elements in each subsystem. An active element is a moving mechanical part or a fundamental electronic component. Some special parts, such as a chassis and fixed mechanical parts

2 Background

with special purpose (e.g., a passive end-effector), are also viewed as active elements even if they are not moving mechanical parts.

For a robotic system (recall that all robotic systems are active by our definition) consisting of n subsystems, we define the degree of replication for the system, D_s , as

$$D_s = \frac{C_{min}}{C_{max}} \cdot \frac{C_{total}}{C_{ave}} \cdot \frac{1}{C_{ave}} \quad (2.3)$$

where

$$\begin{aligned} C_{ave} &= \frac{1}{n} \sum_{i=1}^n C_i \\ C_{max} &= \max C_1, \dots, C_n \\ C_{min} &= \min C_1, \dots, C_n \\ C_{total} &= \sum_{i=1}^n C_i + \sum_{i=1}^{n-1} \sum_{j=i+1}^n C_{ij} \end{aligned}$$

C_i is the number of active elements in M_i

C_{ij} is the number of interconnections between the i -th and j -th subsystems when they are assembled. $C_{ij} = 0$ if the i -th and j -th subsystems are not directly connected by the assembly process.

In formula ?? the first term measures the complexity distribution throughout the subsystems, the second term measures the relative complexity of the total system to the individual subsystems, and the last term penalizes for complex subsystems. To design a robotic system capable of replication from low-level parts, the subsystem complexity should remain low relative to the total system complexity (i.e., high relative complexity).

In addition, we view a system consisting of a few complex parts as more trivial than one composed of many low-complexity parts. A higher value of D_s indicates a more truly replicating system in terms of its complexity distribution and relative complexity. Although the concept of D_s may not generalize to all robotic systems, it is useful as a measure to evaluate robotic replicating systems.

2.3.5 Implementations of kinematic self-replicating robotic systems

Low-complexity self-replication systems from specialized modules

The experimental system developed by **Lee2007Robotic** demonstrates active self-replication using six simple electromechanical subsystems [**Lee2007Robotic**]. The

2 Background

system operates autonomously within a partially structured environment and constructs a functional replica of itself without external control or manipulation. This prototype serves as a proof-of-concept for scalable robotic self-replication using low-complexity parts and embedded environmental cues.

The system is a physical prototype of a self-replicating robot made up of six simple electromechanical subsystems (called M1 to M6) as it shown on the Figure 2.5.

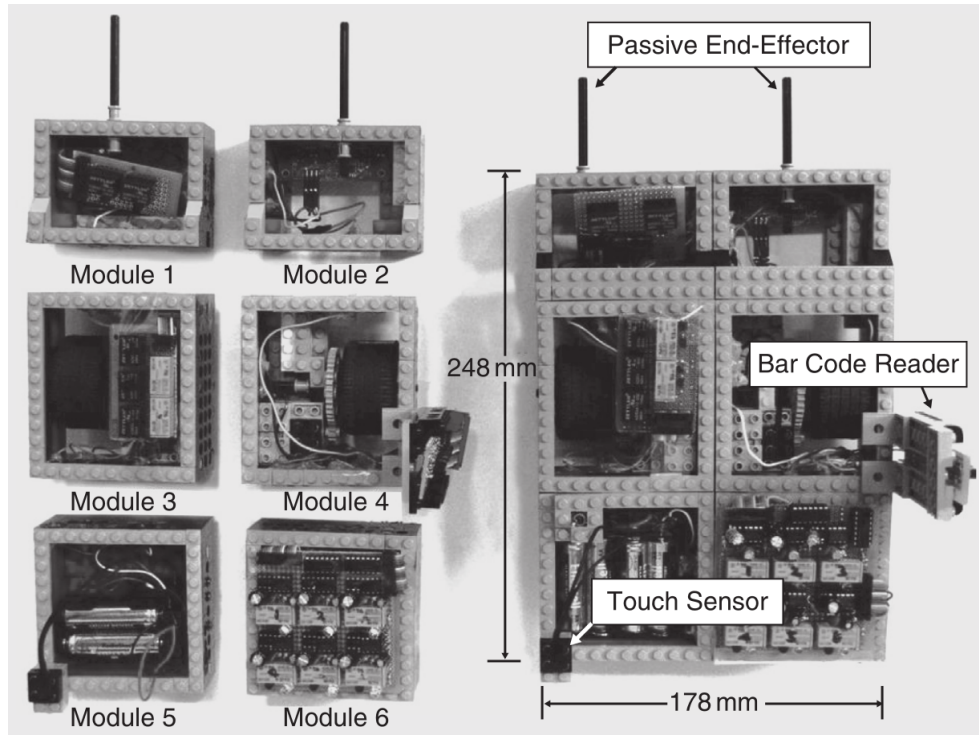


Figure 2.5: Modules (left) and complete robot (right) [Lee2007Robotic]

These modules, when assembled in a specific order, form a complete and functional mobile robot. The purpose of the experiment was to demonstrate that robotic self-replication is possible using relatively low-complexity parts in a partially structured environment—a space that aids replication without directly controlling it. Together, these modules allow the robot to: move along predefined paths, detect locations of components, decide what action to perform and carry and assemble parts.

The robot operates within a partially structured environment—one that contains helpful features, but does not actively participate in the replication. It includes: outer and inner tracks, crossways (branch paths), bar codes, contact codes, sensors, assembly station and wall, used to align the robot during reversing, ensuring precise placement.

2 Background

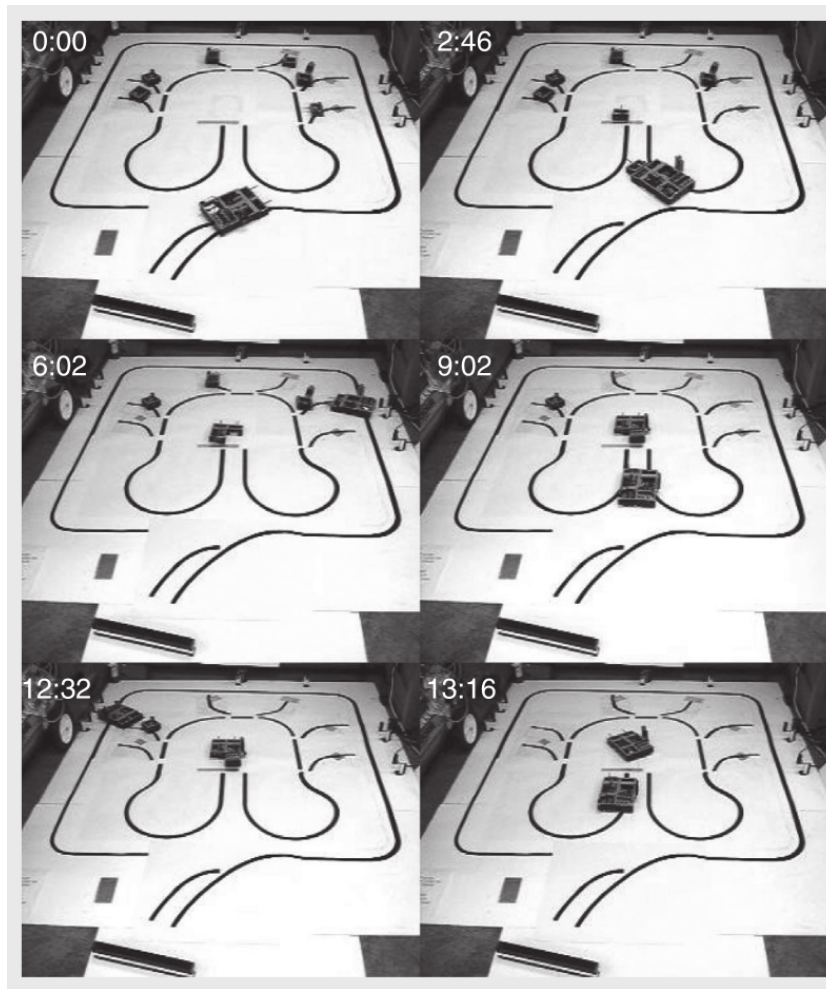


Figure 2.6: The process of self-replication [Lee2007Robotic]

The robot has six states, see Figure 2.7 and three distinctive behaviors for each state: forward line-tracking (line-tracking mode), reversing (reversing mode), and turning left while the timer is on (left-turning mode). The replication process begins with the parent robot navigating along a black track using its line-tracking sensors. As it moves, it detects barcodes placed near the modules it needs to collect; each barcode signals the robot to turn and approach a module. Using its passive end-effector, the robot captures the module and transitions to a new state based on contact sensors. It then transports the collected module to a central assembly station marked by a metal line, which triggers the robot to reverse. The robot backs into position and places the module, using a wall as a physical guide for alignment. After delivering the module, it returns to the track and continues this process for the remaining modules. Each time, a new barcode guides it to the next module, and contact codes confirm successful

2 Background

transitions. The modules are assembled in a layered structure, with specific modules forming the base, middle, and top of the robot. Once all six modules are in place and properly connected, the electrical circuit is completed, and the new robot becomes fully functional. The entire process is autonomous and guided solely by the robot's internal logic and environmental cues successfully constructing a working copy of itself without any external control or assistance.

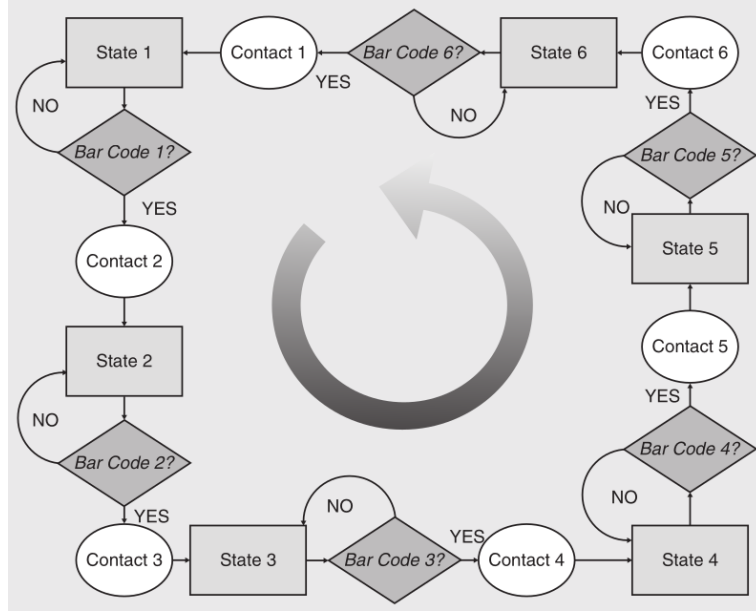


Figure 2.7: State diagram shows behaviors of the robot according to the sensor inputs [Lee2007Robotic].

Using formulas ?? and ?? the system can be represented by

$$R = M_1 + \dots + M_6$$

$$M = M_1 + \dots + M_6$$

$$E = \{\text{tracks, bar codes, contact codes, wall, metal line}\}$$

, the system becomes

$$R' = \{(M_1 + \dots + M_6), (M_1 + \dots + M_6)\}$$

$$M' = \emptyset$$

$$E' = E.$$

2 Background

Kinematic self-replication from unified modules

Different authors proposed kinematic or geometrical self-replication based on unified modules. **Zykov2005SelfReproducing** in **Zykov2005SelfReproducing** demonstrates physical self-replication using autonomous modular robots [**Zykov2005SelfReproducing**].

These machines are built from identical robotic cubes—each a 10 cm module—that can physically attach, detach, and reconfigure to construct replicas of themselves.

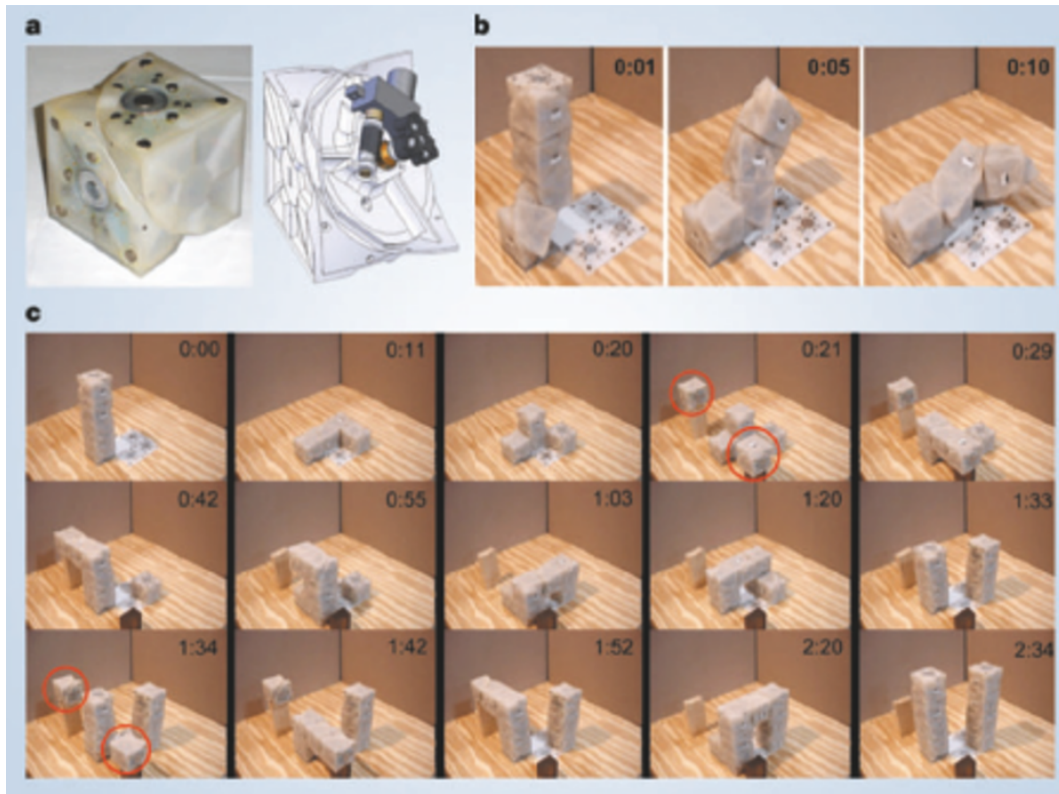


Figure 2.8: Self-replication of a four-module robot [**Zykov2005SelfReproducing**]

Each cube module is split along a diagonal plane and contains internal mechanisms that allow one half to rotate relative to the other, see Figure 2.8a. This ability enables modules to cycle between different configurations by swiveling in few seconds (see Figure 2.8b), allowing connected modules to break apart and reattach in new positions. The modules use electromagnets for selective connection and disconnection, and they receive power and communication signals through conductive face-to-face contacts and a powered baseplate.

The replication process begins with a small robot, composed of four such modules, positioned near two "feeding stations" that contain spare cubes. These cubes are man-

2 Background

ually replenished, but the entire replication procedure is autonomous. Using their internal control logic and distributed microcontrollers, the robot’s modules coordinate motion to pick up new cubes, manipulate them into position, and assemble them into a new, functional robot of the same structure. In one experiment, a four-module robot completed replication in approximately 2.5 minutes (see Figure 2.8), while a simpler three-module robot managed the task in just over one minute. This experiment demonstrates that even simple robotic parts can support mechanical self-replication so that the resulting system can itself repeat the replication process.

Material-robot systems

The material-robotic system proposed by **Jenett2019MaterialRobot** in [**Jenett2019MaterialRobot**] and further developed by **AbdelRahman2022Hierarchical** in [**AbdelRahman2022Hierarchical**] consist of passive structural lattice voxels which form a substrate for the locomotion of purpose-built inchworm modular robots build from active voxels, and capable of rearranging and placing additional voxels, see Figure 2.9.

The combination of voxels and robots forms a system, enabling precise assembly of large structures with simple robots through localized registration to the underlying lattice, and also allows of creation of modular robotic toolkit that utilizes active lattice voxels as the primary structural building blocks.

The active voxels which form the basis of the structural-robotic system are a cuboctahedral (Cuboct) unit cell which offers superior structural properties such as high stiffness at low density with geometric characteristics suited for robotic assembly and fabrication. Specifically the flat Cuboct faces enable fully- constrained connections between single-voxel pairs with four points of contact, as well as a useful decomposition for voxel fabrication.

Combining these active voxels with actuators, control, and power enables unique behaviors including robotic self-replication and hierarchical robot assembly. A single robot, tasked with constructing a large structure, can create a heterogeneous swarm of constructors tailored to maximize material throughput for a given target geometry.

One of possible variations of assembler robots, designed to carry voxel cubes, and a process of self-replication from a set of modules is shown in Figure 2.10.

To perform self-replication, an initial “parent” robot uses its manipulators to pick and place voxels from a nearby pickup station. The parent robot builds a copy of itself—termed a “child” robot—by assembling voxels in a configuration that mirrors its own structure. The replication process is limited to voxels and certain key functional

2 Background

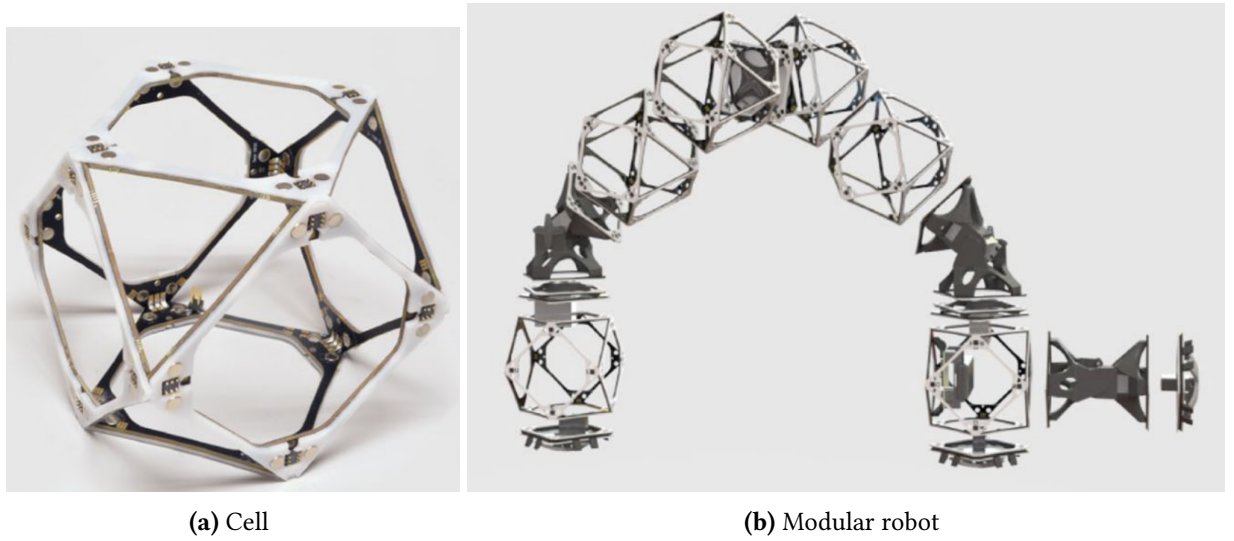


Figure 2.9: Material-robot system [AbdelRahman2022Hierarchical]

modules, including control units, power supplies, and actuated joints (called elbows). Components like wrists and grippers are added afterward through a procedure called “accessorizing,” where the robot attaches these parts to the pre-assembled structure.

A critical constraint in the replication process is that the newly built robot must be free to move upon completion. Therefore, construction cannot proceed directly on the substrate base; instead, it extends out from an anchored platform, ensuring that once finished, the child robot can undock and function independently. The replication logic is programmed into a centralized control system, which assigns tasks, manages timing, and ensures robots do not collide. The control system guides each robot step-by-step in constructing a new robot, allowing for recursive reproduction over multiple generations.

This system’s significance lies in its ability to scale construction tasks by increasing the number of robots through self-replication, rather than requiring the deployment of additional robots externally. As more robots are built, the workload is distributed, theoretically allowing exponential speed-up in large-scale assembly projects. The practical implementation uses inchworm-style locomotion, where robots move stepwise over the voxel lattice using coordinated grippers and joints, making them lightweight, simple, and adaptable to modular construction environments.

While this model is a proof-of-concept, it successfully demonstrates robotic self-replication using standardized parts and relatively low-complexity hardware. The modularity and discrete assembly approach also enhance the system’s potential for error detection, robustness, and reconfiguration. Although scalability is limited by actuator strength and

2 Background

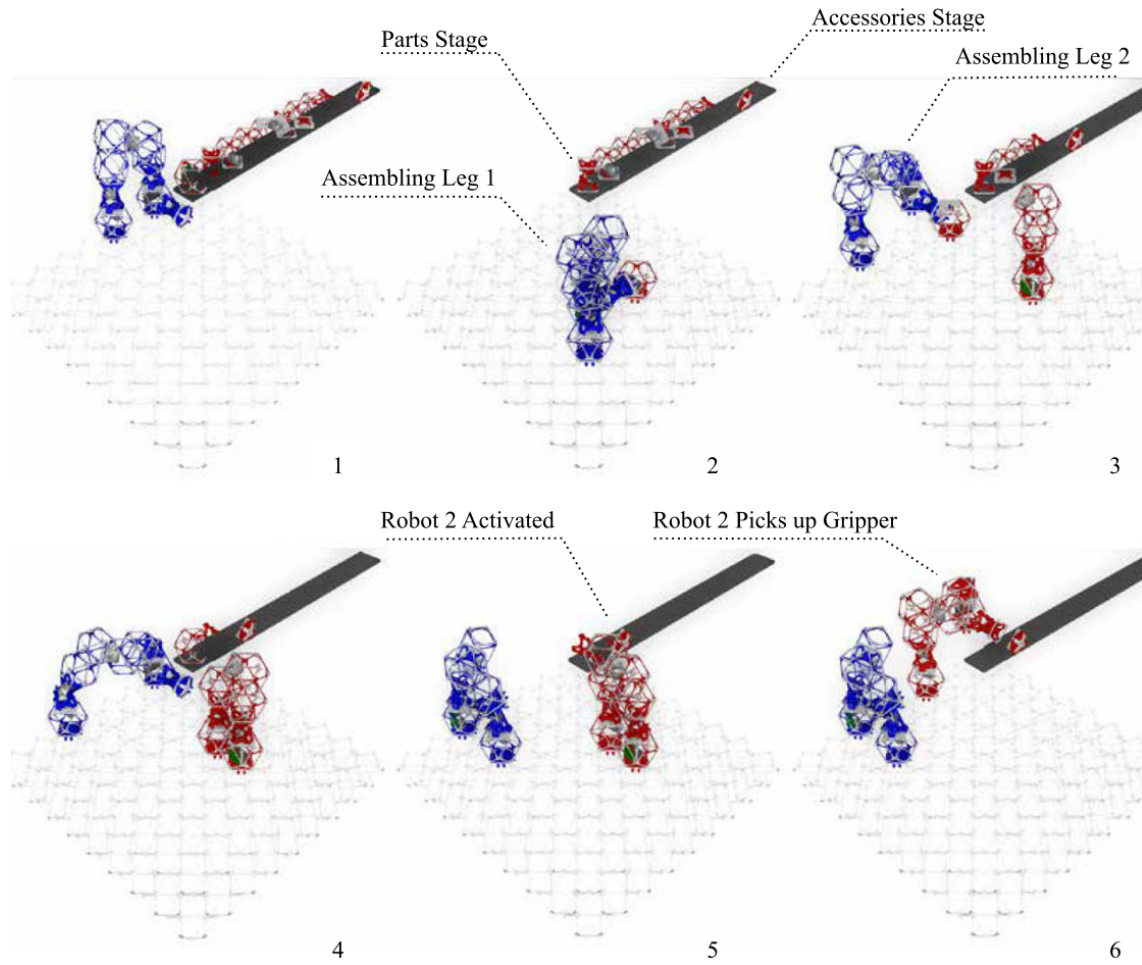


Figure 2.10: Self-replication process [AbdelRahman2022Hierarchical]

coordination complexity, the framework opens new avenues for autonomous robotic manufacturing systems that can grow their own workforce from a finite resource pool.

3 Research questions

This paper will address the following research questions:

1. How can automated model checking and other formal-verification techniques be used for verifying liveness (replication reliability), safety and other properties in self-replicating robotic systems(SRRS), and what domain-specific adaptations are required for practical implementation?
2. What systematic methodology can translate the behaviors and structural constraints of self-replicating robots into expressive temporal-logic models and specifications?
3. What limitations—computational, semantic, or modelling-related – does developed approach face, and how might these limitations be mitigated?
Compare the finite state machine algorithm control with SMV counterexample-based control in simulation by time, flexibility

3.1 Analysis of model checking approaches in self-replicating systems domain

The first research question involves a comprehensive survey and critical evaluation of current state-of-the-art model checking and formal verification tools, such as SPIN, NuSMV, and PRISM, focusing on their applicability to modeling and verifying the unique dynamics of self-replicating robotic systems (SRRS).

This investigation includes a review of relevant literature and existing systems in self-replicating robotics and cellular automata to identify key operational characteristics and formalizable behaviors. It also involves identifying core properties relevant to SRRS, such as liveness (ensuring successful replication within time or resource constraints), safety (avoiding resource conflicts or hazardous outcomes), as well as broader system-level guarantees like termination, deadlock-freedom, and fault-tolerance. The study will further explore how these properties can be expressed in various formal languages, including Linear Temporal Logic (LTL), Computation Tree Logic (CTL), and

3 Research questions

probabilistic extensions. This will require examining what adaptations or abstractions are needed to make these languages suitable for SRRS modeling.

Iterative experimentation with selected tools will help assess the feasibility of encoding and verifying these properties and may lead to the discovery of more complex or emergent behaviors to integrate in later stages.

3.2 Experimental checking

The second research question aims to define a systematic formal modeling and verification workflow tailored to SRRS. This involves developing a formal representation of self-replication processes, structural rules, environmental interactions, and resource usage.

It also includes establishing a methodology to map real-world or simulated robotic behaviors - such as perception, manipulation, and actuation- onto abstract states and transitions that are amenable to formal analysis. Based on this framework, appropriate model-checking tools and specification languages will be selected and justified through experimental evaluation within simulation environments.

A prototype or proof-of-concept implementation will be developed using a simplified SRRS scenario, such modular robotic replication, to demonstrate how this methodology captures essential properties and enables formal verification.

The ultimate goal is to bridge the gap between low-level robotic simulation and control systems and high-level formal models, ensuring that the designed behaviors are both physically feasible and logically justified.

3.3 Limitations and challenges

The third research question addresses the fundamental limitations that current formal methods and model-checking tools face when applied to the complex domain of self-replicating robotic systems. It includes analyzing computational constraints such as state-space explosion, undecidability in dynamic conditions, and potential inefficiencies in scaling existing verification tools.

The study will also examine semantic mismatches between physical or simulated robot behavior and abstract models, particularly in representing kinematic and dynamic aspects, sensor errors, and asynchronous or partially observable environments.

3 Research questions

Additionally, the research will explore modeling limitations related to the representation of dynamic structure generation and adaptive behaviors.

To address these challenges, various mitigation strategies will be proposed, including modular verification, changes in system abstractions and hybrid techniques that combine formal models with simulation-based validation.

The outcome of this part may include recommendations for extending the selected verification toolsets, adapting implementation strategies, or designing new experimental setups specifically for the analysis of SRRS.

4 Methodology

4.1 Research tools

Self-replication research in robotics involves complex interactions between assembly processes, modular design, environmental constraints, and control systems. Robotic simulation environments offer a valuable sandbox to explore these dynamics before physical implementation. The ideal simulation platform must support modularity, kinematics, sensing, and interaction with discrete components.

A set of tools for simulation and control of robotic systems including SRRS can be seen in the Table 4.1

In order to simulate kinematic self-replicating systems and to be able to implement control functions that are also applicable to a physical system, the Gazebo simulator with ROS infrastructure seems to be an optimal choice.

Gazebo supports multiple physics engines, such as ODE, Bullet, and DART, allowing to model the dynamics of contact, friction, collisions, and gravity with a high degree of accuracy.

Modular robot design can be modeled through ROS's URDF (Unified Robot Description Format) and SDF (Simulation Description Format). These formats help to define robotic structures as combinations of interconnected parts and joints, closely mimicking the principles of modular self-replicating systems. Components can be described independently and reused across different robots or configurations. It support simulating a wide array of sensors, including cameras, LiDAR, depth sensors, IMUs, proximity sensors, and contact detectors.

Structured or unstructured environments for SRRS can be modeled in great detail, placing objects with specific physical properties, and simulating real-world challenges like terrain irregularities or sensor noise.

ROS provides a powerful middleware layer that manages communication between different components of the robot and the environment. Through ROS, we can implement and test high-level control algorithms, including perception pipelines, decision-making logic, and task planning. It supports distributed systems, so that multi-robot

4 Methodology

Simulator	Physics fidelity	Modularity support	Custom control	Use case for SRRS
Gazebo+ROS	High	Strong via URDF/SDF	ROS integration, plugins, Python/C++ nodes	Physical replication with mobile robots and manipulators
CoppeliaSim (V-REP)	High	Native modular modeling	Lua, Python /remote APIs	Detailed control of part interaction and modular assembly
PyBullet	Medium to High	Good (via URDF /SDF)	Python API (fast iteration)	Cube/voxel-based systems and contact-rich interactions
Webots	Medium	Moderate	C/C++ /Python controllers	Lightweight simulation of small-scale replication setups
Unity+ ML-Agents	Medium	High (custom prefabs)	C#,Python with ML-agents	Learning-based or perception-driven replication strategies
Isaac Sim (NVIDIA)	High	Strong (via Omniverse)	Python scripting, ROS2	High-fidelity simulation of complex autonomous robots
Golly NetLogo CA tools	None	Tile-based, symbolic	Rule-based/ scripting	Theoretical studies (e.g. von Neumann or Langton automata)

Table 4.1: Comparison of simulation environments for self-replicating robotic systems

scenarios (such as cooperative replication) can be easily simulated. ROS also provides tools for recording and analyzing data, tuning controllers, and implementing feedback loops.

Gazebo and ROS are open-source and highly extensible. Developers can create plugins for new sensors, actuators, or control strategies. This is particularly useful in self-replication research, where novel components or interfaces may need to be modeled. Additionally, the close integration between simulation and real-world systems means that code developed in Gazebo with ROS can often be deployed directly to real hardware with minimal changes.

4 Methodology

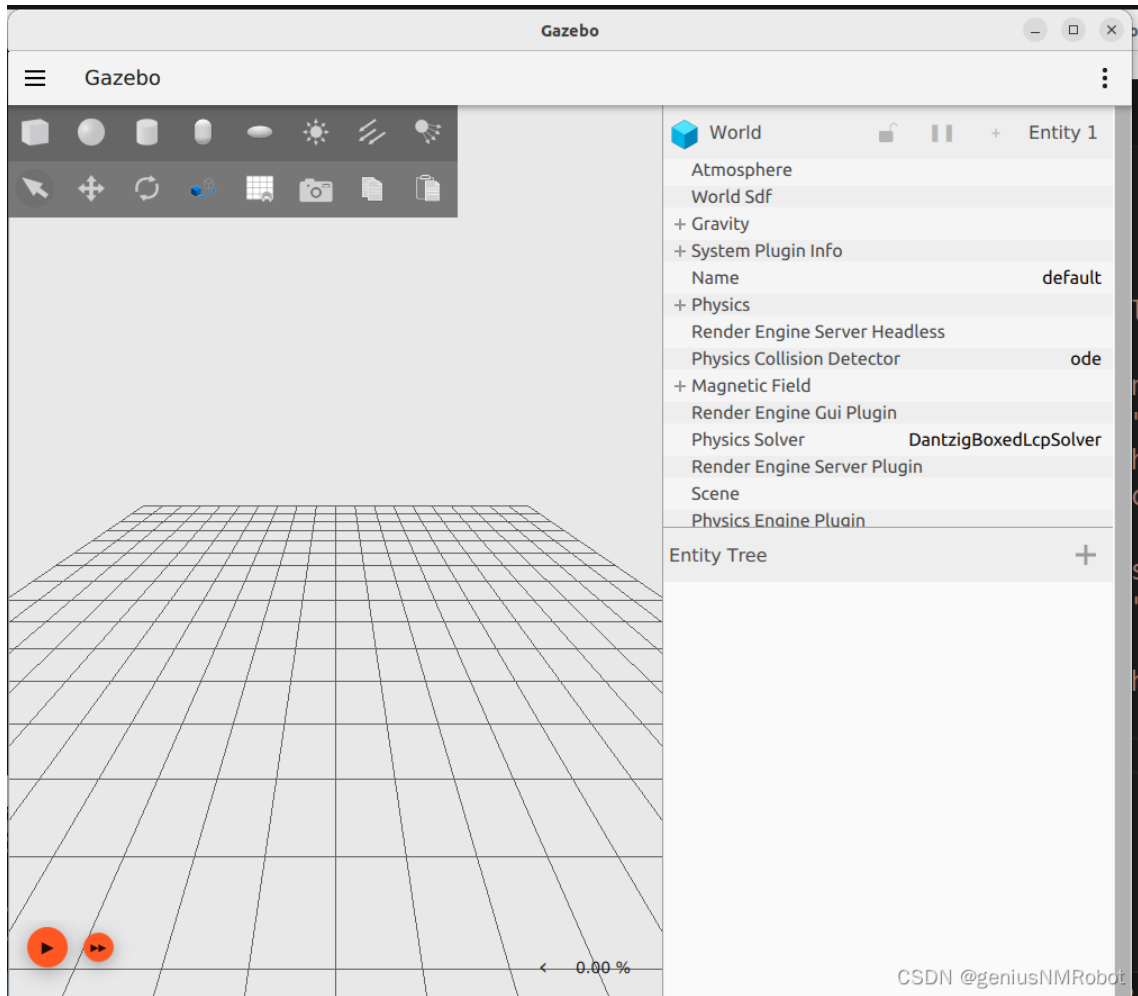


Figure 4.1: Gazebo Harmonic simulation interface

4.2 Proposed approach

Gazebo in combination with ROS can be used to create an interface between simulated robot behavior with formal verification tools, such as NuSMV. Formal methods allow researchers to rigorously verify properties of robotic systems—such as correctness, safety, liveness, and reachability —by constructing abstract models of their behaviors and validating them against logical specifications.

ROS-based systems define robotic structure and dynamics using URDF (Unified Robot Description Format) files and execute behavior through Python or C++ ROS nodes. These two sources—mechanical definitions and control logic—can be systematically translated into finite-state abstractions suitable for model checking with tools like

NuSMV.

The process typically involves analyzing the state transitions implied by the ROS node logic, especially within action sequences (e.g., part pickup, alignment , assembly), and discretizing them into states, inputs, and transitions. Each robot behavior or controller mode (e.g., "move", "grasp", "assemble") is represented as a finite state, while events and sensor conditions define transition rules. The mechanical configuration from URDF can define static properties or initial states in the system.

From this, a states diagram structure can be extracted, encoded in NuSMV's input language, and checked against temporal logic properties, described in CTL or LTL formulas. For instance, one could verify that a self-replicating robot always eventually reaches the assembly state (liveness) or that it never attempts to pick a module unless one is detected (safety).

Automating this translation—using Python scripts, that parse URDF and interpret ROS node behavior—is increasingly feasible, especially for systems designed with modular and rule-based architectures. Scripts or intermediate frameworks can analyze ROS bag files or logs to capture sequences of operations, and encode these as state transitions.

This capability enables hybrid verification workflows, where simulation is used to test behavior under dynamic conditions, and formal verification is applied to prove correctness across all possible system executions. In the context of self-replication, this allows verification of critical properties like "a complete replica is only activated after all modules are assembled" or "robots do not collide while cooperating on part assembly".

By linking Gazebo/ROS to NuSMV, can be created a powerful toolchain to not only test and observe behavior in simulation but also prove correctness and robustness in all logical conditions—an essential feature for deploying self-replicating systems in safety-critical or remote environments.

5 Implementation of model checking in SRRS

5.1 Subject description

In this research we will consider a simplified model of a relative modular robot and its corresponding environment, modeled with the potential for self-replication and based on the design, and material-robot principle, implemented by **Jenett2019MaterialRobot** [Jenett2019MaterialRobot] and **AbdelRahman2022Hierarchical** [AbdelRahman2022Hierarchical].

In our model, as in the original work, the robot, shown on Figure 2.9b) operates by manipulating and connecting parts, shown on Figure 2.9a) that are of the same type as its own building blocks. These parts serve as material for assembling different structures and may be distributed in the environment either in ordered arrangement or arbitrary manner. Despite their functional diversity, all blocks and parts are geometrically identical and take the form of cuboctahedral voxels, a general view of it is represented in Figure 5.1.

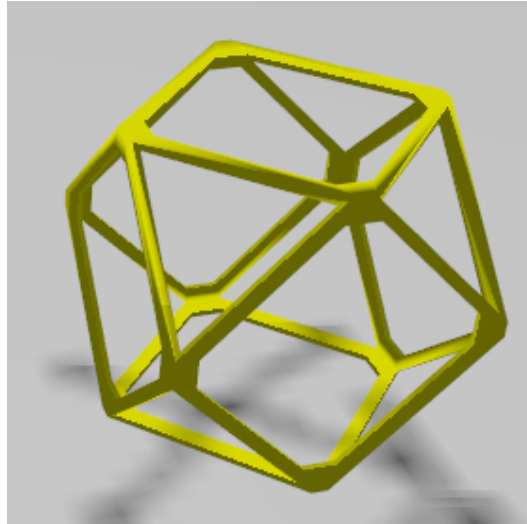


Figure 5.1: Material part or single robot block geometry in Gazebo simulation environment

5 Implementation of model checking in SRRS

Further in text, we refer to elements attached to the robot as "blocks" and to the same entities when distributed in the environment and not connected to the robot as "parts."

The robot is composed of a sequence of blocks of different functional types, an example of a robot is shown in Figure 5.2, where different types of blocks are marked with different colors.

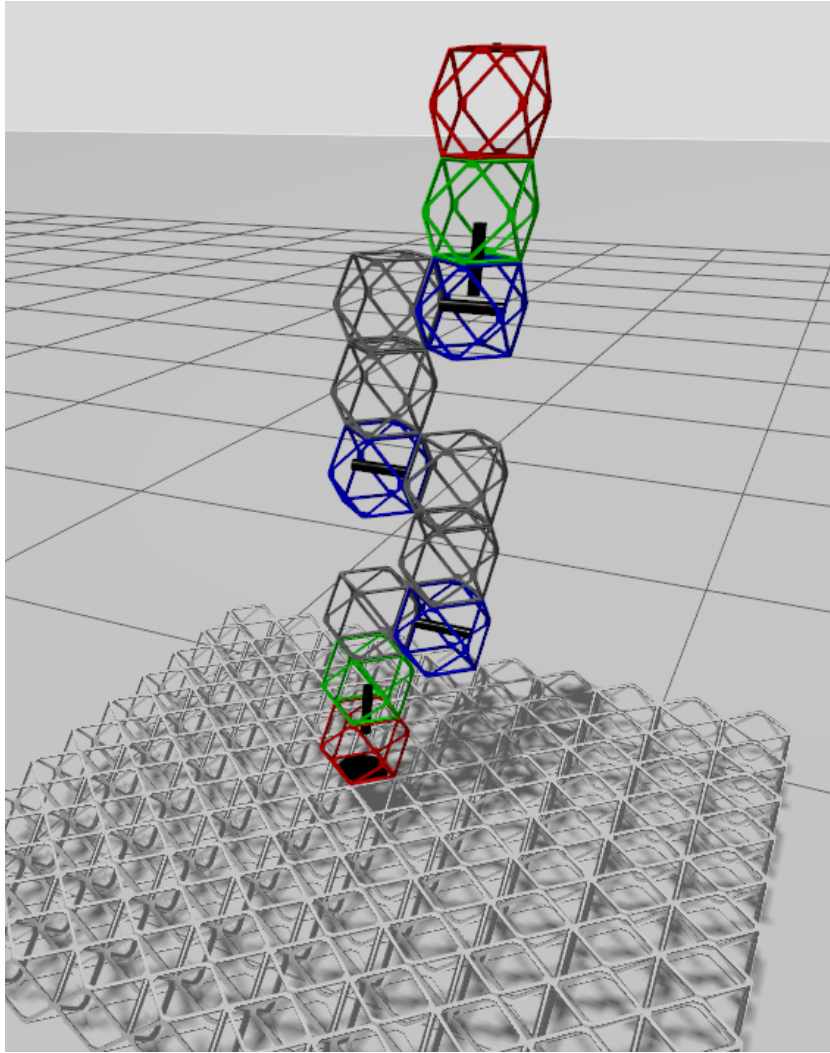


Figure 5.2: Robot structure in Gazebo simulation environment

The first and the last blocks of the sequence serve as end-effectors equipped with grippers, enabling the robot to interact with its environment. The intermediate blocks

5 Implementation of model checking in SRRS

constitute a rigid structure or act as joints, which may provide rotations around different axis, for example, enabling changing in yaw or pitch angles.

This uniform geometry facilitates a modular construction paradigm, where the same physical units can be rearranged to produce different structures and robot morphologies.

At any given time, robot in our model is capable of handling a single part, aligning it with the existing discrete environment and attaching it to some structure or new robot. Through repeated execution of this process, the robot can in principle assemble a replica of itself, thereby demonstrating the fundamental process of self-replication.

The robot operates within a structured environment defined as a lattice field that is geometrically compatible with the cuboctahedral voxel design this field represented as while base structure underneath the robot on a Figure 5.2. This lattice constrains both the robot and the individual parts to occupy discrete positions, ensuring that all spatial relations, and connections can be described in terms of transitions between well-defined integer coordinates. Such discretization not only simplifies the modeling of robot kinematics and assembly operations but also provides a natural basis for formal verification, since the system state space becomes finite and directly available for exploration by model-checking techniques.

In this chapter, we focus on methods to formally analyze the behavior of such systems. Model checking provides a rigorous framework for verifying whether the system satisfies specific properties of interest. In particular, we investigate the use of symbolic model checker - NuSMV, to evaluate both safety and liveness conditions for the modular robot using linear temporal logic (LTL) and branching temporal logic (CTL). Safety properties ensure that undesirable states, such as collisions or inconsistent connections, are never reached. Liveness properties, on the other hand, guarantee that the system is always capable of making progress, for example, that a new block can eventually be attached to a target or that a new robot can continue self-replication process.

Beyond simple verification, we also explore the potential of using NuSMV counterexample generation as a decision-making strategy. Counterexamples provided by the model checker correspond to concrete sequences of state transitions that lead to a violation of a previously negated property. Interpreted constructively, such sequences can serve as action plans or decision sequences for the robot, indicating feasible operations that allow it to achieve its objectives. This approach establishes a bridge between formal verification and automated planning and can increase the reliability and effectiveness of the system, and also add flexibility to decision-making in complex and dynamic conditions.

5.2 Analysis of the system

According to the classification given in Chapter 2.3.2 described self-replicating system for the general configuration of a robot can be classified as active system with hierarchical type of replication and capability for direct and indirect replications.

It has hierarchical replication type because in general it can produce robots capable of self-replication as well as robots or mechanical structures lacking of such capability.

It is active since it is directly engaged in constructing child robots.

The system can be homogeneous or heterogeneous based on initial conditions and programmed behavior. Homogeneous type of SRRS consist of robotically identical units, whereas heterogeneous enables generation of multiple robot types, with potential for specialization. Therefore, our system will be homogeneous if we will use only one type of robot with predefined structure that will replicate itself and heterogeneous if there will be multiple types on start or during replication.

In this chapter we will assume the system is homogeneous but will analyze effect of different types or configuration of robots in different environments on the properties of a system.

Using model proposed by **Lee2007Robotic** in [**Lee2007Robotic**] and described in Chapter 2.3.3 self-replicating process in our system can be formally represented as equation 5.1.

$$(R', M', E) = \Phi(R, M, E, t) \quad (5.1)$$

Where:

M – multiset (set with repetitions) of available parts (material) located in the environment to be used for building replicas robots by an initial system.

R – a multiset of initial robots to be reproduced. In our case one robot with predefined configuration. Each element of R includes specific robot structure in form of sequence of blocks, geometry, kinematic limitations and robot control system.

E – is a multiset of environmental structures and/or elements involved in the replication process (but not replicated). In our experiment is represented by base and optional obstacles.

R', M' – the new robots (or build structures) and the remaining and altered material respectively.

Φ – the replication function of time t .

This model is a special case of a Formula 2.2 for completely structured environment. $E = E'$ since each element of the environment (base) is either fixed at a precise location,

5 Implementation of model checking in SRRS

or described in integer coordinates and there will be no modification or change in any structures during the self-replication process.

This formulation enables modeling dynamic replication scenarios over time and accounts for state changes in materials and robots.

A central process is an act of self-replication – that is an assembly of a replica of itself by the initial robot from a set of parts located on a base. This process shown in state chart on Figure 5.3.

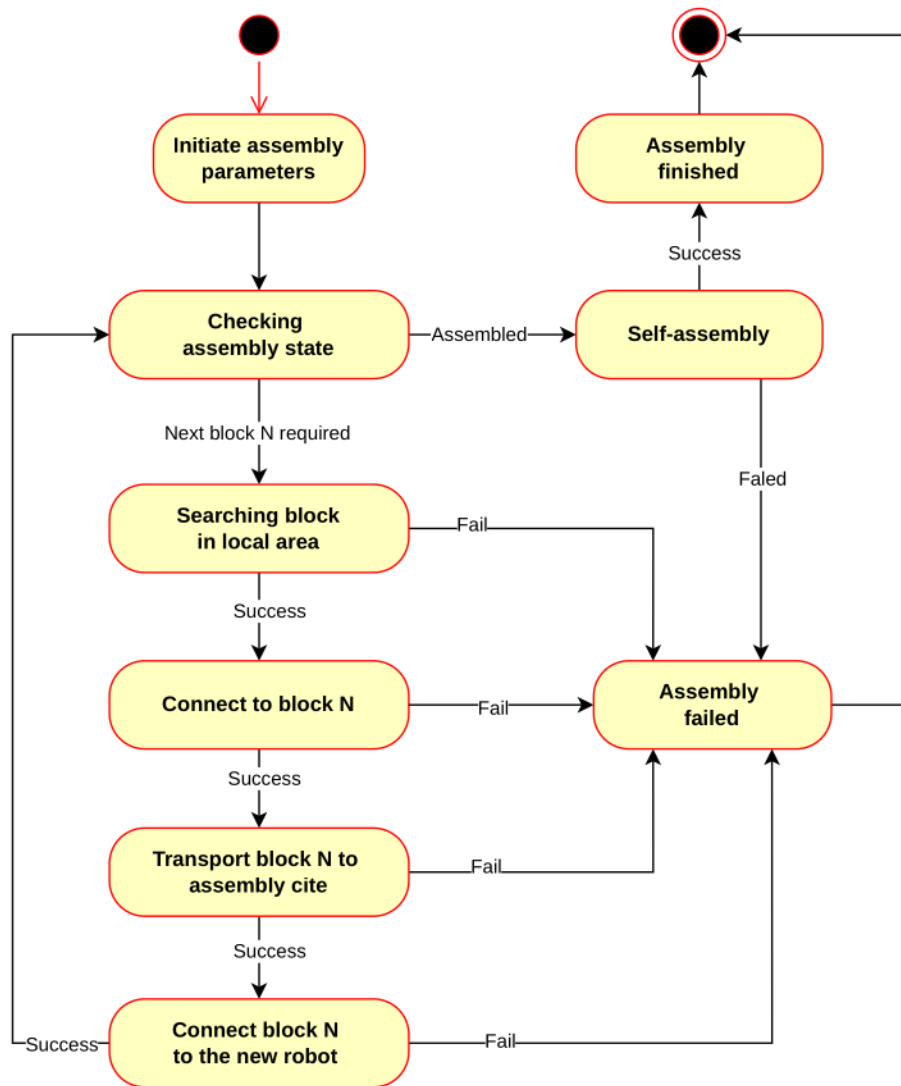


Figure 5.3: State chart of self-replication assembly process

5 Implementation of model checking in SRRS

Such process can be described as transition systems and is therefore suitable for model checking. It can be checked for specific requirements, written in form of temporal logic.

In this self-replication process we can derive following actions:

1. Connection and disconnection of the end-effector to and from the block or base.
2. Localization of the part, or receiving of part coordinates from the sensors. This action is a part of "Checking assembly state" and "Searching block in local area" in state chart.
3. Movement of the end-effector from one point to another, with or without connected parts.

Implementation of connection and disconnection actions is platform dependent be represented in a control system as boolean state.

Implementation of parts localization actions also depends on specific sensors configuration and similarly could be represented as a boolean array. Movement action depends mainly on the robot blocks configuration, blocks parameters, environmental limitations, initial and target points.

Model checking of generalized movement, in contrast, has a potential to substantially improve the overall self-replication process and adapt it for different environment conditions.

In the following sections we will model robot, translate self-replication process in transition system and check following properties using NuSMV:

- Check local reachability of some point or a set of points (region) by the robot end-effector.
- Generate a command sequence for a robot to move the end-effector to target point, based on counterexamples.
- Research global reachability of some point or region by robot end-effector using the walking form of movements.
- Generate a path to a point using walking based on counterexamples.
- Analyze set of robot configurations – check existing configuration or find relevant configurations of blocks based on counterexamples.

5.3 Model checking based control in local space

5.3.1 Overview

A basic verification step for our model consists in checking whether a given point in space is reachable by the robot's end-effector when the blocks are arranged in a specific initial configuration. Establishing such reachability opens the way for identifying potential stages of the assembly process, as illustrated in Figure 5.4.

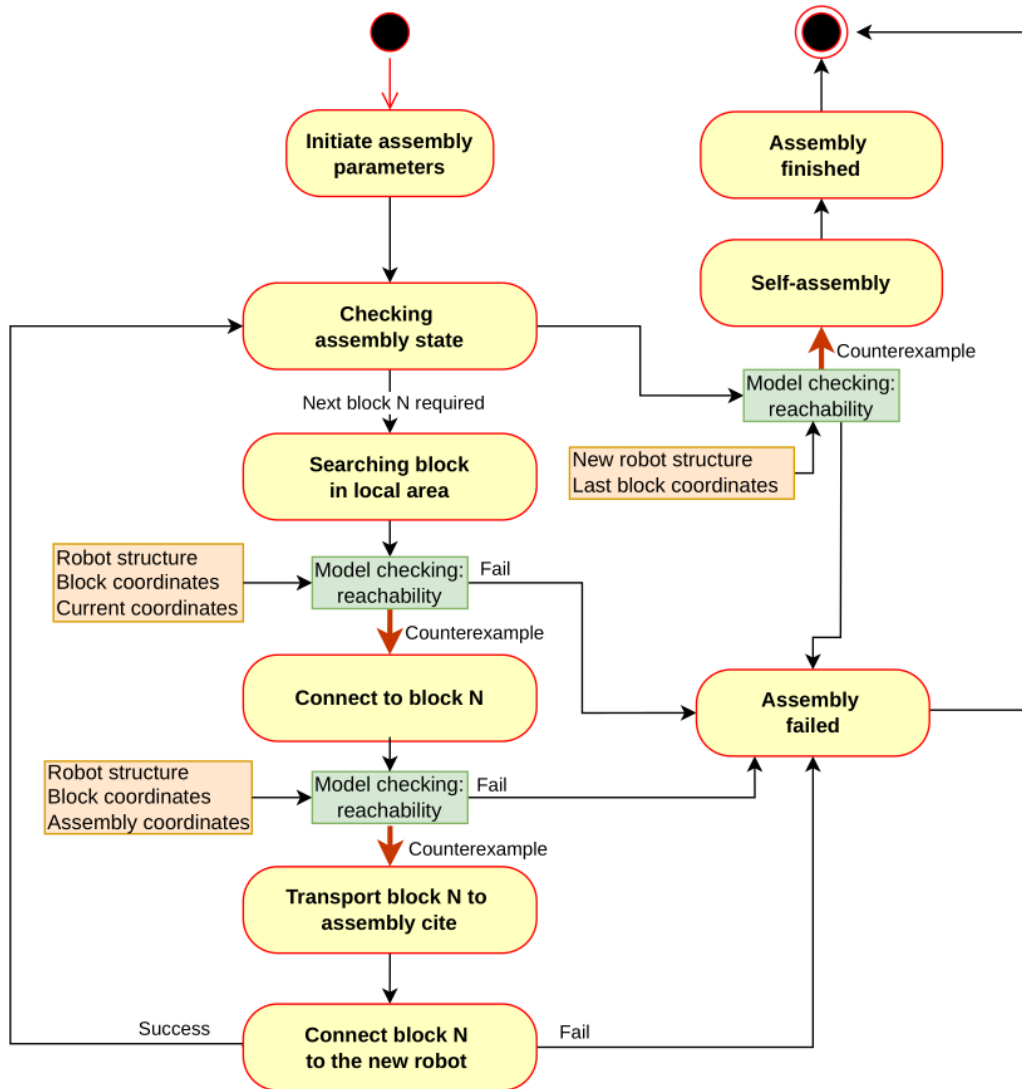


Figure 5.4: Self-replication process with potential model checking stages

5 Implementation of model checking in SRRS

In this context, model checking can be employed to confirm that the robot is capable of executing fundamental operations required for self-replication:

- moving the end-effector from its current location to the position of a target part,
- transporting the part to the designated assembly site,
- moving the newly attached end-effector to connect with the next required block.

In this chapter, we present a NuSMV model that encodes a chosen robot configuration, together with temporal logic specifications for verifying point and region reachability. Furthermore, we introduce Python-based tooling that derives counterexamples from the model checker and interprets them as executable control strategies for guiding the robot through the assembly process.

5.3.2 Robot NuSMV model

The robot structure as it shown in Figure 5.2 is represented as a modular composition of three distinct block types: a fixed base block, blocks with a yaw degree of freedom, and blocks with a pitch degree of freedom. Figure 5.5 illustrates the hierarchical arrangement of these blocks and the flow of coordinate and orientation data between them. This modular abstraction allows the robot structure to be described in a uniform and scalable manner, independent of the number of blocks.

TODO: 2. Correct diagram to SYSML block connection diagram. remove transition system

As the lowest level of a system, as it shown in Figure 5.5 we define a block. Each subsequent block introduces a transformation of the block end position according to its type. Each block has a certain type, in this work i consider three types of blocks:

The *block_base* defines the fixed starting point of the robot. It receives as input the global origin coordinates (x_{orig} , y_{orig} , z_{orig}) and the initial orientation parameters (yaw_{orig} , $pitch_{orig}$) Its role is to provide a reference frame for all subsequent blocks. The base block outputs the coordinates (x_{end} , y_{end} , z_{end}) and orientation values that are propagated forward to the next block in the sequence.

Block types *block_yaw* realise rotation around local axis Z, that oriented along the parent block The *block_yaw* represents a joint capable of rotating around the vertical axis. Internally, the block maintains a discrete variable ϕ , which denotes the yaw angle.

As illustrated in the Figure 5.6, ϕ can evolve in steps of +1 or -1, modulo 36, corresponding to 10-degree discretization over the full circle of 360°. Thus, the block admits cyclic

5 Implementation of model checking in SRRS

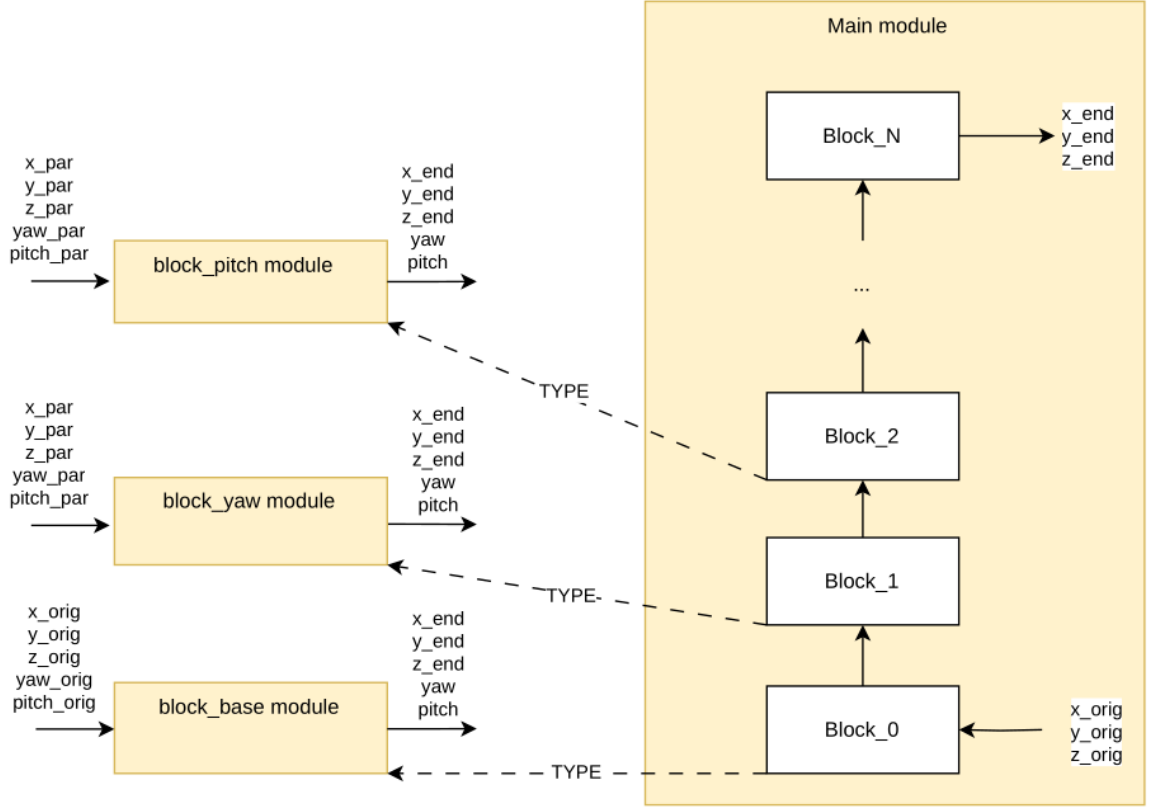


Figure 5.5: Robot configuration NuSMV model

transitions that allow continuous rotation in both clockwise and counterclockwise directions. The block takes as input the parent coordinates $(x_{par}, y_{par}, z_{par})$ and orientation $(yaw_{par}, pitch_{par})$, applies the yaw rotation, and produces updated endpoint coordinates.

$$\begin{cases} \phi \rightarrow \phi \\ \phi \rightarrow (\phi + 1) \bmod 36 \\ \phi \rightarrow (\phi - 1) \bmod 36 \end{cases} \quad (5.2)$$

Block types *block_pitch* realise rotation around local axis Y, that is perpendicular to Z often the parent block. Similarly, the *block_pitch* encodes a joint that rotates around the lateral axis of the preceding link. Its discrete state variable θ :

$$\begin{cases} \theta \rightarrow \theta \\ \theta \rightarrow (\theta + 1) \bmod 36 \\ \theta \rightarrow (\theta - 1) \bmod 36 \end{cases} \quad (5.3)$$

5 Implementation of model checking in SRRS

In combination, these transitions capture the continuous ability to rotate up or down by discrete steps.

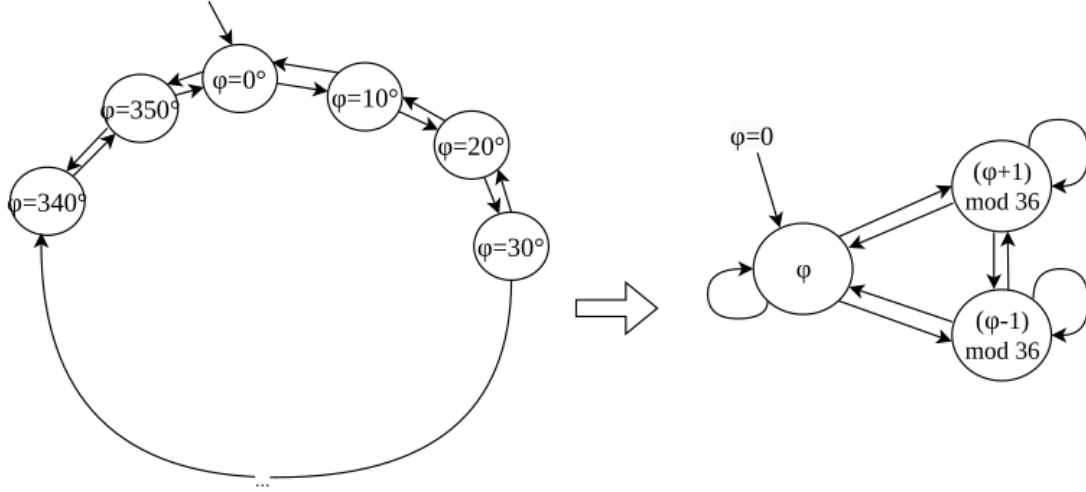


Figure 5.6: Robot block transition system for angle ϕ

At the system level, the robot is built as a chain of such blocks. After the base block, one may attach any sequence of yaw and pitch blocks in arbitrary order, depending on the desired manipulator structure. The right-hand side of Figure 5.5 shows an example with N chained blocks, where Block 0 connects directly to the base and Block N defines the robot's end-effector. The outputs of the last block are new coordinates $(x_{end}, y_{end}, z_{end})$ and updated orientation values. Each block is parameterized by the output of its predecessor, creating a cascading dependency structure.

Thus, on the NuSMV model description level we have:

- main block module - that describes a robot structure as a sequence of block types as variables in VAR section.
- block_base module - describing initial block of a sequence and locating on a ground plane.
- block_yaw module - describing rotation around the parent Z axis (along the block) and changing the yaw angle, the NuSMV code for this module is shown in Listing 5.1.
- block_pitch module - describing rotation around the parent Y axis (perpendicular to the block) and changing the pitch angle.

5 Implementation of model checking in SRRS

```

1 MODULE block_yaw(px, py, pz,
2     xhx, xhy, xhz, yhx, yhy, yhz, zhx, zhy, zhz,
3     L, initYaw)
4 VAR
5     yaw : 0..35;
6     cy : cos(yaw);
7     sy : sin(yaw);
8
9 DEFINE
10    SCALE := 100;
11
12    cos_yaw := cy.val;    -- [-100..100]
13    sin_yaw := sy.val;
14
15    xhx2 := (xhx * cos_yaw - xhy * sin_yaw) / SCALE;
16    xhy2 := (xhx * sin_yaw + xhy * cos_yaw) / SCALE;
17    xhz2 := xhz;
18
19    yhx2 := (yhx * cos_yaw - yhy * sin_yaw) / SCALE;
20    yhy2 := (yhx * sin_yaw + yhy * cos_yaw) / SCALE;
21    yhz2 := yhz;
22
23    zhx2 := zhx;
24    zhy2 := zhy;
25    zhz2 := zhz;
26
27    px2 := px + (zhx2 * L) / SCALE;
28    py2 := py + (zhy2 * L) / SCALE;
29    pz2 := pz + (zhz2 * L) / SCALE;
30
31
32 ASSIGN
33     init(yaw) := initYaw;
34     next(yaw) := { yaw, (yaw+1) mod 36, (yaw+35) mod 36 };

```

Listing 5.1: NuSMV code for block_yaw module

MuSMV modules block_base, block_pitch sin, and cos NuSMV modules shown in Appendix A. This design supports a natural inductive description of the robot in NuSMV: the same block type can be instantiated repeatedly, with only the input-output connections altered.

The modularity of the design has two important advantages.

First, it allows scalability, since additional joints can be incorporated simply by adding new instances of yaw or pitch modules.

Second, it supports module based model checking, because each block is described as a transition system with a finite number of states and transitions. The global state space of the robot is then obtained by synchronous product of the component transition systems, which can be directly analyzed using LTL or CTL specifications in NuSMV.

The dynamics of each yaw or pitch block can be represented as a finite-state transition system. Figure 5.6 illustrates this equivalence explicitly. On the left-hand side, the system is shown as a ring of discrete states corresponding to the possible angle

5 Implementation of model checking in SRRS

```

1 ASSIGN
2   ...
3   init(yaw)    := 0;
4   next(yaw)    := {
5                   yaw,
6                   (yaw+1) mod 36,
7                   (yaw+35) mod 36
8                   };

```

Listing 5.2: Assign part of code for block transitions

values

$$\phi = 0^\circ, 10^\circ, \dots, 350^\circ$$

From each state, there are transitions to the neighboring states at $\phi + 10^\circ$ and $\phi - 10^\circ$ thus modeling incremental rotations in either direction. Since the variable is defined modulo 36, the state $\phi = 350^\circ$ transitions back to $\phi = 0^\circ$, completing the cycle. The result is a finite cyclic transition system with 36 nodes, which represents the discretized continuous rotation of the block.

On the right-hand side of Figure 5.6, this same behavior is captured more compactly. Here, the internal state variable ϕ is updated according to transition rules:

$$\phi = \phi \quad \vee \quad \phi = (\phi + 1) \bmod 36 \quad \vee \quad \phi = (\phi - 1) \bmod 36$$

This concise representation preserves the semantics of the explicit ring but makes the module description more compact and scalable. When combined with coordinate update formulas, it fully captures the contribution of the block to the robot's kinematics.

This transitions and angle changing for each block reflected by the following part of assign block, shown in Listing 5.2.

For each block in a sequence of robot structure the initial value is assigned, and next state can be chosen nondeterministically from three possible transitions.

The full NuSMV model for modules: `block_base`, `block_yaw`, `block_pitch` and `main` presented in Appendix A.

5.3.3 Reachability specifications

Point reachability checking specifications:

In order to verify whether the robot is capable of reaching a desired target point with its free end-effector, in three-dimensional space, we formalize the reachability check directly within the NuSMV model.

In the DEFINE section we provide target point, end-effector coordinates variable and define error threshold, this code fragment is shown in Listing 5.3.

```

1 DEFINE
2 ...
3 error := 1;
4 checkX := 10;
5 checkY := 15;
6 checkZ := 25;
7 endX_inside_limits := (endX <=(checkX+error) & endX >=(checkX-error));
8 endY_inside_limits := (endY <=(checkY+error) & endY >=(checkY-error));
9 endZ_inside_limits := (endZ <=(checkZ+error) & endZ >=(checkZ-error));

```

Listing 5.3: End effector and target point coincidence condition

Here, the variables *endX*, *endY*, and *endZ* represent the Cartesian coordinates of the robot end-effector, which are computed elsewhere in the model based on the joint angles and link lengths.

The parameters *checkX*, *checkY*, and *checkZ* denote the coordinates of a target point that we wish to test for reachability.

Since we can use only countable number of states and have to represent angles as integers, each transformation of angles to linear coordinates is not precise. Correspondingly the distance between actual and target points is between zero and linear delta calculated based on discretization value for angle. For each spatial dimension, Boolean formulas *endX_inside_limits*, *endY_inside_limits*, and *endZ_inside_limits* check whether the end-effector coordinate lies within a small interval around the corresponding target value, effectively defining a cube of side length 2, centered at the target.

The formal property is expressed in Linear Temporal Logic (LTL) in Listing 5.4. states that on every execution path, it will eventually (F) be the case that the robot's end-effector is within the specified tolerance region around the target point. In other words, the property enforces that the end-effector eventually can reach the target zone.

5 Implementation of model checking in SRRS

```
1 LTLSPEC
2 F (endX_inside_limits & endY_inside_limits & endZ_inside_limits)
```

Listing 5.4: Reachability condition in LTL specification

The formal property is expressed in Computation Tree Logic (CTL) in Listing 5.12.

```
1 CTLSPEC
2 EF (endX_inside_limits & endY_inside_limits & endZ_inside_limits)
```

Listing 5.5: Reachability condition in CTL specification

In this specification *EF* means "there exists a path along which eventually holds following condition".

Region reachability checking specifications:

We can check reachability of some 3D region, in our case a box by defining the variables checkX, checkY and checkZ as FROZENVAR. FROZENVAR section in NuSMV declares parameters that are chosen nondeterministically at starting time and then never change along a run for an entire execution (trace). The Listing 5.6 represents an example of a region:

$x = [-10, 10]$, $y = [-10, 10]$, $z = [0, 20]$.

```
1 FROZENVAR
2 checkX : -10..10;
3 checkY : -10..10;
4 checkZ : 0..20;
```

Listing 5.6: Region definition for a reachability checking

Changing in the specification for the region checking using CTL logic shown in Listing 5.12 is not necessary. With the frozen variable region, the formula states: from every initial state (for every admissible target chosen at $t=0$), there exists a sequence of moves that eventually places the end-effector inside the region.

Because NuSMV checks CTL properties over all initial states, this specification proves reachability for every target in the declared ranges (universal over targets, existential over paths).

These specifications therefore provides a rigorous, formally verifiable mechanism to test spatial reachability of robotic manipulators in discrete state-space models. It can be generalized by altering the checkX, checkY, and checkZ parameters, and by varying the tolerance error, to systematically analyze the reachable workspace of the system.

5.3.4 Counterexample processing

Based on specifications described in previous section we use the modeling pattern in NuSMV for reachability: “eventually reach point P”, and negate that requirement to force the model checker to produce a counterexample that reaches target point P.

In our self-replication robotic system this pattern can be implemented to obtain plausible traces on following stages of the self-replication process:

1. To define the trajectory from current point to new part.
2. To define the trajectory of transportation of the part to an assembly point.
3. To define the trajectory for new robot to connect to the last part.

These stages shown on the Figure 5.7, where input parameters shown in orange squares and counterexamples depicted with red arrow.

The formal property is expressed in Linear Temporal Logic (LTL) in Listing 5.7. states that on every execution path, it will always (G) be the case that the robot’s end-effector is not within the specified tolerance region around the target point. In other words, the property enforces that the end-effector infinitely often leaves the target zone, meaning the robot cannot remain in that region permanently.

```
1 LTLSPEC
2 G !(endX_inside_limits & endY_inside_limits & endZ_inside_limits)
```

Listing 5.7: Reachability condition in LTL specification

The formal property is expressed in Computation Tree Logic (CTL) in Listing 5.8.

```
1 CTLSPEC
2 AG !(endX_inside_limits & endY_inside_limits & endZ_inside_limits)
```

Listing 5.8: Reachability condition in CTL specification

The specification:

$$AG \neg (endX_inside_limits \wedge endY_inside_limits \wedge endZ_inside_limits)$$

is interpreted as: “On all computation paths (A), in all states globally (G), it is never the case that the end-effector simultaneously satisfies all three coordinate bounds.” In other words, if the target point defined as in Listing 5.3, the model checker proves that the robot can never reach the point (10,15,25) within the given tolerance.

Conversely, if the property is violated, NuSMV produces a counterexample trace that demonstrates a specific sequence of joint configurations leading the end -effector into

5 Implementation of model checking in SRRS

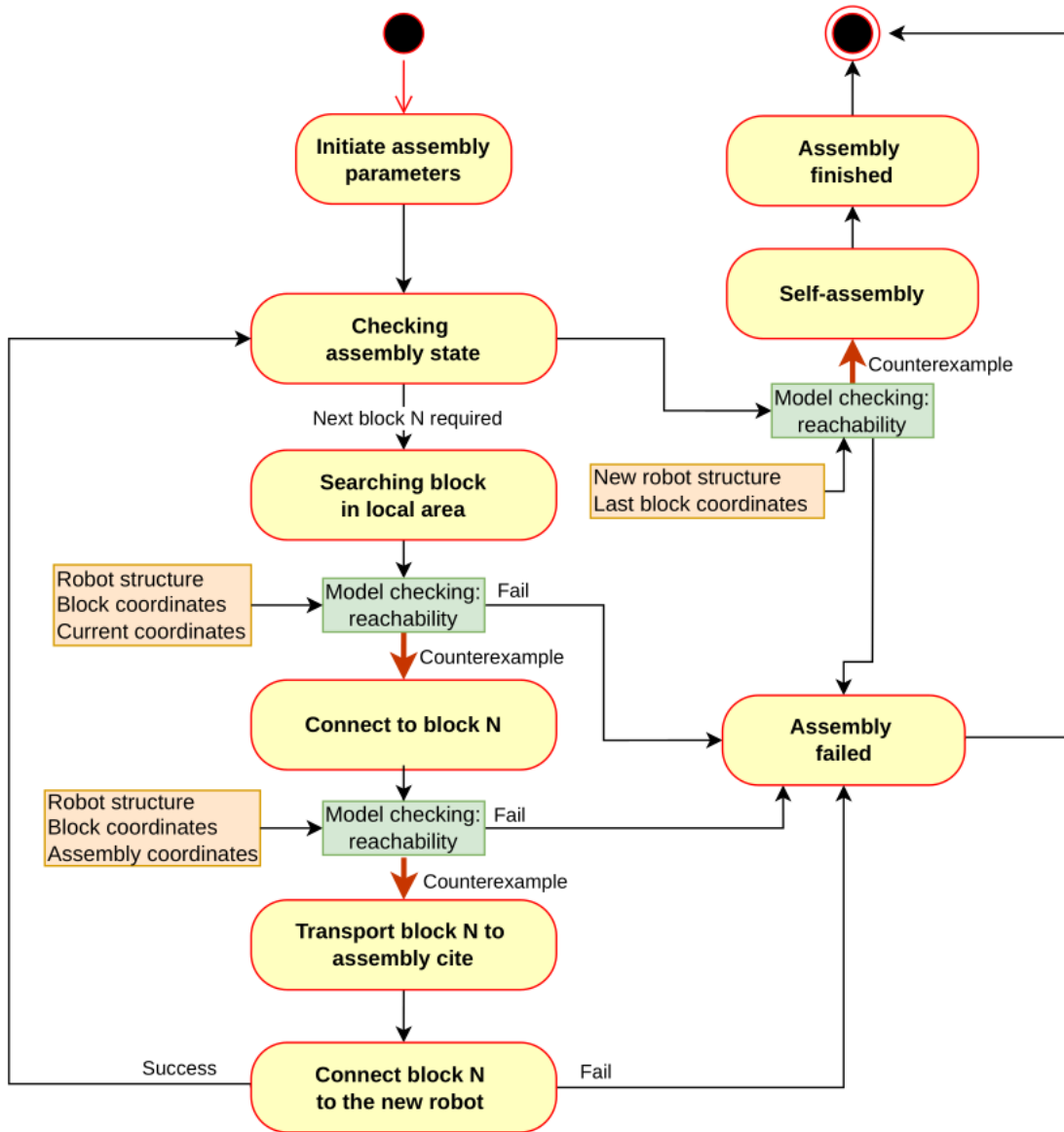


Figure 5.7: Local assembly process without model checking

the defined target region. This counterexample can be directly interpreted as a feasible motion plan achieving the desired reachability.

The implementation of this process centers on the Counterexample class in python and a set of utilities for filtering variables, reconstructing geometry, and plotting results. This class parse NuSMV counterexample, extracts the variables of interest (end-effector coordinates, per-block coordinates, and joint angles), and provides means to render the

5 Implementation of model checking in SRRS

counterexample trace in 3D graph. It also exposes numerically clean sequences that can hand to a controller.

Counterexample class shown in Appendix A.2.1 it takes raw NuSMV output and exposes methods to transform it into structured sequences for visualization and control. The source file is short—easy to audit and adapt—but covers the full parsing loop: reading states, carrying forward unchanged values, projecting specific variables, computing angles in degrees, reconstructing robot geometry step-by-step, and plotting.

Parsing the counterexample text presents a challenge because NuSMV prints only the variables that change between consecutive states. To reconstruct complete state vectors, the `filter_variables()` function employs a carry-forward mechanism. It begins with a list of variable names of interest, such as the coordinates of each block or the end-effector, and then iterates over the textual representation of each NuSMV state. When a variable assignment is found, its value is updated in the active state vector; otherwise, the previous value is retained. This mechanism ensures that every time step in the resulting sequence contains a full snapshot of all relevant variables, even those that remained constant. Once the entire sequence is reconstructed, the program converts the data into numerical form, yielding a well-defined matrix of dimension $\text{steps} \times \text{variables}$ suitable for quantitative analysis. The resulting data provide a clear temporal evolution of the robot configuration.

When angular values are needed, the same mechanism is applied to orientation variables in function `get_angles()`, which are scaled by a discretization factor to convert symbolic or integer representations into physical degrees. This transformation connects the abstract state space of the model checker to the metric quantities used in real robot kinematics.

Beyond parsing, the class includes visualization function `plot_trace()` that convert the numeric data into three-dimensional figures. These plots display the complete end-effector path as well as the robot's configuration at selected steps, an example of the plot is shown in Figure 5.8. Each step is represented as a connected chain of block positions, preserving the spatial structure of the model. The visualization enforces uniform scaling across all axes to maintain geometric fidelity, and the plotted points are labeled to correspond with discrete time indices.

All paths and parameter values, such as the number of robot modules or the NuSMV executable location, are configurable. The implementation follows a transparent design that favors explicitness over abstraction, which makes it accessible to researchers who wish to adapt it to new model structures or naming conventions.

Once the counterexample has been parsed and verified, the data can be integrated into a robotic control pipeline.

5 Implementation of model checking in SRRS

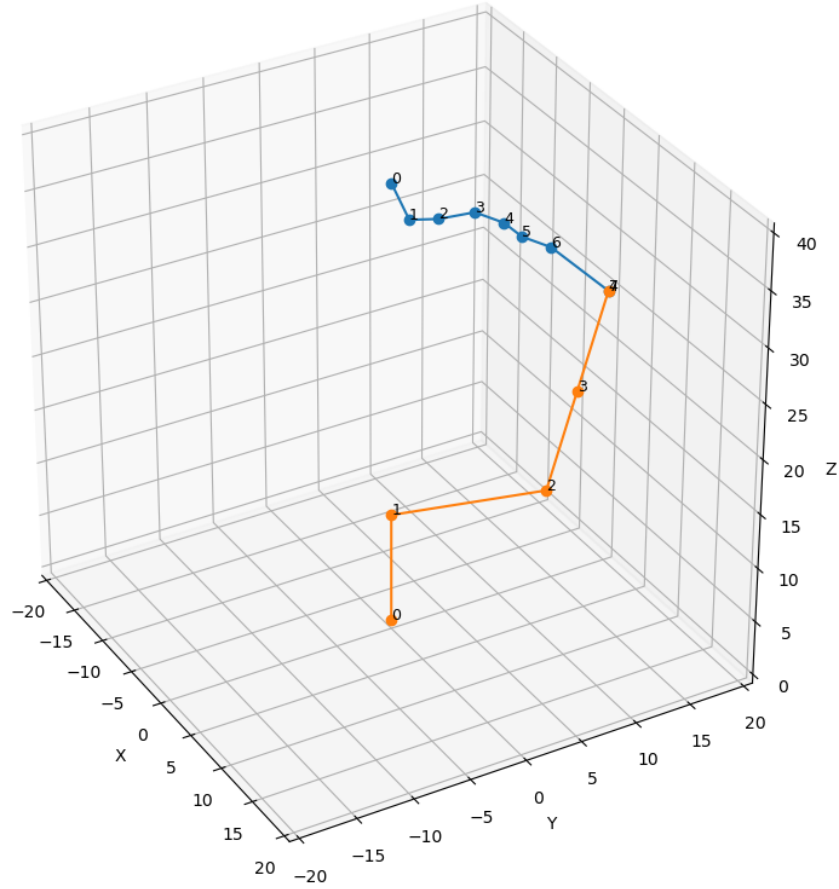


Figure 5.8: Example trace (blue) plot for target point coordinates (10,13,30) and last trace robot position (orange)

Data from counterexample can be used directly as open-loop trajectories, in which each time step of the counterexample corresponds to a discrete command for the robot's end-effector or joint configuration. They may also serve as reference paths for closed-loop control (with feedback from sensors), allowing the robot to reproduce formally verified behaviors under physical constraints.

A typical workflow begins with the definition of the robot structure and target point in a NuSMV model file. The property under verification expresses the negation of the desired reachability condition – formally, the statement that the robot can never reach the point. If NuSMV finds that this negated property is false, it outputs a counterexample that shows precisely how the robot can, in fact, reach the point. The Python parser then extracts from this output the sequence of coordinates or angles, which can be sent to the robot controller as a reference trajectory. Visualization confirms that the

motion reaches the target, and further analysis can evaluate the dynamic parameters, adjust path smoothness, or joint feasibility of the resulting motion.

By systematically reconstructing state trajectories from NuSMV output, visualizing them, and exposing them for control, the Counterexample class enables the practical reuse of verification results in robotic system experiments. Its structure is extensible for larger models, alternative naming conventions, or integration into simulation.

5.3.5 Evaluation

TODO: plan experiments: 1. Test with predefined assembly process (search, move, repeat) with random distribution of the parts
2. Test with predefined assembly process (...) with, criteria and analysis Criteria: success rate, average time, disagreement with control alg

Experiment 1:

Subject: local assembly with shortest path control algorithm

Setup: random distribution of blocks and obstacles in 3D (for 0-15 obstacles) Measure values: average assembly time, average success rate

Experiment 2:

Subject: local assembly with path control algorithm and runtime verification

Setup: random distribution of blocks and obstacles in 3D (for 0-15 obstacles) Measure values: average assembly time, average success rate

Experiment 3:

Subject: local assembly with runtime verification and counterexample based control algorithm

Setup: random distribution of blocks and obstacles in 3D (for 0-15 obstacles) Measure values: average assembly time, average success rate

5.4 Robot configuration model checking

In this part we will check the model of the robot of the certain configuration for the reachability property introduced in

Criteria: reachability of the random distributed blocks, given target point position Validation: optimal structure -> generate xacro, check assembly

5.4.1 Sequential checking of possible configurations

In order to systematically evaluate which robot structures are capable of reaching a target coordinates, implemented an automated configuration checking procedure that integrates model generation, model checking with NuSMV, and result filtering. The overall workflow is illustrated in Figure 5.9.

The process of checking robot configurations is performed by integrating Python -based model generation with the NuSMV model checker. The workflow begins with two primary inputs: the set of available block types, such as base, yaw-rotating, and pitch-rotating blocks, and the maximum robot length expressed as the number of blocks. These inputs are used to generate the complete set of allowed configurations of length N , ensuring that every possible arrangement of building blocks within the specified limit is considered. Each configuration is represented as an ordered sequence of blocks, starting with the mandatory base block and followed by a selection of yaw and pitch blocks. Once the set of configurations is created, the process iterates over each configuration individually.

The block sequence is extracted and combined with:

- coordinates of an initial block of a robot $(x_{orig}, y_{orig}, z_{orig})$
- assembly site coordinates relative to a robot $(x_{asm}, y_{asm}, z_{asm})$
- target block coordinates relative to a robot: [Block(x_1, y_1, z_1), Block2(x_2, y_2, z_2), ...]

This information forms the basis for generating a concrete NuSMV model through a Python script. The script instantiates the appropriate modules corresponding to each block type, establishes the connections between their input and output variables, and integrates the positional and orientation parameters across the robot chain. In addition, the script embeds the temporal logic specifications provided as LTL or CTL formulas. These specifications define the verification goals, such as whether the end-effector can reach a given target coordinates and avoids certain unsafe areas.

5 Implementation of model checking in SRRS

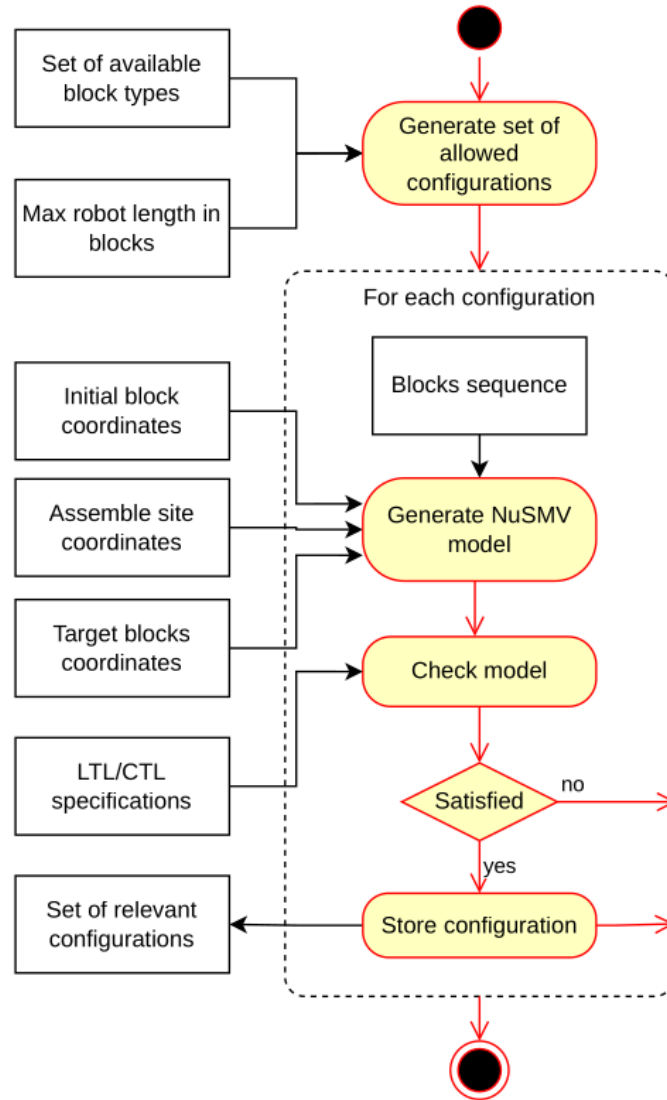


Figure 5.9: Activity diagram of searching for relevant configurations

The generated model is then passed to NuSMV, which executes symbolic model checking to evaluate whether the specifications are satisfied. The model checker systematically explores the entire state space of the configuration, taking into account the discrete transitions of yaw and pitch blocks as well as the coordinate propagation along the chain.

If the specification is not satisfied, the algorithm discards the configuration and proceeds to the next candidate in the sequence.

If the specification is satisfied, the configuration is stored in a set of relevant configura-

rations, which represents the final output of the process.

5.4.2 Implementation in Python

The implementation of the workflow described in Figure 5.9 was performed in configurations checking system written in Python. The architecture of the design corresponds directly to the class diagram shown on the Figure 5.10, where the class *Configuration_checking* orchestrates the process, the class *Robot* assembles a sequence of blocks, object of the class *Block* encapsulates the description of an individual block.

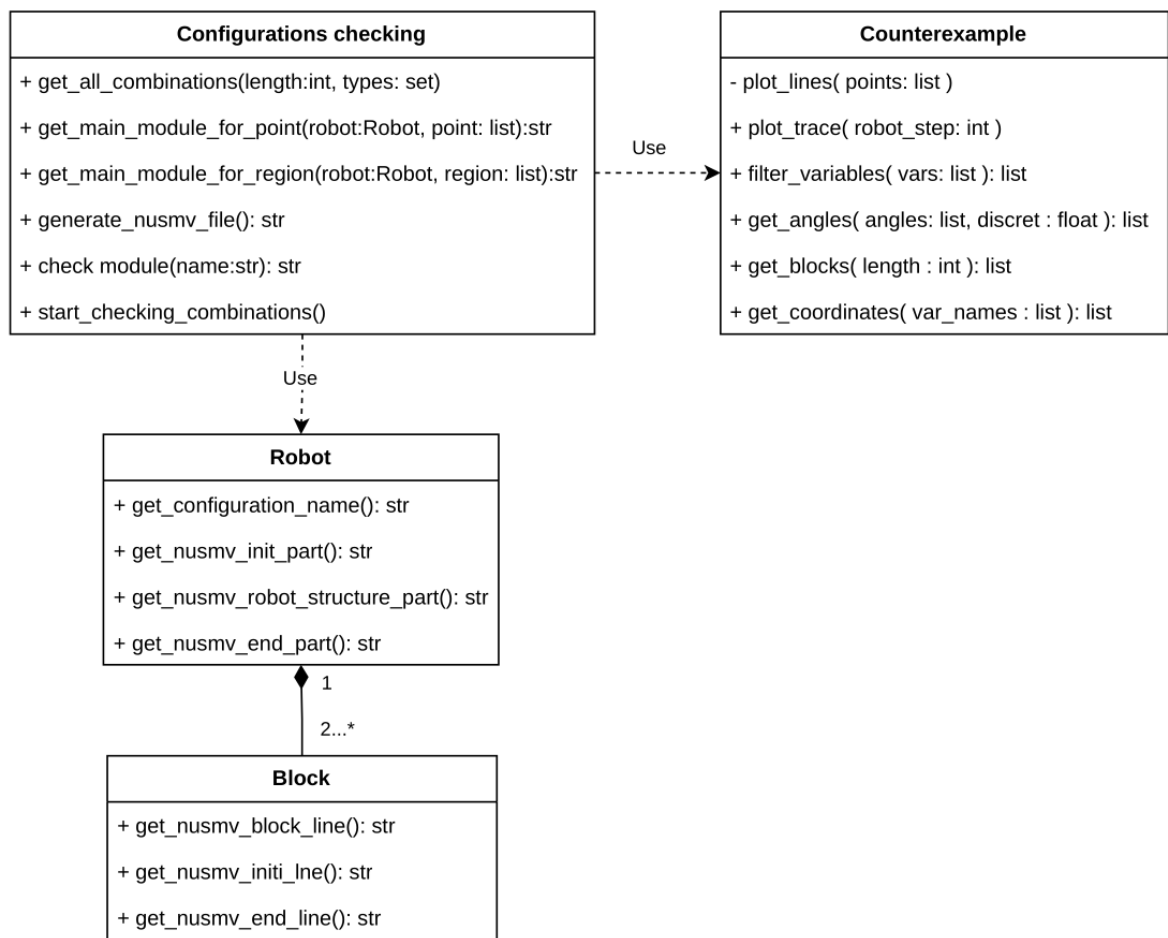


Figure 5.10: Class diagram of searching for relevant configurations

A *Block* class is the minimal self-contained representation of a kinematic segment in the robot block sequence. Each block is identified by an integer number and a type

5 Implementation of model checking in SRRS

such as “base”, “yaw”, or “pitch”, and it stores the index of its parent block in order to define the topology and orientation. Geometric information is reduced to a single parameter length, while all pose and orientation information is described later through the NuSMV model. The block provides three critical outputs in the form of NuSMV code fragments: a structure line that instantiates the variable with appropriate block module and wires it to its parent, an initialisation line that sets link length and default joint angle values, and, for the final block, assignment lines that expose the end-effector coordinates. For non-base blocks, angle variable names are generated systematically (e.g., yaw2, pitch3), guaranteeing uniqueness and readability. In this way, each block fully describes its structural role without knowledge of the entire robot, making the class simple and robust.

The Robot class acts as a container and assembler for blocks. Constructed from a list of block types, it automatically creates a chain of Block instances, numbers them, and links each to its parent. It maintains the list of blocks as well as a handle to the final block, which represents the robot’s end-effector. The Robot then concatenates the fragments provided by its blocks into complete sections suitable for insertion into a NuSMV model. It can return the structure of the chain as a sequence of module instantiations, the initialisation of link lengths and angles, and the assignment of global end-effector coordinates. Additionally, it encodes the configuration in a readable name such as “base_yaw_pitch_yaw”, which is used both in logging and file management. The Robot therefore translates an abstract sequence of types into a structured collection of blocks, and from there into concrete model code that describes the robot in full.

The third class, Configuration_checking, governs the entire process. Its role is to generate and test all possible configurations of a specified chain length composed of a given set of block types. The first block is always fixed to a base, and the remaining positions are filled with every possible combination of the admissible types. This results in an exponential search space, for example two types and four positions yielding sixteen unique robots. Configuration_checking takes as additional input a target point in space and stores both this point and the error tolerance around it, against which reachability is later checked. For each generated morphology, the class constructs a Robot, retrieves its NuSMV code, and splices it into a template file. The method `create_nusmv_from_template` searches for the region between “MODULE main” and a predefined end marker, replaces that region with a new main module generated for the current robot, and leaves the rest of the file untouched. This ensures that all library code, specifications, and fairness constraints in the template remain intact. The new module contains the structure lines from the robot, the initialisation of lengths and angles, the fixed base pose and basis vectors, the end-effector definitions, and the error-band predicates that compare the end coordinates to the desired target. This lay-

5 Implementation of model checking in SRRS

ering is explicit in the class diagram Figure 5.10: Configuration_checking depends on Robot, which is composed of Block instances.

Instead of hard-coding a complete NuSMV model, the system rewrites the MAIN module fragment inside a reusable template file. This makes the pipeline resilient to changes in property specifications or library modules—the template remains the source of truth, while the classes inject only the structure and constants.

The overall workflow combines automatic model generation, model checking, and storing of satisfiable robot configurations. By using a Python script to handle tasks of configuration expansion and NuSMV code generation, the approach ensures scalability and reduces the potential errors. The inclusion of temporal logic specifications provides a basis for determining whether a configuration is functionally capable of achieving the task requirements. The final set of relevant configurations captures only those robot designs that satisfy the given conditions, providing a reliable foundation for further analysis or deployment in robotic applications.

The result of the robot with length 4 and 5 blocks for reachability of point $x = 5, y = 5, z = 10$ shown in the Listing 5.9

```
1 Checking self.X=5, self.Y=5, self.Z=10
2 self.length=4
3 number of combinations:8
4 1/8:base_pitch_pitch_pitch_: not reachable [0.377735 sec]
5 2/8:base_pitch_pitch_yaw_: not reachable [0.039332 sec]
6 3/8:base_pitch_yaw_pitch_: not reachable [0.576335 sec]
7 4/8:base_pitch_yaw_yaw_: not reachable [0.01432 sec]
8 5/8:base_yaw_pitch_pitch_: reachable [0.242966 sec]
9 6/8:base_yaw_pitch_yaw_: not reachable [0.034844 sec]
10 7/8:base_yaw_yaw_pitch_: not reachable [0.062835 sec]
11 8/8:base_yaw_yaw_yaw_: not reachable [0.013127 sec]
12
13 Checking self.X=5, self.Y=5, self.Z=10
14 self.length=5
15 number of combinations:16
16 1/16:base_pitch_pitch_pitch_pitch_: not reachable [15.916151 sec]
17 2/16:base_pitch_pitch_pitch_yaw_: not reachable [0.344371 sec]
18 3/16:base_pitch_pitch_yaw_pitch_: not reachable [35.386106 sec]
19 4/16:base_pitch_pitch_yaw_yaw_: not reachable [0.041149 sec]
20 5/16:base_pitch_yaw_pitch_pitch_: reachable [24.917136 sec]
21 6/16:base_pitch_yaw_pitch_yaw_: not reachable [0.727521 sec]
22 7/16:base_pitch_yaw_yaw_pitch_: not reachable [1.013776 sec]
23 8/16:base_pitch_yaw_yaw_yaw_: not reachable [0.028078 sec]
24 9/16:base_yaw_pitch_pitch_pitch_: reachable [4.10417 sec]
25 ...
```

Listing 5.9: The output of configuration model checking for lengths of robot 4 and 5 and target region $x=-5..5, y=-5..5, z=0..5$

5 Implementation of model checking in SRRS

We found one configuration for the robot with 4 blocks and two configurations for the robot with 5 blocks. For these configurations we can plot the trace and robot pose, they are on the Figure 5.11

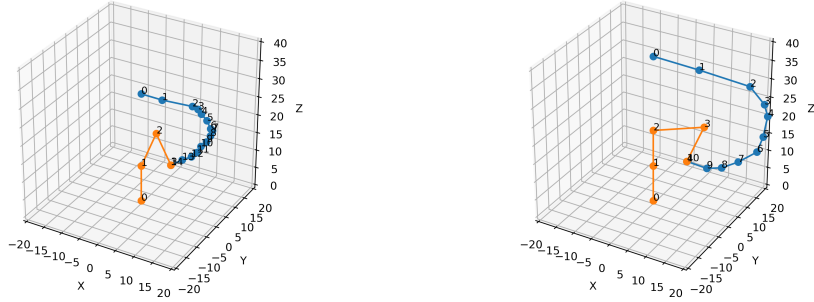


Figure 5.11: Trace for robot length 4 (left) and length 5 (right) for configurations that can reach the target region

5.5 Aggregated configuration checking in NuSMV

Alternative approach is to check all possible combinations of blocks in a robot in NuSMV itself. To implement this we add a new block to the NuSMV model described in Chapter 5.3.2. That block works as a universal interface between main module and modules for different types of blocks, the code for this block module shown in Listing 5.10.

```
1 MODULE block_general(type,px, py, pz, xhx, xhy, xhz, yhx, yhy, yhz,
   zhx, zhy, zhz, L, angle)
2 VAR
3   block_y : block_yaw( px, py, pz, xhx, xhy, xhz, yhx, yhy, yhz,
   zhx, zhy, zhz, L, angle );
4   block_p : block_pitch(px, py, pz, xhx, xhy, xhz, yhx, yhy, yhz,
   zhx, zhy, zhz, L, angle );
5 DEFINE
6   xhx2 := case type=0: block_y.xhx2; TRUE : block_p.xhx2; esac;
7   xhy2 := case type=0: block_y.xhy2; TRUE : block_p.xhy2; esac;
8   xhz2 := case type=0: block_y.xhz2; TRUE : block_p.xhz2; esac;
9   yhx2 := case type=0: block_y.yhx2; TRUE : block_p.yhx2; esac;
10  yhy2 := case type=0: block_y.yhy2; TRUE : block_p.yhy2; esac;
11  yhz2 := case type=0: block_y.yhz2; TRUE : block_p.yhz2; esac;
12  zhx2 := case type=0: block_y.zhx2; TRUE : block_p.zhx2; esac;
13  zhy2 := case type=0: block_y.zhy2; TRUE : block_p.zhy2; esac;
14  zhz2 := case type=0: block_y.zhz2; TRUE : block_p.zhz2; esac;
```

5 Implementation of model checking in SRRS

```

15     px2      := case type=0: block_y.px2; TRUE : block_p.px2; esac;
16     py2      := case type=0: block_y.py2; TRUE : block_p.py2; esac;
17     pz2      := case type=0: block_y.pz2; TRUE : block_p.pz2; esac;

```

Listing 5.10: General block serves as interface to choose and access specific types of modules

We also introduced a new variable in FROZENVAR part of the MAIN module that defines type for each block during model checking. In NuSMV, FROZENVAR are variables whose values are fixed throughout the entire execution of the model. They are assigned a value at the initialization step but do not change during state transitions, effectively acting as constant parameters that can differ between model instances. This construct is useful for representing static elements of our system that remain invariant while the system evolves, such as robot configuration and target region coordinates.

The robot structure part in MAIN module changes accordingly, to link to a general_block and pass var_block variable that specifies which type of block to use. These changes shown in the Listing 5.11.

```

1 MODULE main
2 FROZENVAR
3     var_block1 : {0,1};
4     var_block2 : {0,1};
5     ...
6 VAR
7     block0 : block_base(base_px, base_py, base_pz,
8         base_xhx, base_xhy, base_xhz,
9         base_yhx, base_yhy, base_yhz,
10        base_zhx, base_zhy, base_zhz, L0);
11    block1 : block_general(var_block1, block0.px2, block0.py2, block0
12        .pz2,
13        block0.xhx2, block0.xhy2, block0.xhz2,
14        block0.yhx2, block0.yhy2, block0.yhz2,
15        block0.zhx2, block0.zhy2, block0.zhz2, L1, angle1 );
16    block2 : block_general(var_block2, block1.px2, block1.py2, block1
17        .pz2,
18        block1.xhx2, block1.xhy2, block1.xhz2,
19        block1.yhx2, block1.yhy2, block1.yhz2,
20        block1.zhx2, block1.zhy2, block1.zhz2, L2, angle2 );
21    ...
22 DEFINE
23     ...
24     angle1 := 0;
25     angle2 := 0;
26     ...

```

Listing 5.11: Changes in main module to go over different types of blocks

TODO: describe counterexample of a specification to find an unreachable point

5 Implementation of model checking in SRRS

```
1 CTLSPEC  
2 EF (endX_inside_limits & endY_inside_limits & endZ_inside_limits)
```

Listing 5.12: Reachability condition in CTL specification

5.6 Reachability model checking in global space

5.6.1 Overview

In this section we will extend the model checking of a reachability that was described in Chapter 5.3.

Input: Initial position, target position, set of obstacles Robot structure (xacro) -> Define step algorithm -> optimal step? -> Generate NuSMV -> check model, get counterexample -> derive path -> compare with shortest path algorithm (implement Deikstra/ A* in 2D)

Evaluation: comparithon with shortest path algorithms Validation: assembly simulation in global space in Gazebo

5.6.2 Evaluation

5.7 General System Model checking

5.7.1 Overview

Input: Initial position, target position, set of obstacles Robot structure (xacro) -> Define step algorithm -> optimal step? -> Generate NuSMV -> check model, get counterexample -> derive path -> compare with shortest path algorithm (implement Deikstra/ A* in 2D)

Evaluation: comparithon with shortest path algorithms Validation: assembly simulation in global space in Gazebo

5.7.2 Evaluation

5.8 Collective behavior model checking

5.8.1 Overview

6 Simulation of the SRRS

In this chapter, we describe a simulation-based platform designed and implemented for self-replicating robotic systems, combining the ROS 2 middleware, Gazebo physics simulation, and a modular set of Python controllers and sensor nodes. The principal goal of this platform was not only to design a single robot simulation, but also to create an experimental platform that allows us to test, compare and validate different control approaches, including based on NuSMV, for self-replicating robotic systems.

We structured this simulation system as a collection of ROS 2 nodes and launch files that collectively define the robotic environment, the robot itself, the material in form of parts, and the control interface.

6.1 System Architecture

For the simulation we used ROS 2 Jazzy middleware [**ROS2Jazzy**], Gazebo Harmonic simulator [**GazeboHarmonic**], a set of ROS2 launch files and nodes in Python 3.12. The full code base and list of used libraries shown in [**GitHub**]

The simulated environment consists of three categories of entities:

- The main robot, defined in `robot.xacro`, which acts as an assembler of the new robot.
- Material parts, defined in `part.xacro`, which represent the components that the robot manipulates to assemble a new robot.
- The environment represented by base model, described in `base.sdf`, which provides a fixed reference plane, define optional obstacles and a physical structure for attachment operations of robots.

All these elements are spawned into the Gazebo Harmonic simulation using the ROS2 launch mechanism. 3D models for these entities were created in SolidWorks 2023 CAD software based on the design proposed by [**Jenett2019MaterialRobot**], and converted to STL mesh file.

6 Simulation of the SRRS

In order to control this simulation was developed following architecture, shown in Figure fig:SimulationClassDiagram.

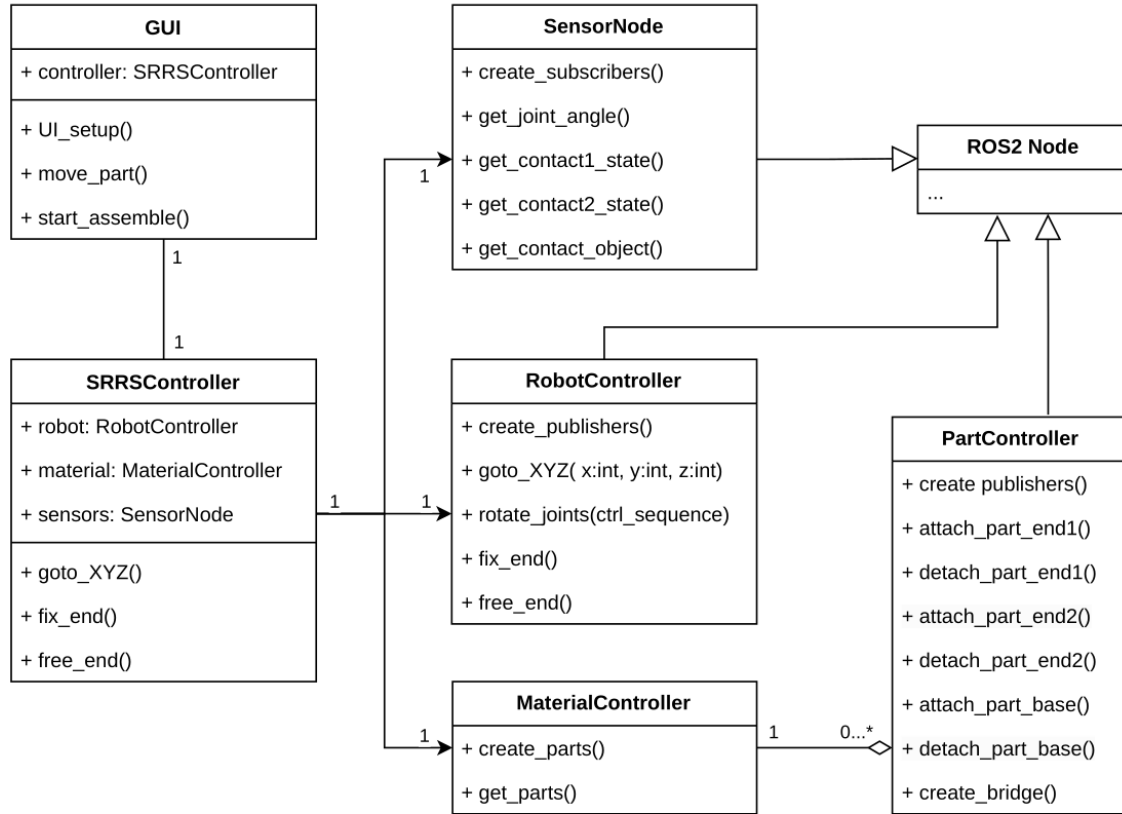


Figure 6.1: Class diagram of the simulation system

The system is designed with a layered approach, enabling interaction between the user interface, high-level control modules, and ROS 2 nodes communicating with Gazebo via the `ros_gz_bridge`. The architecture allows for coordinated manipulation, sensing, and part assembly in a simulated self-replicating robotic scenario.

We organized the software in several layers, from low-level motion control to high-level behavior orchestration:

- Initial launch file: `robot.launch`
- Low-level control node: `RobotController` class, shown in `robotController.py`.
- Material and part interface node: `MaterialController` and `PartController` classes, shown in `partController.py`.

6 Simulation of the SRRS

- Supervisory control node: `SRRScontrollerNode`, shown in `SRRScontrollerNode.py`.
- Sensor acquisition node: `SRRSsensorsNode` class.
- Graphical user interface: GUI class in `robot_controller_gui.py`.

We start the simulation by running `robot.launch` file, it initiates Gazebo world and starts the graphical user interface.

GUI class provides the main entry point for user interaction. It designed to enable manual and semi-automated control of the robotic movement and assembly process. The main functions included in GUI class are: `UI_setup()`, which initializes the user interface layout, `move_part()` to command manipulations of individual elements, and `start_assemble()` to trigger the assembly sequence. Internally, the GUI maintains a reference to the `SRRSController`, representing the supervisory control logic of the entire system. Through this connection, user commands are propagated down to robot controller, material manager, and sensor interfaces.

The `SRRSController` class acts as the coordination layer that integrates the functional subsystems. It maintains three key associations: the `RobotController`, `MaterialController`, and `SensorNode`. These relationships form the backbone of the system's modularity, where each controller is responsible for a well-defined subset of functionalities. The `SRRSController` provides functions like `goto_XYZ()` - to move the free end-effector to given coordinates or `fix_end()` to fix on the base or on the part for transportation, that abstract the complexity of low-level motion and interaction control. When the user initiates an action through the GUI, the `SRRSController` orchestrates the appropriate response by invoking methods from subordinate controllers.

An object of `RobotController` class handles direct motion control of the robot within Gazebo. It designed to generate and publish control commands to the simulation environment through ROS2 topics. This class represents the bridge between high-level assembly planning and low-level joint- space control, encapsulating motion primitives into callable functions accessible by the `SRRSController`.

The `SensorNode` component provides perception capabilities. It functions as a ROS2 node with several subscribers to Gazebo topics relayed through the `ros_gz_bridge`. These topics provide information about joint angles, contact states, and detected objects in the simulation. We implemented methods such as `get_joint_angle()`, `get_contact1_state()`, `get_contact2_state()`, and `get_contact_object()` to access structured sensory information. By maintaining real-time synchronization between the Gazebo simulation and the ROS2 communication layer, the `SensorNode` allows the control system to react dynamically to physical interactions occurring during assembly— such as contact detection when two parts align.

The `MaterialController` manages the set of parts available for assembly. It gives a possibility to create and manipulate parts in the Gazebo world, using functions `create_parts()` and `get_parts()`. These parts are represented as Gazebo models that can be spawned, attached, or detached through dedicated ROS2 launch file. The `MaterialController` establishes a one-to-many association with the `PartController`, reflecting that a single material manager may handle multiple part controllers simultaneously. Each `PartController` instance corresponds to a specific simulated component within the environment.

The `PartController` provides fine-grained control over individual parts in the simulation. It includes methods for creating ROS2 publishers and service clients, as well as specialized attachment and detachment procedures for various connection points, with corresponding detachment methods. The `create_bridge()` function establishes the communication link with Gazebo through `ros_gz_bridge`, ensuring that changes to part states in Gazebo (such as attachment or detachment) are mirrored within the ROS 2 ecosystem.

Collectively, this architecture enables the robot in simulation to perform complex movements and assembly tasks manually or according to predefined program. The GUI layer allows human supervision and intervention, while the `SRRSController` coordinates motion and assembly actions. Since each class corresponds to a logical unit of functionality, the framework can easily integrate additional controllers, such as vision modules or planning algorithms, by extending the existing class hierarchy.

6.2 Gazebo simulation elements

6.2.1 Base model

Base model spawn by initial launch file (`robot.launch`) and represented by static and fixed in world 3D model of lattice. In our simulation the base described in SDF file `base.sdf` and has a dimension of 10x10 cells, it is shown in Figure 6.2.

The Base code provided in Appendix A.4.1 and consist of a link with geometry, and fixed joint that connect link to a world.

The base represents environment and can be supplemented with additional static structures, by spawning parts with fixed joint from the same `robot.launch` file.

6 Simulation of the SRRS

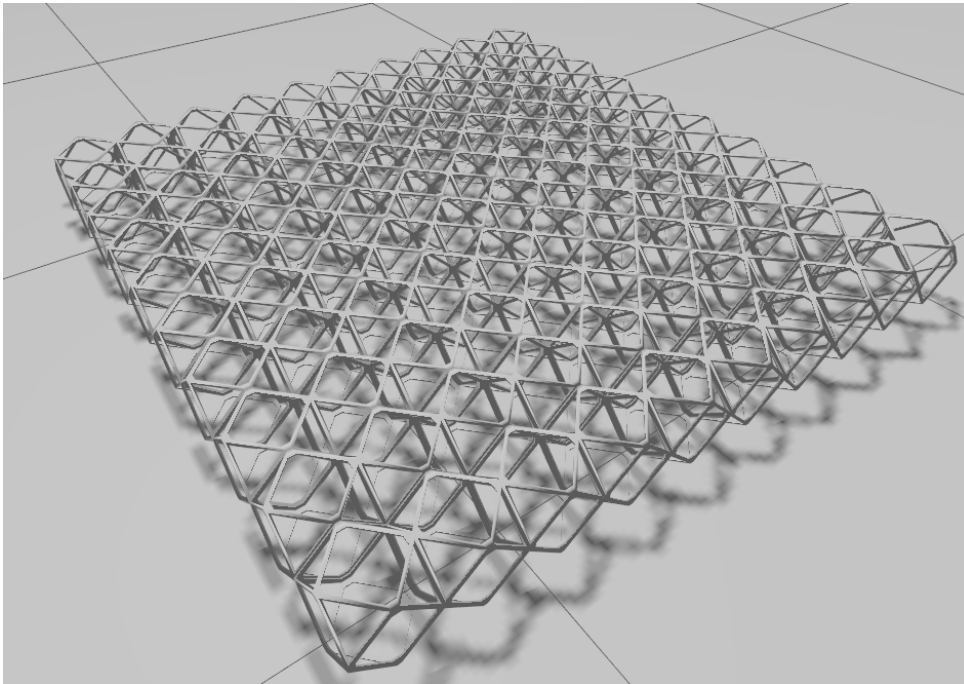


Figure 6.2: Base model in Gazebo simulator

6.2.2 Parts simulation and behavior

In our simulation part represents the simplest element of a material, used to assemble a new robot. It is defined in part.xacro file and consist of a main link, shown in Listing 6.1, and detachable plugins that describe connection to base, robot end-effectors and other parts.

```
1 <link name="part">
2   <visual>
3     <geometry>
4       <mesh filename="meshes/Voxel.stl" scale="0.001 0.001
5         0.001"/>
6     </geometry>
7   </visual>
8   <collision name="part_collision">
9     <geometry>
10      <mesh filename="meshes/Voxel.stl" scale="0.001 0.001
11        0.001"/>
12    </geometry>
13  </collision>
14  <inertial>
15    <mass value="0.1"/>
16  </inertial>
17 </link>
```

6 Simulation of the SRRS

```
14     <inertia ixx="1.0" ixy="0.0" ixz="0.0" iyy="1.0" iyz="0.0"  
15         izz="1.0"/>  
16 </inertial>  
17 </link>
```

Listing 6.1: Part link code inside of part.xacro

One of plugins for detachable joint is shown in Listing 6.2.

The detachable joint controlled from a ROS2 with two topic messages of type Empty, separately for attach and detach actions. In xacro file we defined topic names, and child model and link in URDF tree for connection.

```
1  <gazebo>  
2    ...  
3    <plugin  
4      filename="libgz-sim8-detachable-joint-system.so"  
5      name="gz::sim::systems::DetachableJoint">  
6      <parent_link>part</parent_link>  
7      <child_model>robot</child_model>  
8      <child_link>block0</child_link>  
9      <attach_topic>attach_end1_part${xacro.arg('part_num')}</  
10         attach_topic>  
11      <detach_topic>detach_end1_part${xacro.arg('part_num')}</  
12         detach_topic>  
13      <output_topic>detachable_obj_end1_state${xacro.arg('  
14         part_num')}</output_topic>  
15      <suppress_child_warning>true</suppress_child_warning>  
16    </plugin>  
17    ...  
18  </gazebo>
```

Listing 6.2: Detachable joint example from partxacro

For low level control of a part we created a PartController class that is an extension of ROS2 Node, the code shown in Listing A.3.2

The PartController object provides an individual control over specific part in the simulation. It has following functionality:

- Create publishers for all topics for mechanical attachment and detachment of the part to base and both end-effectors of a robot.
- Provides methods to publish control topics in publishers.
- Starts ros-gz-bridge that connect topics (control messages) in distributed system of ROS2 with Gazebo simulated environment.

6.3 Robot simulation and control

6.3.1 Gazebo representation of a robot

As the robot configuration for further simulation we take a five-joint manipulator with two fixed ends, enabling alternating attachment and detachment to the base during locomotion and assembly, the initial configuration is shown in Figure 6.3.

The modularity of this structure allows us to simulate replication cycles where the robot assembles a set of parts into a new robot instance. This provides a flexible testbed to evaluate different control policies- deterministic sequences, feedback-based behaviors, or Model checking based strategies -without modifying the core simulator.

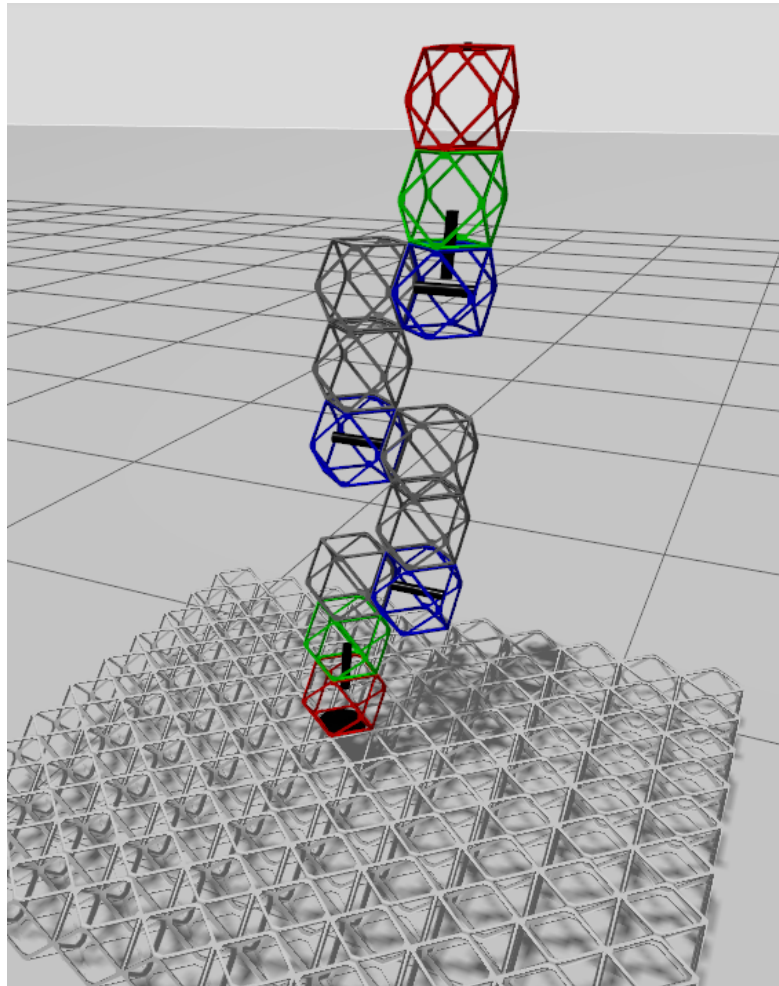


Figure 6.3: Robot structure in Gazebo simulation environment

6 Simulation of the SRRS

This design emulates the behavior of a linear self- reconfiguring structure similar to a modular relative robot.

The physical description of a robot encoded in robot.xacro file and defines the structural and kinematic model of the self-replicating robot used in our simulation. It serves as a expanded description of the robot geometry, joints, sensors, and actuation interfaces, compatible with both ROS2 and Gazebo. The use of Xacro (XML Macros) provides modularity and reusability, enabling to easily modify or replicate robot configurations during simulation. Each link and joint is described using standard URDF elements, wrapped in XACRO macros to support parameter substitution,

The main structure of a robot described in Listing 6.3. Each line in it represents a block of type: block_hor, block_ver, block_fix or block_end (block_end is equivalent to "voxel" type of block0). All of the blocks have a number, parent, coordinates relative to a parent and their types described in separate macros.

```
1 <xacro:voxel name="block0" origin="0 0 0" color="${red}"/>
2 <xacro:block_ver block_num="1" parent_num="0" x="0" y="0" z="0"/>
3 <xacro:block_fix block_num="2" parent_num="1" x="0" y="0" z="1"/>
4 <xacro:block_hor block_num="3" parent_num="2" x="0" y="1" z="1"/>
5 <xacro:block_fix block_num="4" parent_num="3" x="0" y="0" z="0.5"/>
6 <xacro:block_fix block_num="5" parent_num="4" x="0" y="0" z="1.5"/>
7 <xacro:block_hor block_num="6" parent_num="5" x="0" y="-1" z="1.5"/>
8 <xacro:block_fix block_num="7" parent_num="6" x="0" y="0" z="0.5"/>
9 <xacro:block_fix block_num="8" parent_num="7" x="0" y="0" z="1.5"/>
10 <xacro:block_hor block_num="9" parent_num="8" x="0" y="1" z="1.5"/>
11 <xacro:block_ver block_num="10" parent_num="9" x="0" y="0" z="-0.5"/>
12 <xacro:block_end block_num="11" parent_num="10" x="0" y="0" z="0"/>
```

Listing 6.3: Robot configuration as a sequence of blocks

In simulation and on a Figure 5.2 we define following color convention:

- red - base block (block_end or voxel)
- black - fixed block (block_fix)
- blue - initially horizontal rotating block (block_hor)
- green - initially vertical rotating block (block_ver)

Some blocks represent static structures and are fixed to a previous blocks or base like block_fix in the Listing 6.4.

```
1 <xacro:macro name="block_fix" params="block_number parent_number
2   x y z " >
  <xacro:voxel name="block${block_number}" origin="${x*
    cell_step} ${y*cell_step} ${z*cell_step}" color="${grey}"
  />
```

6 Simulation of the SRRS

```
3      <xacro:joint_fix name="fixed${parent_number}_${block_number}"
4          parent="block${parent_number}" child="block${block_number}
          " />
      </xacro:macro>
```

Listing 6.4: Block type block_fix as a macros

Others have rotating joints, that can be controlled, the example of a block_hor macros shown in Listing 6.5, its

```
1      <xacro:macro name="block_hor" params="block_number parent_number
2          x y z " >
3          <xacro:joint_revolution name="rev${parent_number}_${
4              block_number}" parent="block${parent_number}" child="
              block${block_number}" axis="0 1 0" origin="${x*cell_step}
              ${y*cell_step} ${(z+0.5)*cell_step}" rpy="1.57 0 0"/>
          <xacro:voxel name="block${block_number}" origin="${0} ${0} $
              {-0.5*cell_step}" color="${blue}"/>
      </xacro:macro>
```

Listing 6.5: Block type block_hor as a macros

Each block defined with a name, parent block relative to which it rotates, rotation axis, coordinates and color.

Each joint_revolution is connected to a ros2_control plugin, described in the Listing 6.6 and connected joints to ROS2 system.

```
1      <ros2_control name="GazeboSimSystem" type="system">
2          <hardware>
3              <plugin>gz_ros2_control/GazeboSimSystem</plugin>
4          </hardware>
5          <joint name="rev0_1">
6              <command_interface name="position"/>
7              <state_interface name="position"/>
8          </joint>
9          <joint name="rev2_3">
10             <command_interface name="position"/>
11             <state_interface name="position"/>
12         </joint>
13         <joint name="rev5_6">
14             <command_interface name="position"/>
15             <state_interface name="position"/>
16         </joint>
17         <joint name="rev8_9">
18             <command_interface name="position"/>
19             <state_interface name="position"/>
20         </joint>
21         <joint name="rev9_10">
22             <command_interface name="position"/>
```

```

23 <state_interface name="position"/>
24 </joint>
25 </ros2_control>

```

Listing 6.6: Block type block_hor as a macros

Robot also contains global parameters, that define physics and interaction with parts and base. Global parameters allow consistent scaling of geometry, mass, and joint limits across the entire robot. We consider a `link_length` parameter to be a unified length unit in our discrete environment, it is a size of a voxel side, and one step on a base lattice. shown in Listing 6.7

```

1 <xacro:property name="link_length" value="0.134"/>
2 <xacro:property name="link_mass" value="0.2"/>
3 <xacro:property name="joint_limit" value="3.1416"/>

```

Listing 6.7: Parameters of a robot in robot.xacro

6.3.2 Low level robot controller

The as described in previous section Gazebo model of a robot provides an ability to connect to parts and base and to rotate around multiple revolute joints by name (`rev0_1`, `rev2_3`, `rev5_6`, `rev8_9`, `rev9_10` in Listing 6.6) allowing high-degree-of-freedom manipulation.

We developed a separate class `RobotController` that extends a ROS2 Node in order to control this functions on a low level by generating and publishing control commands through ROS2 topics. Methods of a class include:

- `create_publishers()` for initializing communication channels
- `goto_XYZ(x, y, z)` for commanding Cartesian movements
- `rotate_joints(ctrl_sequence)` for performing articulated joint rotations.
- `fix_end()` and `free_end()` methods are used to attach or detach end-effectors, mimicking real-world manipulative behaviors.

We modeled the motion of the robot's free end in Cartesian coordinates (x, y, z) relative to the fixed base. The function `goto_XYZ(x, y, z)` defines the high-level command for moving the robot's end-effector to a desired position. The function recursively decomposes the trajectory into steps on each step if the point is not reachable (length is more than a step length) robot perform a step movement in target direction, lines 6-28 in Listing 6.8. And when it came closer the function invokes the lower-level function

6 Simulation of the SRRS

calculate_joint_angles() to calculate joint angles for necessary target point, line 30 in Listing 6.8.

Listing 6.8: Function goto_XYZ() of RobotController class

```
1  def goto_XYZ(self, x, y, z, step_size=3):
2      x = float(x)
3      y = float(y)
4      z = float(z)
5      # Choose if target point in near or far form (0,0)
6      if abs(x) > step_size or abs(y) > step_size:
7          # going to a far located point
8          if x > step_size:
9              stepX = step_size
10             x = x - step_size
11         elif x < -step_size:
12             stepX = -step_size
13             x = x + step_size
14         else:
15             stepX = 0
16         if y > step_size:
17             stepY = step_size
18             y = y - step_size
19         elif y < -step_size:
20             stepY = -step_size
21             y = y + step_size
22         else:
23             stepY = 0
24         # Step algorithm:
25         self.goto_XYZ(stepX, stepY, 1)
26         self.goto_XYZ(stepX, stepY, 0)
27         self.swap_fix_block() # swap "legs"
28         self.goto_XYZ(x, y, z, step_size)
29     else: # near pose calculation
30         command_sequences = self.calculate_joint_angles(x, y, z)
31         self.rotate_joints(command_sequences)
```

Joint angle calculation function, shown in Listing A.3.1, serves to calculate joint angles based on chosen combination of blocks and works only for our blocks parameters.

Mathematically, for a given target point (x, y, z), we compute three internal angular parameters:

$$\alpha = \arcsin\left(\frac{\sqrt{r_0^2 + z^2}}{4}\right)$$
$$\beta = \arctan\left(\frac{y}{x}\right)$$

6 Simulation of the SRRS

$$r = \sqrt{y^2 + x^2}$$
$$\gamma = \arctan\left(\frac{z}{r_0}\right)$$

,where:

α - is a pitch of middle block without correction

β - yaw of a first block

γ - correction of α

$r_0 = \sqrt{x^2 + y^2 - 1}$ is a correction term depending on the robot geometry.

These angles are used to derive the five joint commands:

$$q_1 = \beta_0$$
$$q_2 = \alpha - \gamma$$
$$q_3 = 180^\circ - 2\alpha$$
$$q_4 = \alpha + \gamma - \sigma^z$$
$$q_5 = \beta_0$$

with σ^z being a vertical correction term (0° or 90° depending on z). The controller converts these degrees to radians and publishes them as Float64MultiArray messages to each joint topic "position_controllerN" commands. The motion is considered complete when the difference between the target and current joint angles is below a threshold, in our example it is $\epsilon = 2^\circ$.

The described approach constitutes an open-loop control with feedback verification: we do not continuously adjust the trajectory, but we ensure that each commanded position is reached before the next step. The convergence condition is implemented through the function `wait_movement_finish()` using ROS 2 timers and events.

6.4 Base and Part Interaction

To emulate self-assembly and replication, we introduced attach/detach mechanisms between the robot's end effectors, parts, and base. In the Gazebo simulation, attachment is realized through service calls bridged from ROS to Gazebo's internal link-joining mechanism. The following attachment states are defined:

6 Simulation of the SRRS

$A_{R1,B}$: end-effector 1 attached to base

$A_{R2,B}$: end-effector 2 attached to base

A_{R1,P_i} : end-effector 1 attached to part i

A_{R2,P_i} : end-effector 2 attached to part i

Each of these is represented by a ROS topic such as /attach_end1_part1 or /attach_part1_base. Detachment events are the complementary /detach_* topics. The PartController node, shown in Listing A.3.2, automatically creates and maintains all necessary publishers for these topics. When a new PartController(number) instance is created, it initializes a corresponding Gazebo bridge through the ros_gz_bridge parameter bridge:

Each part is modeled as a single block that can attach to either of the robot's two end effectors or to the base via virtual "attachment joints." These joints are controlled through ROS topics using the std_msgs/msg/Empty message type to send attach/detach commands, bridged to Gazebo via ros_gz_bridge.

/attach_end1_partnum@std_msgs/msg/Empty@gz.msgs.Empty

/detach_end1_partnum@std_msgs/msg/Empty@gz.msgs.Empty

and similarly for other combinations. Each bridge connects a ROS 2 topic to a Gazebo message interface, ensuring low-latency coupling between control and simulation.

The base model (base.sdf) serves as an inert reference body with defined attachment points aligned with the robot's initial configuration. It also defines the inertial frame for all coordinate calculations in goto_XYZ().

Sensor Integration
The SRRSensorsNode provides the sensory feedback needed for validation of contact and joint dynamics. We subscribe to the following topics:

/joint_states – to obtain joint positions and velocities,

/robot/contact1 and /robot/contact2 – to detect physical interactions between the end effectors and other objects,

/robot/imu1 – for inertial measurements of base acceleration and orientation.

The SRRSensorsNode interprets messages of type ros_gz_interfaces/ msg/Contacts, parses them to extract collision partner names, and publishes simplified string notifications on /contact1/change_state and /contact2 /change_state. These allow the controller and GUI to display real- time contact information.

The contact state logic is time-based: if no new contact message is received for more than $\Delta t = 0.5s$, the system resets the contact flag. This ensures robustness against

transient or spurious collision events. We use ROS 2 timers to evaluate these conditions asynchronously.

The joint angle information from this node is also made available to the GUI for continuous visual updates. Through this feedback loop, we can evaluate not only the logical success of assembly sequences but also the physical consistency of movements and the timing behavior of the control algorithms.

6.5 Supervisory Control and GUI Integration

The starting point of the simulation is the launch file `robot.launch`, which initiates the Gazebo world, spawns the robot and the base environment, and launches the graphical interface for control (`robot_controller_gui.py`). The design ensures that the entire system can be initialized with a single command and that all subsystems—control, sensing, visualization—operate concurrently under ROS2 multi-threaded executor.

High-level behavior orchestration is handled by the `SRRScontrollerNode` class (defined in `SRRScontrollerNode.py`). This node creates instances of both `RobotController` and `PartController` and coordinates their interactions. It exposes simple methods such as `fix_end1_part(num)`, `free_end2_part(num)`, `fix_end1_base()`, and others, which are directly callable from the GUI.

The graphical user interface (`robot_controller_gui.py`) constitutes a crucial part of the system. It serves as both a human control console and an experiment orchestration tool. Built in Tkinter, it allows the operator to:

- Input target coordinates (X, Y, Z) and issue movement commands.

- Fix or release robot ends or parts through dedicated buttons.

- Observe current joint angles, contact states, and camera feeds.

- Execute the automatic assembly process via `start_assemble()`. The `robot_controller_gui.py` file launches and connects all these nodes, forming the interactive entry point to the SRRS simulation. The GUI shown in Figure 6.4 is implemented in Tkinter, providing manual and automated control capabilities, live feedback of joint states, and camera streams.

When the "Start Assemble" button is pressed, the GUI calls `start_assemble()`, which executes an automated sequence of motions to assemble a new robot from scattered parts.

6 Simulation of the SRRS

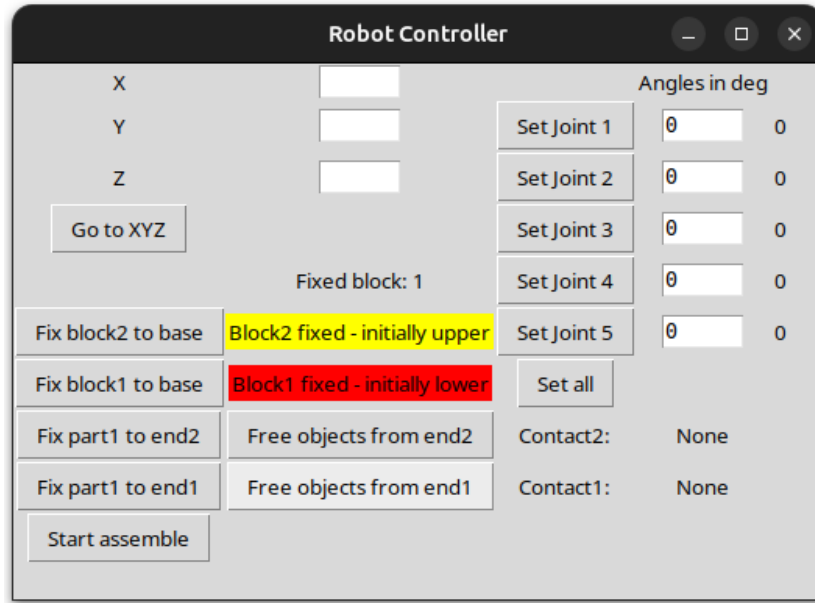


Figure 6.4: Graphical user interface for simulation control

The GUI continuously updates sensor readings using scheduled callbacks (`root.after()`), ensuring a refresh rate of approximately 30–100 ms for angles and contact information. Camera frames are displayed using Pillow's ImageTk interface, showing two synchronized camera streams from Gazebo topics `/camera1/image` and `/camera2/image`.

When the user initiates `start_assemble()`, the GUI executes a predefined sequence:

Fix the first end of the robot to the base.

Release all part connections.

Set assembly coordinates (typically $(2, -2, 0)$).

Move to each part, pick it up, and place it at the assembly location.

Increment the z-coordinate for each subsequent layer.

This sequence is implemented in the method `move_part(num, x, y)`, which demonstrates a loop of high-level pick-and-place actions using the primitive `goto_XYZ()` calls and attachment commands. Each movement is performed in small steps to avoid collisions, and the GUI waits for the movement threads to complete before issuing the next command.

Launch and Execution Pipeline

The entire simulation is initiated through `robot.launch`, which performs the following actions sequentially:

Launches the Gazebo Harmonic world with the base model loaded (`base.sdf`).

Spawns the robot using `robot.xacro` through the standard `ros2 run ros_gz_sim create` command.

Spawns several parts defined in `part.xacro` distributed across the environment.

Starts the GUI and underlying control nodes (`SRRScontrollerNode`, `SRRSsensorsNode`, and camera subscribers).

The environment thus initialized represents a closed-loop experimental setup where the robot perceives, acts, and interacts with parts and the base under simulated physics. Because ROS 2 Jazzy supports real-time communication and multithreading, the GUI and control nodes can run in parallel without blocking, enabling responsive interaction.

Control Strategy Validation Framework

The main purpose of this project was to create a validation framework for different control strategies rather than to fix a single deterministic behavior. Within our architecture, alternative controllers can be plugged in at several levels:

Kinematic policy: replacing the `calculate_joint_angles()` function with a neural or optimization-based inverse kinematics solver.

Trajectory planning: modifying `goto_XYZ()` to use continuous path generation instead of recursive steps.

Assembly sequencing: altering `start_assemble()` to implement adaptive planning based on part detection or reinforcement learning.

Sensor fusion: integrating visual feedback from the camera nodes to enable autonomous part localization.

The modular design of ROS 2 nodes facilitates this substitution without affecting the overall system. For example, the `RobotController` class exposes clear interfaces (`rotate_joint()`, `rotate_joints()`, `goto_XYZ()`), and the communication topics are standardized. Therefore, any new controller can publish to the same topics and subscribe to the same sensor messages, ensuring compatibility.

Validation is achieved by comparing the success rate, time to completion, and energy metrics (approximated by cumulative joint rotations) of different strategies. The GUI's visual and logged data provide qualitative insights, while the ROS 2 message traces can be quantitatively analyzed.

6.5.1 Mathematical Abstraction of Assembly Process

We can represent the SRRS assembly process as a discrete sequence of states S_t , each describing the configuration of all parts and the robot at time t . Each state includes the position vector of the robot end

Our platform enables direct observation of these transitions under different controllers f , providing a testbed for evaluating stability, robustness, and repeatability.

6.5.2 Implementation Challenges and Design Considerations

Developing this system required addressing several challenges intrinsic to both ROS 2 and Gazebo:

Synchronization of simulated physics and ROS 2 communication. Gazebo Harmonic introduces slight delays in joint updates; therefore, we implemented event-based waiting (`threading.Event`) rather than time-based sleeps to detect motion completion.

Attachment semantics. Because Gazebo joint creation for attachments can introduce instabilities, we used lightweight Empty message topics rather than service calls, improving responsiveness.

GUI concurrency. To prevent blocking of ROS 2 threads by Tkinter, we ran the ROS 2 executor (`MultiThreadedExecutor`) in a separate daemon thread, allowing GUI and simulation updates to coexist without race conditions.

Scalability. The number of parts can be increased dynamically using `spawn_part()`. This function invokes a new ROS 2 process through a Bash command that calls `ros2 run ros_gz_sim create`, ensuring that each part receives a unique name and topic namespace.

Extensibility. Because each node is encapsulated, new sensors or controllers can be added by extending the base class `rclpy.node.Node`.

Through this architecture, we achieved a balance between scientific flexibility and practical usability.

6.6 Simulation Workflow

In a typical experiment, we initialized the simulation with the robot fixed at the base and three parts distributed at different coordinates. When we executed the automated assembly via `start_assemble()`, the robot alternated between fixing one end to the base, moving the free end to a part, attaching it, and transporting it to the assembly point. This process was visualized in Gazebo and confirmed through GUI feedback of contact states.

We observed that the recursive stepping algorithm ensured stable motion without overshoot. The joint convergence tolerance of 2 degrees provided reliable positioning, and the end-effector’s attach/detach operations were successfully synchronized with the movement sequence. The assembled structure maintained physical integrity under Gazebo’s physics engine, confirming that the connection logic and control synchronization were valid.

Extensibility Toward Autonomous Self-Replication

While our current implementation focuses on deterministic sequences, the architecture inherently supports higher-level autonomy. By replacing the manual coordinate inputs with vision-based part detection, the robot could identify parts automatically and plan assembly paths. The `SRRScontrollerNode` can be extended to integrate a planning layer using the MoveIt 2 framework or a reinforcement learning policy trained via the ROS 2–Gazebo interface.

Furthermore, because all messages are standard ROS 2 types, the same system could be deployed on physical robots with appropriate hardware drivers. The self-replication process would then move from simulation to reality, enabling experimental validation of robotic “growth” behaviors.

6.6.1 Conclusion

Through the development of this project, we established a modular and extensible simulation environment for studying self-replicating robotic systems. Our implementation demonstrates that the integration of ROS 2 Jazzy and Gazebo Harmonic provides a sufficiently realistic and flexible platform to test control strategies involving attachment, detachment, and multi-body coordination. The structure of the codebase—especially the clear separation between `RobotController`, `PartController`, and `SRRSsensorsNode`—enables straightforward substitution of alternative algorithms, whether model-based or data-driven.

6 Simulation of the SRRS

In terms of validation, we confirmed that deterministic control sequences can reliably assemble multiple parts into a replica structure under physics -based constraints. The architecture also provides quantitative metrics such as joint trajectories, contact durations, and execution times, allowing systematic evaluation of alternative strategies.

TODO: reference in text simulation to a previous NuSMV part

TODO: simulate reachable configurations from configuration checking part

TODO: simulate counterexample based trace as a command sequence for movement

7 Evaluation and validation

For the validation of properties checked in previous chapters and for demonstrating the feasibility of the counterexample-guided control approach, we complement the formal analysis with an experimental simulation platform. The platform is realized in the Gazebo environment, which provides a physics-based simulation of the modular robot and its interaction with voxel-like blocks. The robot is controlled through ROS2 nodes, enabling modular and distributed control strategies. By comparing the outcomes of formal verification with simulation results, we aim to assess both the soundness and the practical utility of model-checking-based methods for modular self-replicating robots.

7.1 Validation of the counterexample based robot control

To validate the counterexample based approach, introduced in chapter 5.3.4 we will integrate the counterexample generation pipeline into ROS2-Gazebo simulation environment. We choose robot blocks configuration described in Chapter 6.3, robot initial position and target point with coordinates [50, 50, 10]. These initial parameters reflected in the lines 1-3 of the Listing 7.1 lines.

Listing 7.1: Function goto_XYZ() of RobotController class

```
1 angle_vars = ["block1.yaw", "block2.pitch", "block3.pitch", "block4.  
    pitch", "block5.yaw"]  
2 sequence = ["base", "yaw", "pitch", "pitch", "pitch", "yaw"]  
3 lengths = [10, 20, 30, 30, 10, 20]  
4 robot = Robot(sequence, lengths)  
5 generator = RobotNusmvGenerator(robot, output_smv_path="./smv/")  
6 generator.set_target_point(50, 50, 10)  
7 generator.generate_from_template("project_robot")  
8 result = Counterexample.run_nusmv_file("./smv/robot.smv")  
9 counterexample = Counterexample(result, len(sequence))  
10 angles = counterexample.get_angles(angle_vars)  
11 counterexample.plot_trace(-1, show_plot=True)
```


7 Evaluation and validation

Trace derived from the counterexample consist out of 9 steps and shown in Figure 7.1

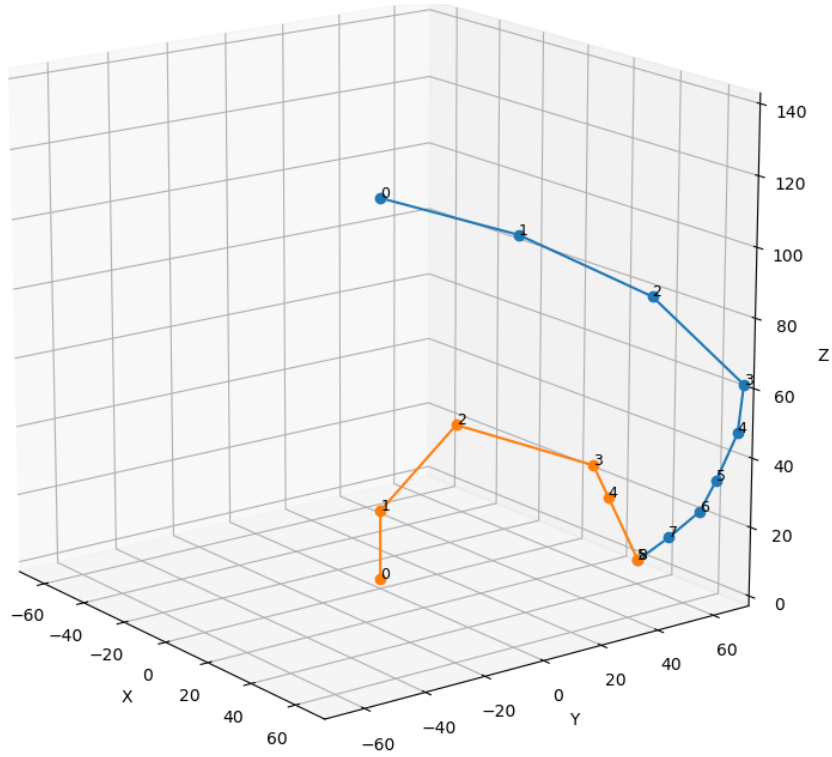


Figure 7.1: Mean time of model checking with number of blocks in robot

Resulting joint angles derived from the trace presented as a list steps, where each step is a list of control angles for each joint:

[0, 0, 0, 0, 0]

[0, -10, -10, -10, -10]

[0, -20, -20, -20, -20]

[0, -30, -30, -30, -30]

[0, -30, -40, -40, -40]

[0, -30, -50, -50, -50]

[0, -30, -60, -50, -60]

[0, -30, -70, -50, -60]

[0, -30, -80, -50, -60]

7 Evaluation and validation

We will publish these angles as a sequence of commands for robot joints using `rotate_joints()` function in `RobotController` class, described in Chapter 6.3. Before simulation we have to adjust position and rotation of the gazebo coordinate system and coordinate system recieved from counterexample. The simulation result can be seen on the Figure 7.2

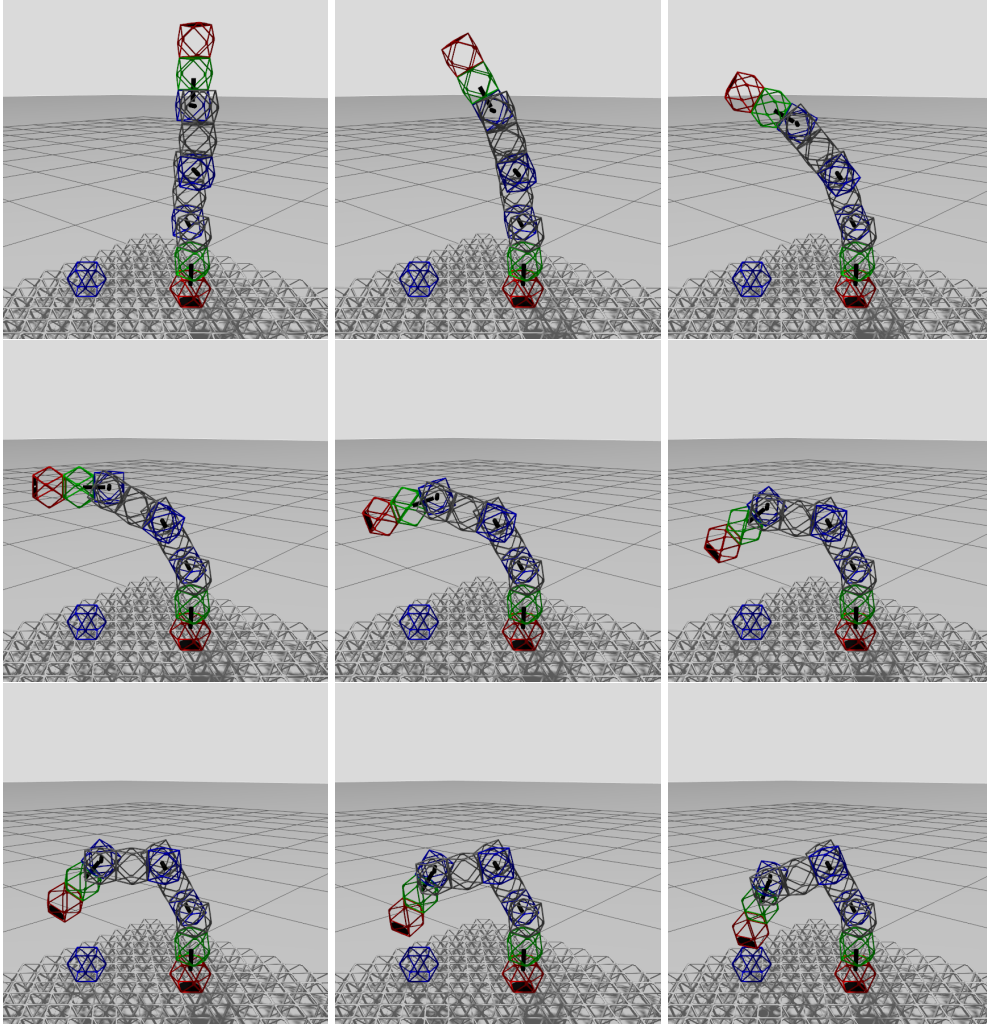


Figure 7.2: Simulation steps to achieve point $[5,5,1]$ from counterexample

For considered target point the distance from robot end to target is around 0.16.

Total discrepancy between the target point and the center of the end effector consists of the angle discretization error in NuSMV and the geometric offset of the blocks in the simulation model. Maximal discrepancy during simulation of 10 different reachable points was 27% of length unit in the simulation (size of the voxel).

7.2 Validation of the robot configuration checking

7.3 Complexity evaluation

TODO: Add comparison of a time to preform an assembly action

TODO: Add graph of a model checking time for different lengths

We can assess the complexity of the configuration checking with sequential model checking, using a target point with coordinates: $x=5$, $y=5$, $z=10$. The changing of mean time required for checking relative to a number of blocks in robot shown in the Figure 7.3

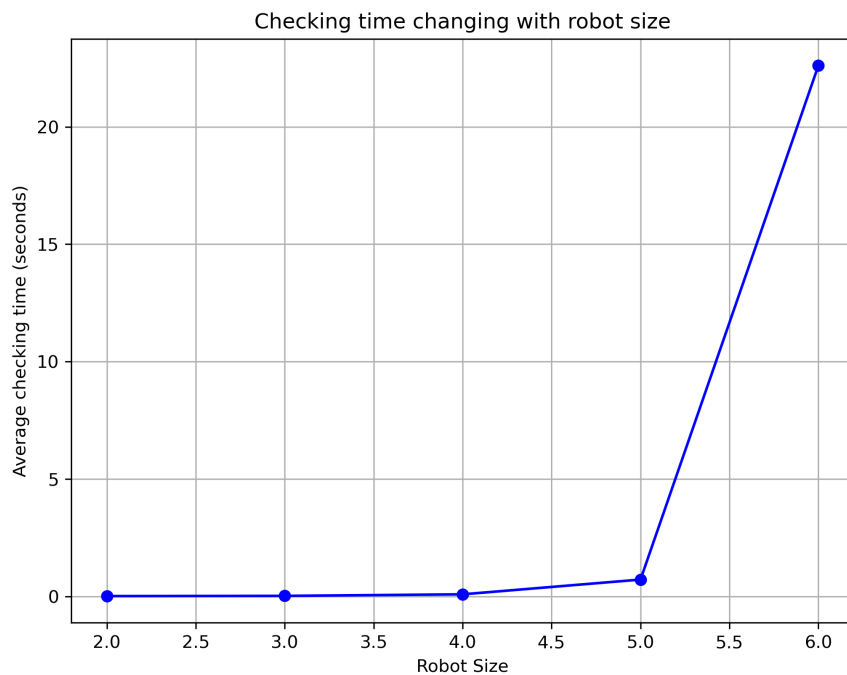


Figure 7.3: Mean time of model checking with number of blocks in robot

8 Conclusion

Future work will focus on incorporating autonomous perception and adaptive control: integrating visual localization of parts, dynamic path planning , and learning-based decision modules. By doing so, the SRRS framework can evolve from a manually orchestrated demonstration toward a truly self-replicating robotic organism capable of operating in unstructured environments.

In conclusion, the SRRS project serves as both a scientific and engineering contribution: scientifically, as a platform for testing hypotheses about self-replicating mechanisms; and technically, as a fully functional ROS 2–Gazebo system implementing attachment-based assembly. The methods and code we developed—spanning from the launch orchestration (`robot.launch`) to the detailed node-level control (`robotController.py`, `SRRScontrollerNode.py`, `partController.py`)-collectively provide a reproducible foundation for future research in autonomous manufacturing and self-replication robotics.

A Appendix

A.1 NuSMV models

A.1.1 robot_structure.smv

Trigonometric modules

```
1 MODULE sin(angle)
2 DEFINE
3   val:= case
4     angle = 0 : 0;   angle = 1 : 17;   angle = 2 : 34;
5     angle = 3 : 50;  angle = 4 : 64;   angle = 5 : 77;
6     angle = 6 : 87;  angle = 7 : 94;   angle = 8 : 98;
7     angle = 9 : 100; angle = 10 : 98;   angle = 11 : 94;
8     angle = 12 : 87; angle = 13 : 77;   angle = 14 : 64;
9     angle = 15 : 50; angle = 16 : 34;   angle = 17 : 17;
10    angle = 18 : 0;   angle = 19 :-17;  angle = 20 :-34;
11    angle = 21 :-50;  angle = 22 :-64;   angle = 23 :-77;
12    angle = 24 :-87;  angle = 25 :-94;   angle = 26 :-98;
13    angle = 27 :-100; angle = 28 :-98;   angle = 29 :-94;
14    angle = 30 :-87;  angle = 31 :-77;   angle = 32 :-64;
15    angle = 33 :-50;  angle = 34 :-34;   angle = 35 :-17;
16    TRUE : 0;
17  esac;
18
19 MODULE cos(angle)
20 DEFINE
21   val:= case
22     angle = 0 : 100;  angle = 1 : 98;   angle = 2 : 94;
23     angle = 3 : 87;   angle = 4 : 77;   angle = 5 : 64;
24     angle = 6 : 50;   angle = 7 : 34;   angle = 8 : 17;
25     angle = 9 : 0;    angle = 10 :-17;  angle = 11 :-34;
26     angle = 12 :-50;  angle = 13 :-64;  angle = 14 :-77;
27     angle = 15 :-87;  angle = 16 :-94;  angle = 17 :-98;
28     angle = 18 :-100; angle = 19 :-98;  angle = 20 :-94;
29     angle = 21 :-87;  angle = 22 :-77;  angle = 23 :-64;
30     angle = 24 :-50;  angle = 25 :-34;  angle = 26 :-17;
31     angle = 27 : 0;   angle = 28 : 17;   angle = 29 : 34;
32     angle = 30 : 50;  angle = 31 : 64;   angle = 32 : 77;
33     angle = 33 : 87;  angle = 34 : 94;   angle = 35 : 98;
34    TRUE : 0;
```

block_base module

A Appendix

```
1 MODULE block_base(px, py, pz, xhx, xhy, xhz, yhx, yhy, yhz, zhx, zhy, zhz, L)
2 DEFINE
3   xhx2 := xhx;
4   xhy2 := xhy;
5   xhz2 := xhz;
6
7   yhx2 := yhx;
8   yhy2 := yhy;
9   yhz2 := yhz;
10
11   zhx2 := zhx;
12   zhy2 := zhy;
13   zhz2 := zhz;
14
15   px2 := px;
16   py2 := py;
17   pz2 := pz;
```

block_yaw module

```
1 MODULE block_yaw(px, py, pz,
2   xhx, xhy, xhz, yhx, yhy, yhz, zhx, zhy, zhz,
3   L, initYaw)
4 VAR
5   yaw : 0..35;
6   cy : cos(yaw);
7   sy : sin(yaw);
8
9 DEFINE
10  SCALE := 100;
11
12  cos_yaw := cy.val;    -- [-100..100]
13  sin_yaw := sy.val;
14
15  xhx2 := (xhx * cos_yaw - xhy * sin_yaw) / SCALE;
16  xhy2 := (xhx * sin_yaw + xhy * cos_yaw) / SCALE;
17  xhz2 := xhz;
18
19  yhx2 := (yhx * cos_yaw - yhy * sin_yaw) / SCALE;
20  yhy2 := (yhx * sin_yaw + yhy * cos_yaw) / SCALE;
21  yhz2 := yhz;
22
23  zhx2 := zhx;
24  zhy2 := zhy;
25  zhz2 := zhz;
26
27  px2 := px + (zhx2 * L) / SCALE;
28  py2 := py + (zhy2 * L) / SCALE;
29  pz2 := pz + (zhz2 * L) / SCALE;
30
31
32 ASSIGN
33   init(yaw) := initYaw;
34   next(yaw) := { yaw, (yaw+1) mod 36, (yaw+35) mod 36 };
```

block_pitch module

```
1 MODULE block_pitch(px, py, pz,
2   xhx, xhy, xhz, yhx, yhy, yhz, zhx, zhy, zhz,
```

A Appendix

```
3           L, initPitch)
4  VAR
5      pitch : 0..35;
6
7      cp : cos(pitch);
8      sp : sin(pitch);
9
10 DEFINE
11     SCALE := 100;
12
13     cos_p := cp.val;    -- [-100..100]
14     sin_p := sp.val;
15
16     xhx2 := (xhx * cos_p + zhx * sin_p) / SCALE;
17     xhy2 := (xhy * cos_p + zhy * sin_p) / SCALE;
18     xhz2 := (xhz * cos_p + zhz * sin_p) / SCALE;
19
20     yhx2 := yhx;  yhy2 := yhy;  yhz2 := yhz;
21
22     zhx2 := (-xhx * sin_p + zhx * cos_p) / SCALE;
23     zhy2 := (-xhy * sin_p + zhy * cos_p) / SCALE;
24     zhz2 := (-xhz * sin_p + zhz * cos_p) / SCALE;
25
26     px2 := px + (zhx2 * L) / SCALE;
27     py2 := py + (zhy2 * L) / SCALE;
28     pz2 := pz + (zhz2 * L) / SCALE;
29
30 ASSIGN
31     init(pitch) := initPitch;
32     next(pitch) := { pitch, (pitch+1) mod 36, (pitch+35) mod 36 };
```

main module

```
1  MODULE main
2
3  FROZENVAR
4
5      checkX : -10..10;
6      checkY : -10..10;
7      checkZ : 0..10;
8  VAR
9      block0 : block_base(base_px, base_py, base_pz,
10                          base_xhx, base_xhy, base_xhz,
11                          base_yhx, base_yhy, base_yhz,
12                          base_zhx, base_zhy, base_zhz,
13                          L1);
14      block1 : block_yaw(block0.px, block0.py, block0.pz,
15                          block0.xhx, block0.xhy, block0.xhz,
16                          block0.yhx, block0.yhy, block0.yhz,
17                          block0.zhx, block0.zhy, block0.zhz,
18                          L1, yaw1 );
19      block2 : block_pitch(block1.px2, block1.py2, block1.pz2,
20                          block1.xhx2, block1.xhy2, block1.xhz2,
21                          block1.yhx2, block1.yhy2, block1.yhz2,
22                          block1.zhx2, block1.zhy2, block1.zhz2,
23                          L2, pitch2);
24      block3 : block_pitch(block2.px2, block2.py2, block2.pz2,
25                          block2.xhx2, block2.xhy2, block2.xhz2,
26                          block2.yhx2, block2.yhy2, block2.yhz2,
27                          block2.zhx2, block2.zhy2, block2.zhz2,
```

A Appendix

```

28         L3, pitch3);
29     block4 : block_yaw(block3.px, block3.py, block3.pz,
30         block3.xhx, block3.xhy, block3.xhz,
31         block3.yhx, block3.yhy, block3.yhz,
32         block3.zhx, block3.zhy, block3.zhz,
33         L4, yaw4 );
34     block5 : block_pitch(block4.px2, block4.py2, block4.pz2,
35         block4.xhx2, block4.xhy2, block4.xhz2,
36         block4.yhx2, block4.yhy2, block4.yhz2,
37         block4.zhx2, block4.zhy2, block4.zhz2,
38         L5, pitch5);
39 DEFINE
40     L1 := 10;
41     yaw1 := 0;
42     L2 := 10;
43     pitch2 := 0;
44     L3 := 10;
45     pitch3 := 0;
46     L4 := 10;
47     yaw4 := 0;
48     L5 := 10;
49     pitch5 := 0;
50
51     base_px := 0;      base_py := 0;      base_pz := 0;
52
53     base_xhx := 100;   base_xhy := 0;      base_xhz := 0;
54     base_yhx := 0;     base_yhy := 100;    base_yhz := 0;
55     base_zhx := 0;     base_zhy := 0;      base_zhz := 100;
56
57     endX := block3.px2;
58     endY := block3.py2;
59     endZ := block3.pz2;
60
61     error := 1; -- +-1 (error=2) endX and endY coordinates error while checking
        reachability
62     endX_inside_limits := (endX <= (checkX + error) & endX >= (checkX - error));
63     endY_inside_limits := (endY <= (checkY + error) & endY >= (checkY - error));
64     endZ_inside_limits := (endZ <= (checkZ + error) & endZ >= (checkZ - error));
65
66 CTLSPEC
67     EF (endX_inside_limits & endY_inside_limits & endZ_inside_limits ) --&
        blocks_above_serface)

```


A.2 Python scripts

A.2.1 Counterexample.py class

```

1 import subprocess
2 import matplotlib.pyplot as plt
3
4
5 class Counterexample:
6     """Class to parse nusmv counterexample, retrieve trace states,
7     filter variables and represent traces as graphs"""
8
9     def __init__(self, nusmv_result: str):
10         self.nusmv_result = nusmv_result
11         self.robot_len = 5
12
13     def plot_lines(self, ax, points):
14         """
15         Plot a list of 3D points connected with lines and save the figure.
16         """
17         xs, ys, zs = zip(*points)
18         ax.plot(xs, ys, zs, marker="o", linestyle="-")
19         for i, (x, y, z) in enumerate(points):
20             ax.text(x, y, z, f"{i}", fontsize=9, color="black")
21
22     def plot_trace(self, robot_step=0, save_path="3d_plot.png"):
23         """Plot a end-effector trace from counterexample."""
24         end_coordinates = ["endX", "endY", "endZ"]
25         trace = self.filter_variables(end_coordinates)
26         fig = plt.figure()
27         ax = fig.add_subplot(111, projection="3d")
28         ax.set_xlim(-20, 20) # X axis range
29         ax.set_ylim(-20, 20) # Y axis range
30         ax.set_zlim(0, 40) # Z axis range
31         ax.set_box_aspect([1, 1, 1])
32         self.plot_lines(ax, trace)
33         robot_pos0 = self.get_robot_blocks(self.robot_len)[robot_step]
34         self.plot_lines(ax, robot_pos0)
35         ax.set_xlabel("X")
36         ax.set_ylabel("Y")
37         ax.set_zlabel("Z")
38         plt.savefig(save_path, dpi=300)
39         plt.show()
40         plt.close(fig) # Close the figure to free memory
41         print(f"3D plot saved to {save_path}")
42
43     def filter_variables(self, vars: list[str]) -> list[list[int]] | None:
44         """returns the list of steps, each step is a list of states with defined
45         variable"""
46         all_steps = [vars]
47         states_list = self.nusmv_result.split("-> State")
48         states_list = states_list[1:]
49         previous_step_values = vars[:]
50         for state in states_list:
51             step_values = previous_step_values
52             for line in state.split(sep="\n"):
53                 for i, var in enumerate(vars):
54                     if var in line:
55                         value = line.split("=")[1]

```

A Appendix

```
55         step_values[i] = value
56     all_steps.append(step_values)
57     previous_step_values = step_values[:]
58     try:
59         all_steps = [[int(variable) for variable in step] for step in all_steps
60                       [1:]]
61         return all_steps
62     except ValueError:
63         print(
64             "Error while str -> int transformation, check if all elements are int:"
65         )
66         return None
67
68 def get_robot_blocks(self, robot_size):
69     """Return robot blocks coordinates for each step."""
70     blocks = []
71     steps = []
72     # get changin by blocks by steps [[changing block0], [changing block1] ...]
73     for i in range(robot_size):
74         blocks_coordinates = [
75             f"block{i}.px2",
76             f"block{i}.py2",
77             f"block{i}.pz2",
78         ]
79         block_steps = self.filter_variables(blocks_coordinates)
80         blocks.append(block_steps)
81     # transpose the matrix (blocks to steps)
82     for i in range(len(blocks[0])):
83         step = []
84         for j in range(len(blocks)):
85             step.append(blocks[j][i])
86         steps.append(step)
87     print("Robot blocks by steps:")
88     self.print_steps(steps)
89     return steps
90
91 def get_angles(
92     self, angle_vars=[".yaw", ".pitch"], angle_descritization=10
93 ) -> list[list[int]] | None:
94     """returns the list of states with angles in degrees for each step"""
95     all_steps = self.filter_variables(angle_vars)
96     all_steps = [
97         [angle * angle_descritization for angle in step] for step in all_steps
98     ]
99     return all_steps
100
101 def print_steps(self, all_steps):
102     """Print list"""
103     for i in all_steps:
104         print(i)
105
106 def get_target_coordinates(
107     self, target_coordinates=["checkX", "checkY", "checkZ"]
108 ) -> list[list[int]] | None:
109     """returns target coordinates values"""
110     return self.filter_variables(target_coordinates)
111
112 def get_end_coordinates(
113     self, end_coordinates=["endX", "endY", "endZ"]
114 ) -> list[list[int]] | None:
115     """list of end-effector coordinates by steps"""
116     return self.filter_variables(end_coordinates)
```

A Appendix

```
116
117 @staticmethod
118 def run_nusmv_file(smv_file="./smv/robot_structure.smv"):
119     """runs smv file and if there is a counterexample store the result in file"""
120     nusmv_path = "../NuSMV-2.7.0-linux64/bin/NuSMV"
121     file = smv_file
122     output_file = "result.txt"
123     result = subprocess.run(
124         [
125             nusmv_path,
126             "-dynamic",
127             file,
128         ],
129         capture_output=True,
130         text=True,
131     )
132     stdout = result.stdout
133     if "is true" in result.stdout:
134         print("The LTL property holds.")
135     else:
136         with open(output_file, "w") as result_file:
137             print("Generating counterexample..")
138             result_file.write(stdout)
```

A.3 ROS2 nodes

A.3.1 RobotController class

```

1 from rclpy.node import Node
2 from std_msgs.msg import Float64MultiArray, Empty
3 from sensor_msgs.msg import JointState
4 from typing import List
5 import threading
6 import math
7
8
9 class RobotController(Node):
10     def __init__(self, sensorNode):
11         super().__init__("robot_controller")
12         self.sensorNode = sensorNode
13         self.create_publishers()
14         self.create_subscribers()
15         self.fixed_end = 1
16         self.joint_diff_threshold = 2 # [degrees]
17         self.joint_angles_current = [0, 0, 0, 0, 0]
18
19     def create_publishers(self):
20         """
21         publishers for attach and detach topics: robot to base, voxel to robot ends
22         """
23         self.attach_publisher1 = self.create_publisher(Empty, "/attach_end1", 10)
24         self.detach_publisher1 = self.create_publisher(Empty, "/detach_end1", 10)
25         self.attach_publisher2 = self.create_publisher(Empty, "/attach_end2", 10)
26         self.detach_publisher2 = self.create_publisher(Empty, "/detach_end2", 10)
27         self.command_publisher1 = self.create_publisher(
28             Float64MultiArray, "/position_controller1/commands", 10
29         )
30         self.command_publisher2 = self.create_publisher(
31             Float64MultiArray, "/position_controller2/commands", 10
32         )
33         self.command_publisher3 = self.create_publisher(
34             Float64MultiArray, "/position_controller3/commands", 10
35         )
36         self.command_publisher4 = self.create_publisher(
37             Float64MultiArray, "/position_controller4/commands", 10
38         )
39         self.command_publisher5 = self.create_publisher(
40             Float64MultiArray, "/position_controller5/commands", 10
41         )
42         self.command_publishers = [
43             self.command_publisher1,
44             self.command_publisher2,
45             self.command_publisher3,
46             self.command_publisher4,
47             self.command_publisher5,
48         ]
49
50     def create_subscribers(self):
51         self.joints_angles_subscriber = self.create_subscription(
52             JointState, "/joint_states", self.joint_state_changed, 10
53         )
54
55     def goto_XYZ(self, x, y, z, step_size=3):

```

A Appendix

```
56     """
57     High level function to go closer and move free end to the target coordinates x,
58         y,z
59     """
60     if x == "\n":
61         x = 0
62     if y == "\n":
63         y = 0
64     if z == "\n":
65         z = 0
66     x = float(x)
67     y = float(y)
68     z = float(z)
69     if abs(x) > step_size or abs(y) > step_size:
70         # going to a far located point
71         if x > step_size:
72             stepX = step_size
73             x = x - step_size
74         elif x < -step_size:
75             stepX = -step_size
76             x = x + step_size
77         else:
78             stepX = 0
79         if y > step_size:
80             stepY = step_size
81             y = y - step_size
82         elif y < -step_size:
83             stepY = -step_size
84             y = y + step_size
85         else:
86             stepY = 0
87         self.get_logger().info(f"\nX: {x}\n Y:{y}\n Z:{z}")
88         # Step algorithm:
89         self.goto_XYZ(stepX, stepY, 1)
90         self.goto_XYZ(stepX, stepY, 0)
91         self.swap_fix_block() # change base
92         self.goto_XYZ(x, y, z, step_size)
93     # near pose calculation
94     else:
95         command_sequences = self.calculate_joint_angles(x, y, z)
96         self.rotate_joints(command_sequences)
97
98 def calculate_joint_angles(self, x, y, z) -> List[float]:
99     command_sequences = []
100     alpha, beta_0, gamma = self.relative_angles(x, y, z)
101     if z < 4:
102         joint4 = alpha + gamma
103     elif z >= 4 and z <= 6:
104         joint4 = alpha + gamma - 90
105     else:
106         raise Exception("z is more than 6")
107     joint1 = beta_0
108     joint2 = alpha - gamma
109     joint3 = 180 - 2 * alpha
110     joint5 = beta_0
111     if self.fixed_end == 1:
112         joint1 = joint1
113         joint5 = joint5
114         command_sequences = [joint1, joint2, joint3, joint4, joint5]
115     elif self.fixed_end == 2:
116         joint1 = joint1 + 180
117         joint5 = joint5 + 180
```

A Appendix

```
117         command_sequences = [joint5, joint4, joint3, joint2, joint1]
118     else:
119         self.get_logger().info("ERROR: End blocks not fixed")
120     return command_sequences
121
122 def relative_angles(self, x, y, z):
123     """Calculate angles:
124     alpha - pitch/2 of block2 without gamma
125     beta - yaw of block1
126     gamma - correction of alpha
127     """
128     beta = math.degrees(math.atan2(y, x))
129     if z < 4:
130         r = math.sqrt(x * x + y * y)
131         r_0 = math.sqrt(abs(x * x + y * y - 1))
132     elif z >= 4 and z <= 6:
133         z = z - 2
134         r = math.sqrt(x * x + y * y)
135         r_0 = math.sqrt(abs(x * x + y * y - 1)) - 2
136
137     self.get_logger().info(f"r={r}\nr_0={r_0}")
138     if r != 0:
139         beta_0 = beta - math.degrees(math.asin(1 / r))
140     else:
141         beta_0 = 0
142     alpha = math.degrees(math.asin((math.sqrt(r_0 * r_0 + z * z)) / 4))
143     gamma = math.degrees(math.atan2(z, abs(r_0)))
144     return alpha, beta_0, gamma
145
146 def rotate_joints(self, command_sequences):
147     for joint_index in range(len(command_sequences)):
148         self.rotate_joint(joint_index, float(command_sequences[joint_index]))
149     self.wait_movement_finish(command_sequences)
150
151 def rotate_joint(self, joint_index, angle):
152     command = Float64MultiArray()
153     # Degrees to Radians
154     try:
155         command.data = [float(angle) * math.pi / 180.0]
156         self.command_publishers[joint_index].publish(command)
157     except:
158         self.get_logger().info(
159             f"Error: No data for joint {joint_index}: {command.data}"
160         )
161
162 def wait_movement_finish(self, joints_angles_target: list[float]):
163     """Wait until movement finished joint target angles in degrees"""
164     self.current_target_angles = joints_angles_target
165     move_finished = threading.Event()
166     timer_rate = 0.1
167     max_waiting_time = 5.0
168
169     def isAchieved() -> bool:
170         joints_angles_target_deg = [float(a) for a in self.current_target_angles]
171         joint_angles_current_deg = [
172             float(angle) * 180.0 / math.pi for angle in self.joint_angles_current
173         ]
174         diff = [
175             abs(a - b)
176             for a, b in zip(joints_angles_target_deg, joint_angles_current_deg)
177         ]
178         if max(diff) < self.joint_diff_threshold:
```

A Appendix

```
179         move_finished.set()
180         timer.cancel() # or: self.destroy_timer(timer)
181         return True
182     else:
183         return False
184
185     timer = self.create_timer(timer_rate, isAchieved)
186     move_finished.wait(timeout=max_waiting_time)
187     if isAchieved():
188         self.get_logger().info(f"Target achived")
189     else:
190         self.get_logger().info(f"ERROR: target point not achived \n")
191     timer.cancel() # or: self.destroy_timer(timer)
192
193     def joint_state_changed(self, msg):
194         joint_angles_current_dict = dict(zip(msg.name, msg.position))
195         sorted_names = ["rev0_1", "rev2_3", "rev5_6", "rev8_9", "rev9_10"]
196         self.joint_angles_current = [joint_angles_current_dict[i] for i in sorted_names
197                                     ]
198
199     def get_joint_angle(self, joint_num):
200         joint = float(self.joint_angles_current[joint_num]) * 180.0 / math.pi
201         return joint
202
203     def get_fixed_end(self):
204         return self.fixed_end
205
206     def swap_fix_block(self):
207         if self.get_fixed_end() == 1:
208             self.fix_end2_base()
209         elif self.get_fixed_end() == 2:
210             self.fix_end1_base()
211
212     def fix_end1_base(self):
213         msg = Empty()
214         self.attach_publisher1.publish(msg)
215         self.get_logger().info("Published attach1 message.")
216         self.fixed_end = 1
217         self.free_end2_base()
218         self.get_logger().info(f"fixed_end=1")
219
220     def fix_end2_base(self):
221         msg = Empty()
222         self.attach_publisher2.publish(msg)
223         self.get_logger().info("Published attach2 message.")
224         self.fixed_end = 2
225         self.free_end1_base()
226         self.get_logger().info(f"fixed_end=2")
227
228     def free_end1_base(self):
229         msg = Empty()
230         self.detach_publisher1.publish(msg)
231         self.get_logger().info("Published detach1 message.")
232
233     def free_end2_base(self):
234         msg = Empty()
235         self.detach_publisher2.publish(msg)
236         self.get_logger().info("Published detach2 message.")
```

A Appendix

A.3.2 PartController class

```
1 from rclpy.node import Node
2 from std_msgs.msg import Float64MultiArray, Empty
3
4 import subprocess
5 import signal
6
7
8 class PartController(Node):
9     def __init__(self, number) -> None:
10         super().__init__("part_controller")
11         self.part_number = number
12         self.create_publishers()
13         self.create_bridge()
14         self.empty_msg = Empty()
15         self.detach_part_end1() # send detach end-effectors from parts
16         self.detach_part_end2() # send detach end-effectors from parts
17
18     def create_publishers(self):
19         """
20         Create publishers to send commands to the part
21         """
22         num = self.part_number
23         # attach/detach PART to END-EFFECTORS 1 and 2
24         self.end1_attach_publisher = self.create_publisher(
25             Empty, f"/attach_end1_part{num}", 10
26         )
27         self.end2_attach_publisher = self.create_publisher(
28             Empty, f"/attach_end2_part{num}", 10
29         )
30         self.end1_detach_publisher = self.create_publisher(
31             Empty, f"/detach_end1_part{num}", 10
32         )
33         self.end2_detach_publisher = self.create_publisher(
34             Empty, f"/detach_end2_part{num}", 10
35         )
36         # attach/detach PART to BASE
37         self.base_attach_publisher = self.create_publisher(
38             Empty, f"/attach_part{num}_base", 10
39         )
40         self.base_detach_publisher = self.create_publisher(
41             Empty, f"/detach_part{num}_base", 10
42         )
43         # joint controller publisher
44         self.command_publisher = self.create_publisher(
45             Float64MultiArray, f"/position_controller{num}/commands", 10
46         )
47
48     def attach_part_end1(self):
49         """send command to a part to attach to end1"""
50         self.end1_attach_publisher.publish(self.empty_msg)
51
52     def detach_part_end1(self):
53         """send command to a part to detach from end1"""
54         self.end1_detach_publisher.publish(self.empty_msg)
55
56     def attach_part_end2(self):
57         """send command to a part to attach to end 2"""
58         self.end2_attach_publisher.publish(self.empty_msg)
59
```


A Appendix

```
60 def detach_part_end2(self):
61     """send command to a part to detach from end2"""
62     self.end2_detach_publisher.publish(self.empty_msg)
63
64 def attach_part_base(self):
65     """send command to a part to attach to end 1"""
66     self.base_attach_publisher.publish(self.empty_msg)
67
68 def detach_part_base(self):
69     """send command to a part to attach to end 2"""
70     self.base_detach_publisher.publish(self.empty_msg)
71
72 def create_bridge(self):
73     """Create ROS-Gazebo bridges to send commands"""
74     num = self.part_number
75     BRIDGE_RULES = [
76         f"/attach_end1_part{num}@std_msgs/msg/Empty@gz.msgs.Empty",
77         f"/detach_end1_part{num}@std_msgs/msg/Empty@gz.msgs.Empty",
78         f"/attach_end2_part{num}@std_msgs/msg/Empty@gz.msgs.Empty",
79         f"/detach_end2_part{num}@std_msgs/msg/Empty@gz.msgs.Empty",
80         f"/attach_part{num}_base@std_msgs/msg/Empty@gz.msgs.Empty",
81         f"/detach_part{num}_base@std_msgs/msg/Empty@gz.msgs.Empty",
82         f"/model/part{num}/pose@geometry_msgs/msg/Pose@gz.msgs.Pose",
83     ]
84     argv = ["ros2", "run", "ros_gz_bridge", "parameter_bridge"]
85     argv.extend(BRIDGE_RULES)
86
87     # Optionally remap namespaces or add ROS args:
88     # argv += ["--ros-args", "-r", "__ns:=/sim"]
89
90     self.get_logger().info(f"Starting ros_gz_bridge: {' '.join(argv)}")
91     # Start detached but manage its lifetime
92     self.proc = subprocess.Popen(
93         argv, stdout=subprocess.PIPE, stderr=subprocess.STDOUT, text=True, bufsize
94         =1
95     )
96
97 def destroy_node(self):
98     """Destroy this Node with closing bridges"""
99     try:
100         if self.proc and self.proc.poll() is None:
101             self.get_logger().info("Stopping ros_gz_bridge...")
102             self.proc.send_signal(signal.SIGINT)
103             try:
104                 self.proc.wait(timeout=5)
105             except subprocess.TimeoutExpired:
106                 self.proc.kill()
107     finally:
108         super().destroy_node()
```

A.4 Gazebo sdf,urdf and xacro code

A.4.1 base.sdf

```

1 <?xml version="1.0"?>
2 <sdf version="1.10">
3   <model name="base">
4     <static>true</static>
5     <link name="base_link">
6       <pose>0 0 0 0 0 0</pose>
7       <visual name="body_visual">
8         <geometry>
9           <mesh>
10            <uri>/srrs_sim/meshes/base10x10.stl</uri>
11            <scale> 0.001 0.001 0.001 </scale>
12          </mesh>
13        </geometry>
14        <material>
15          <ambient>0.8 0.8 0.8 1</ambient>
16          <diffuse>0.8 0.8 0.8 1</diffuse>
17        </material>
18      </visual>
19      <collision name="body_collision">
20        <geometry>
21          <box>
22            <size>2 2 0.265 </size>
23          </box>
24        </geometry>
25      </collision>
26    </link>
27    <joint name="fixed_to_world" type="fixed">
28      <parent>world</parent>
29      <child>base_link</child>
30      <pose>0 0 0 0 0 0</pose>
31    </joint>
32  </model>
33 </sdf>

```

A.4.2 robot.xacro

```

1 <?xml version="1.0" ?>
2 <robot name="rel_robot" xmlns:xacro="http://www.ros.org/wiki/xacro">
3   <xacro:property name="green" value="0 0.8 0 1" />
4   <xacro:property name="red" value="0.8 0 0 1" />
5   <xacro:property name="blue" value="0 0 0.8 1" />
6   <xacro:property name="yellow" value="1 1 0 1" />
7   <xacro:property name="white" value="1 1 1 1" />
8   <xacro:property name="cell_step" value="0.134"/>
9   <xacro:property name="grey" value="0.2 0.2 0.2 1" />
10  <xacro:property name="black" value="0 0 0 1" />
11
12  <!-- MACROS -->
13
14  <xacro:macro name="voxel" params="name origin color" >
15    <link name="${name}">
16      <visual>
17        <geometry>

```

A Appendix

```
18         <mesh filename="/home/lao/Documents/Masterarbeit/git/  
19             SRRS_gazebo_sim/ros2_ws/src/srrs_sim/meshes/Voxel.stl" scale="  
20             0.001 0.001 0.001"/>  
21     </geometry>  
22     <origin xyz="{origin}"/>  
23 </visual>  
24 <collision name="{name}_collision">  
25     <geometry>  
26         <mesh filename="/home/lao/Documents/Masterarbeit/git/  
27             SRRS_gazebo_sim/ros2_ws/src/srrs_sim/meshes/Voxel.stl" scale="  
28             0.001 0.001 0.001"/>  
29     </geometry>  
30 </collision>  
31 <inertial>  
32     <mass value="0.1"/>  
33     <inertia ixx="1.0" ixy="0.0" ixz="0.0" iyy="1.0" iyz="0.0" izz="1.0"/>  
34 </inertial>  
35 </link>  
36 <gazebo reference="{name}">  
37     <visual>  
38         <material>  
39             <ambient>{color}</ambient>  
40             <diffuse>{color}</diffuse>  
41         </material>  
42     </visual>  
43 </gazebo>  
44 </xacro:macro>  
45  
46 <xacro:macro name="joint_fix" params="name parent child" >  
47     <joint name="{name}" type="fixed">  
48         <origin xyz="0 0 0"/>  
49         <parent link="{parent}"/>  
50         <child link="{child}"/>  
51     </joint>  
52 </xacro:macro>  
53  
54 <xacro:macro name="joint_end" params="name parent child origin axis rpy" >  
55     <joint name="{name}" type="revolute">  
56         <axis xyz="{axis}"/>  
57         <origin xyz="{origin}"/>  
58         <parent link="{parent}"/>  
59         <child link="{child}"/>  
60         <limit lower="0" upper="0" effort="1000" velocity="0.0"/>  
61         <dynamics damping="0.1" friction="0.1"/>  
62     </joint>  
63 </xacro:macro>  
64  
65 <xacro:macro name="joint_revolution" params="name parent child origin axis rpy" >  
66     <joint name="{name}" type="revolute">  
67         <axis xyz="{axis}"/>  
68         <origin xyz="{origin}"/>  
69         <parent link="{parent}"/>  
70         <child link="{child}"/>  
71         <limit effort="50.0" lower="-6.29" upper="6.29" velocity="1"/>  
72         <dynamics damping="0.1" friction="0.1"/>  
73     </joint>  
74     <link name="{name}visual">  
75         <visual>  
76             <geometry>  
77                 <cylinder radius="0.01" length="0.1"/>  
78             </geometry>  
79             <origin rpy="{rpy}" xyz="{origin}"/>
```

A Appendix

```
76     </visual>
77   </link>
78   <xacro:joint_fix name="${name}visualfixed" parent="${parent}" child="${name}
79     visual"/>
80 </xacro:macro>
81
82 <!-- BLOCK MACKROS -->
83
84 <xacro:macro name="block_end" params="block_number parent_number x y z " >
85   <xacro:voxel name="block${block_number}" origin="${x*cell_step} ${y*cell_step}
86     ${z*cell_step}" color="${red}"/>
87   <xacro:joint_end name="base${parent_number}_${block_number}" parent="block${
88     parent_number}" child="block${block_number}" axis="0 0 1" origin="${x*
89     cell_step} ${y*cell_step} ${(z+1)*cell_step}" rpy="0 0 0"/>
90 </xacro:macro>
91
92 <xacro:macro name="block_fix" params="block_number parent_number x y z " >
93   <xacro:voxel name="block${block_number}" origin="${x*cell_step} ${y*cell_step}
94     ${z*cell_step}" color="${grey}"/>
95   <xacro:joint_fix name="fixed${parent_number}_${block_number}" parent="block${
96     parent_number}" child="block${block_number}"/>
97 </xacro:macro>
98
99 <xacro:macro name="block_ver" params="block_number parent_number x y z " >
100   <xacro:voxel name="block${block_number}" origin="${0} ${0} ${0}" color="${green}
101     "/>
102   <xacro:joint_revolution name="rev${parent_number}_${block_number}" parent="
103     block${parent_number}" child="block${block_number}" axis="0 0 1" origin="
104     ${x*cell_step} ${y*cell_step} ${(z+1)*cell_step}" rpy="0 0 0"/>
105 </xacro:macro>
106
107 <xacro:macro name="block_hor" params="block_number parent_number x y z " >
108   <xacro:joint_revolution name="rev${parent_number}_${block_number}" parent="
109     block${parent_number}" child="block${block_number}" axis="0 1 0" origin="
110     ${x*cell_step} ${y*cell_step} ${(z+0.5)*cell_step}" rpy="1.57 0 0"/>
111   <xacro:voxel name="block${block_number}" origin="${0} ${0} ${-0.5*cell_step}"
112     color="${blue}"/>
113 </xacro:macro>
114
115 <!-- SENSORS MACROS -->
116
117 <xacro:macro name="contact_plate" params="name parent origin" >
118   <link name="${name}">
119     <visual>
120       <geometry>
121         <cylinder radius="0.05" length="0.002"/>
122       </geometry>
123       <!-- <origin xyz="${origin}"/> -->
124     </visual>
125     <collision name="${name}_collision">
126       <geometry>
127         <cylinder radius="0.05" length="0.002"/>
128       </geometry>
129     </collision>
130   </link>
131
132   <joint name="${name}_joint" type="fixed">
133     <origin xyz="${origin}"/>
134     <parent link="${parent}"/>
135     <child link="${name}"/>
136   </joint>
```

A Appendix

```
126
127 <gazebo reference="${name}">
128   <sensor name="${name}_sensor" type="contact">
129     <contact>
130       <!-- If doesnt work - check contact collision name in SDF, transfer
131         xacro to urdf to sdf -->
132       <collision>${parent}_fixed_joint_lump__${name}
133         _collision_collision_1</collision>
134       <topic>/robot/${name}</topic>
135     </contact>
136     <always_on>true</always_on>
137     <update_rate>50</update_rate>
138   </sensor>
139 </gazebo>
140
141 </xacro:macro>
142
143 <xacro:macro name="camera_sensor" params="name parent origin_xyz origin_rpy" >
144   <link name="${name}">
145     <visual name="${name}_visual">
146       <geometry>
147         <box size="0.01 0.01 0.01"/>
148       </geometry>
149     </visual>
150   </link>
151
152   <joint name="${name}_joint" type="fixed">
153     <parent link="${parent}" />
154     <child link="${name}" />
155     <origin xyz="${origin_xyz}" rpy="${origin_rpy}" />
156   </joint>
157
158   <link name="${name}_optical"></link>
159
160   <joint name="${name}_optical_joint" type="fixed">
161     <parent link="${name}" />
162     <child link="${name}_optical" />
163     <origin xyz="0 0 0" rpy="${-pi/2} 0 ${-pi/2}" />
164   </joint>
165
166   <gazebo reference="${name}">
167     <sensor name="${name}_sensor" type="camera">
168       <visualize>true</visualize>
169       <camera>
170         <horizontal_fov>1.3962634</horizontal_fov>
171         <image>
172           <width>640</width>
173           <height>480</height>
174           <format>R8G8B8</format>
175         </image>
176         <clip>
177           <near>0.1</near>
178           <far>15</far>
179         </clip>
180         <optical_frame_id>${name}_optical</optical_frame_id>
181         <camera_info_topic>/${name}/camera_info</camera_info_topic>
182       </camera>
183       <always_on>1</always_on>
184       <update_rate>20</update_rate>
185       <visualize>true</visualize>
186       <topic>/${name}/image</topic>
```

A Appendix

```

186         </sensor>
187     </gazebo>
188 </xacro:macro>
189
190 <xacro:macro name="imu_sensor" params="name parent">
191     <gazebo reference="{parent}">
192         <sensor name="imu_sensor" type="imu">
193             <always_on>1</always_on>
194             <update_rate>50</update_rate>
195             <visualize>true</visualize>
196             <topic>/robot/{name}</topic>
197         </sensor>
198     </gazebo>
199 </xacro:macro>
200
201 <!-- MAIN STRUCTURE -->
202 <!-- For Revolute joints parent_coord_system->joint_coordinate_system->
203     child_coordinate_system -->
204 <!-- joint CS is relative to parent CS -->
205 <!-- child CS is relative to joint -->
206 <!-- for fixed joint all CS relative to world -->
207
208 <xacro:voxel name="block0" origin="0 0 0" color="{red}"/>
209
210 <xacro:block_ver block_number="1" parent_number="0" x="0" y="0" z="0" />
211 <xacro:block_fix block_number="2" parent_number="1" x="0" y="0" z="1" />
212 <xacro:block_hor block_number="3" parent_number="2" x="0" y="1" z="1" />
213 <xacro:block_fix block_number="4" parent_number="3" x="0" y="0" z="0.5" />
214 <xacro:block_fix block_number="5" parent_number="4" x="0" y="0" z="1.5" />
215 <xacro:block_hor block_number="6" parent_number="5" x="0" y="-1" z="1.5" />
216 <xacro:block_fix block_number="7" parent_number="6" x="0" y="0" z="0.5" />
217 <xacro:block_fix block_number="8" parent_number="7" x="0" y="0" z="1.5" />
218 <xacro:block_hor block_number="9" parent_number="8" x="0" y="1" z="1.5" />
219 <xacro:block_ver block_number="10" parent_number="9" x="0" y="0" z="-0.5" />
220 >
221 <xacro:block_end block_number="11" parent_number="10" x="0" y="0" z="0" />
222
223 <!-- CAMERA PART -->
224 <xacro:camera_sensor name="camera1" parent="block0" origin_xyz="0 0 0" origin_rpy="
225     0 {pi/2} 0"/>
226 <xacro:camera_sensor name="camera2" parent="block11" origin_xyz="0 0 {1*cell_step}
227     " origin_rpy="0 {-pi/2} 0"/>
228
229 <!-- CONTACT sensors -->
230 <xacro:contact_plate name="contact1" parent="block0" origin="0 0 0"/>
231 <xacro:contact_plate name="contact2" parent="block11" origin="0 0 {1*cell_step}"/>
232
233 <!-- IMU sensors -->
234 <xacro:imu_sensor name="imu1" parent="block0"/>
235 <xacro:imu_sensor name="imu2" parent="block11"/>
236

```

A Appendix

```
244
245 <!-- JOINTS CONTROL -->
246 <ros2_control name="GazeboSimSystem" type="system">
247   <hardware>
248     <plugin>gz_ros2_control/GazeboSimSystem</plugin>
249   </hardware>
250   <joint name="rev0_1">
251     <command_interface name="position"/>
252     <state_interface name="position"/>
253   </joint>
254   <joint name="rev2_3">
255     <command_interface name="position"/>
256     <state_interface name="position"/>
257   </joint>
258   <joint name="rev5_6">
259     <command_interface name="position"/>
260     <state_interface name="position"/>
261   </joint>
262   <joint name="rev8_9">
263     <command_interface name="position"/>
264     <state_interface name="position"/>
265   </joint>
266   <joint name="rev9_10">
267     <command_interface name="position"/>
268     <state_interface name="position"/>
269   </joint>
270 </ros2_control>
271
272
273
274 <gazebo>
275   <plugin filename="gz_ros2_control-system" name="
276     gz_ros2_control::GazeboSimROS2ControlPlugin">
277     <!-- <parameters>$(find srss_sim)/config/two_blocks_controller.yaml</
278       parameters> -->
279     <parameters>$(find srss_sim)/config/robot_controller.yaml</parameters>
280   </plugin>
281
282   <plugin
283     filename="libgz-sim8-detachable-joint-system.so"
284     name="gz::sim::systems::DetachableJoint">
285     <parent_link>block0</parent_link>
286     <child_model>base</child_model>
287     <child_link>base_link</child_link>
288     <attach_topic>attach_end1</attach_topic>
289     <detach_topic>detach_end1</detach_topic>
290     <output_topic>detachable_link_state1</output_topic>
291   </plugin>
292
293   <plugin
294     filename="libgz-sim8-detachable-joint-system.so"
295     name="gz::sim::systems::DetachableJoint">
296     <parent_link>block11</parent_link>
297     <child_model>base</child_model>
298     <child_link>base_link</child_link>
299     <attach_topic>attach_end2</attach_topic>
300     <detach_topic>detach_end2</detach_topic>
301     <output_topic>detachable_link_state2</output_topic>
302   </plugin>
303
304   <plugin
305     filename="gz-sim-contact-system"
```

A Appendix

```
304         name="gz::sim::systems::Contact">
305     </plugin>
306 </gazebo>
307
308 </robot>
```

A.4.3 part.xacro

```
1 <?xml version="1.0" ?>
2 <robot name="part" xmlns:xacro="http://www.ros.org/wiki/xacro">
3     <xacro:arg name="part_num" default="0"/>
4     <!-- <xacro:arg name="parent_model" default="0"/> -->
5     <!-- <xacro:arg name="parent_link" default="0"/> -->
6
7     <xacro:property name="green" value="0 0.8 0 1" />
8     <xacro:property name="red" value="0.8 0 0 1" />
9     <xacro:property name="orange" value="1 0.65 0 1" />
10    <xacro:property name="blue" value="0 0 0.8 1" />
11    <xacro:property name="yellow" value="1 1 0 1" />
12    <xacro:property name="white" value="1 1 1 1" />
13    <xacro:property name="grey" value="0.2 0.2 0.2 1" />
14    <xacro:property name="black" value="0 0 0 1" />
15
16    <link name="part">
17        <visual>
18            <geometry>
19                <mesh filename="/home/lao/Documents/Masterarbeit/git/SRRS_gazebo_sim/
20                    ros2_ws/src/srrs_sim/meshes/Voxel.stl" scale="0.001 0.001 0.001"/>
21                <!-- <box size="0.1 0.1 0.1"/> -->
22            </geometry>
23        </visual>
24        <collision name="part_collision">
25            <geometry>
26                <mesh filename="/home/lao/Documents/Masterarbeit/git/SRRS_gazebo_sim/
27                    ros2_ws/src/srrs_sim/meshes/Voxel.stl" scale="0.001 0.001 0.001"/>
28                <!-- <box size="0.1 0.1 0.1"/> -->
29            </geometry>
30        </collision>
31        <inertial>
32            <mass value="0.1"/>
33            <inertia ixx="1.0" ixy="0.0" ixz="0.0" iyy="1.0" iyz="0.0" izz="1.0"/>
34        </inertial>
35    </link>
36
37    <gazebo reference="part">
38        <visual>
39            <material>
40                <ambient>${black}</ambient>
41                <diffuse>${black}</diffuse>
42            </material>
43        </visual>
44    </gazebo>
45    <gazebo>
46        <!-- <plugin filename="gz_ros2_control-system" name="
47            gz_ros2_control::GazeboSimROS2ControlPlugin">
48            <parameters>$(find srrs_sim)/config/robot_controller.yaml</parameters>
49        </plugin> -->
50
51        <plugin
```


A Appendix

```
50     filename="libgz-sim8-detachable-joint-system.so"
51     name="gz::sim::systems::DetachableJoint">
52     <parent_link>part</parent_link>
53     <child_model>robot</child_model>
54     <child_link>block0</child_link>
55     <attach_topic>attach_end1_part${xacro.arg('part_num')}</attach_topic>
56     <detach_topic>detach_end1_part${xacro.arg('part_num')}</detach_topic>
57     <output_topic>detachable_obj_end1_state${xacro.arg('part_num')}</
58         output_topic>
59     <suppress_child_warning>true</suppress_child_warning>
60 </plugin>
61
62 <plugin
63     filename="libgz-sim8-detachable-joint-system.so"
64     name="gz::sim::systems::DetachableJoint">
65     <parent_link>part</parent_link>
66     <child_model>robot</child_model>
67     <child_link>block11</child_link>
68     <attach_topic>attach_end2_part${xacro.arg('part_num')}</attach_topic>
69     <detach_topic>detach_end2_part${xacro.arg('part_num')}</detach_topic>
70     <output_topic>detachable_obj_end2_state${xacro.arg('part_num')}</
71         output_topic>
72     <suppress_child_warning>true</suppress_child_warning>
73 </plugin>
74
75 <!-- TODO: correct plugin to connect to previous block -->
76 <plugin
77     filename="libgz-sim8-detachable-joint-system.so"
78     name="gz::sim::systems::DetachableJoint">
79     <parent_link>part</parent_link>
80     <child_model>base</child_model>
81     <child_link>base_link</child_link>
82
83     <!-- <child_model>${xacro.arg('parent_model')}</child_model> -->
84     <!-- <child_link>${xacro.arg('parent_link')}</child_link> -->
85
86     <attach_topic>attach_part${xacro.arg('part_num')}_base</attach_topic>
87     <detach_topic>detach_part${xacro.arg('part_num')}_base</detach_topic>
88     <!-->
89     <!-- <output_topic>detachable_obj_state${xacro.arg('parent_model')}</
90         output_topic> -->
91
92     <suppress_child_warning>true</suppress_child_warning>
93 </plugin>
94
95 <plugin filename="gz-sim-pose-publisher-system" name="
96     gz::sim::systems::PosePublisher">
97     <!-- <publish_model_pose>true</publish_model_pose> -->
98     <use_pose_vector_msg>false</use_pose_vector_msg>
99     <update_frequency>30</update_frequency>
100     <publish_nested_model_pose>true</publish_nested_model_pose>
101 </plugin>
102 </gazebo>
103 </robot>
```